

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
МИРЭА – РОССИЙСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ

А.Н. МИРОНОВ, Ю.А. ВОРОНЦОВ, Е.К. МИХАЙЛОВА, С.М. ТРУШИН

СИСТЕМНАЯ И ПРОГРАММНАЯ ИНЖЕНЕРИЯ

УЧЕБНОЕ ПОСОБИЕ

Москва — 2022

УДК 004.4
ББК 32.972
С 40

А.Н. Миронов Системная и программная инженерия [Электронный ресурс]: Учебное пособие / Миронов А.Н., Воронцов Ю.А., Михайлова Е.К., Трушин С.М. — М.: МИРЭА – Российский технологический университет, 2022. — 1 электрон. опт. диск (CD-ROM)

Учебное пособие разработано в качестве основного справочного материала по дисциплине «Системная и программная инженерия». В состав учебного пособия включается 15 разделов. В первых разделах изучаются вопросы инженерии требований. В них рассмотрены методики сбора требований и приводится методика систематизации требований на основе матрицы требований. Разделы с 5 по 8 посвящены проектированию архитектуры системы, а также формированию технического задания на основе собранных требований. В разделах 9 и 10 рассматриваются вопросы документирования системы. Разделы с 11 по 14 посвящены вопросам организации различных видов испытаний программных и автоматизированных систем. В завершающем разделе приведены требования и подсказки к процессу демонстрации и защиты проделанной работы.

Учебное пособие предназначено для студентов 3 курса Института информационных технологий, обучающихся по направлениям подготовки: 01.03.04 «Прикладная математика», 09.03.01 «Информатика и вычислительная техника», 09.03.03 «Прикладная информатика», 09.03.04 «Программная инженерия».

Учебное пособие издается в авторской редакции.

Авторский коллектив: Миронов Антон Николаевич, Воронцов Юрий Алексеевич, Михайлова Евгения Константиновна, Трушин Степан Михайлович.

Рецензенты:

Бочаров Никита Алексеевич, к.т.н., Начальник отдела 3.4.5 «Защита информации», ПАО «ИНЭУМ им. И.С. Брука»

Николаева Светлана Владимировна, д.т.н., доцент, профессор кафедры физики им. В.А. Фабриканта, Национальный исследовательский университет «Московский энергетический институт»

Минимальные системные требования:

Наличие операционной системы Windows, поддерживаемой производителем.

Наличие свободного места в оперативной памяти не менее 128 Мб.

Наличие свободного места в памяти хранения (на жестком диске) не менее 30 Мб.

Наличие интерфейса ввода информации.

Дополнительные программные средства: программа для чтения pdf-файлов (Adobe Reader).

Подписано к использованию по решению Редакционно-издательского совета

МИРЭА – Российского технологического университета от _____ 2022 г.

Объем ____ Мб

Тираж 10

© Миронов А.Н., Воронцов Ю.А., Михайлова Е.К.,
Трушин С.М. 2022

© МИРЭА – Российский технологический университет, 2022

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	7
1. ОБСЛЕДОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ	9
1.1. Выбор предметной области для разработки и распределение на команды	9
1.2. Задание	11
2. ОПРЕДЕЛЕНИЕ ФУНКЦИОНАЛЬНЫХ И ПОЛЬЗОВАТЕЛЬСКИХ ТРЕБОВАНИЙ	12
2.1. Формирование назначения системы	12
2.1.1. Процесс определения пользовательских требований	12
2.1.2. Плохие user story	13
2.1.3. Процесс определения функциональных требований	14
2.2. Задание	15
3. НАЧАЛЬНОЕ МОДЕЛИРОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ ПРОЕКТА	17
3.1. Портрет пользователя	17
3.2. USE case	17
3.3. Построения USE case	20
3.4. Диаграмма взаимодействия пользователя	21
3.5. Событийная диаграмма	23
3.5.1. Обозначения, используемые в нотации EPC:	23
3.6. Задание	27
4. МАТРИЦА ТРЕБОВАНИЙ	28
4.1. Формирование нефункциональных требований	28
4.1.1. Какие бывают нефункциональные требования	28
4.1.2. Формирование матрицы требований	30
4.2. Задание	33
5. СОЗДАНИЕ СТРУКТУРНОЙ ДИАГРАММЫ СИСТЕМЫ	34
5.1. Структурная диаграмма	34
5.1.1. Диаграмма классов	34
5.1.2. Диаграмма объектов	35
5.1.3. Порядок построения структурной диаграммы	35
5.1.4. Взаимосвязи в структурных диаграммах	36
5.1.5. Различия между взаимосвязями: композиция и агрегация	37
5.2. Построение процессов проекта в нотациях IDEF0, BPMN 2.0, EPC	37
5.2.1. Нотация IDEF0	38
5.2.2. Нотация BPMN	40
5.3. Задание	41

6. СОЗДАНИЕ ИНФОРМАЦИОННОЙ ДИАГРАММЫ СИСТЕМЫ И ЛОГИЧЕСКОЙ МОДЕЛИ БАЗЫ ДАННЫХ.....	42
6.1. Диаграмма в нотации DFD	42
6.1.1. Различия внутри DFD	42
6.1.2. Элементы нотации DFD	43
6.1.3. Порядок построения диаграммы потоков данных.....	44
6.2. Модель базы данных	46
6.2.1. Типы связей в модели базы данных	46
6.2.2. Реляционная модель базы данных.....	47
6.2.3. Различия между логической и физической моделью БД.....	48
6.3. Задание	49
7. АРХИТЕКТУРА СИСТЕМЫ	50
7.1. Критерии хорошей архитектуры.....	50
7.2. Монолитная архитектура.....	52
7.2.1. Плюсы монолитной архитектуры.....	53
7.2.2. Минусы монолитной архитектуры.....	54
7.3. Трехуровневая архитектура.....	54
7.3.1. Уровень представления (Клиент)	55
7.3.2. Уровень приложений (Сервер приложений)	55
7.3.3. Уровень данных (Сервер БД).....	55
7.3.4. Слои и уровни	55
7.3.5. Преимущества трехуровневой архитектуры	56
7.3.6. Недостатки трехуровневой архитектуры.....	56
7.4. Сервис-ориентированная архитектура (SOA)	57
7.4.1. Плюсы SOA	58
7.4.2. Минусы SOA	58
7.5. Микросервисная архитектура	58
7.5.1. Плюсы микросервисной архитектуры	60
7.5.2. Минусы микросервисной архитектуры	60
7.6. Концептуальная архитектура (для игр).....	61
7.7. Дополнение матрицы требований.....	63
7.8. Задание	67
8. ФОРМИРОВАНИЕ ТЕХНИЧЕСКОГО ЗАДАНИЯ	68
8.1. ГОСТ 34.602-2020	68
8.1.1. Наполнение разделов в ГОСТ 34.602-2020	68
8.2. ГОСТ 19.201-78	77
8.3. Задание	79

9. ОРГАНИЗАЦИЯ РАЗРАБОТКИ.....	80
9.1. Описание выбора методологии управления процессом разработки	80
9.2. Waterfall (каскадная модель)	80
9.3. Итерационная, спиральная и инкрементная модели.....	81
9.3.1. Спиральная модель	82
9.3.2. Инкрементная модели	83
9.4. Agile и Lean: принципы разработки ПО.....	84
9.4.1. Agile.....	84
9.4.2. Lean.....	84
9.5. Основные методы разработки ПО: гибкие методологии	85
9.6. Организация процесса разработки.....	87
9.7. Задание	87
10. ДОКУМЕНТИРОВАНИЕ РАЗРАБОТКИ ПО	88
10.1. Документация разработчика	88
10.1.1. Doxygen	88
10.1.2. Sphinx	89
10.2. Документация пользователя.....	89
10.2.1. FANDOM	90
10.2.2. Mediawiki	90
10.3. Задание	91
11. ПОДГОТОВКА К ТЕСТИРОВАНИЮ ПРОДУКТА.....	92
11.1. Тестирование программного продукта	92
11.2. Этапы тестирования	92
11.3. Типы тестирования.....	93
11.4. Тестовые случаи	93
11.4.1. Виды тестовых случаев	94
11.4.2. Детализация описания тест кейсов (Test Case Specification)	94
11.5. Дополнение матрицы требований.....	95
11.6. Задание	96
12. ФОРМИРОВАНИЕ ПРОГРАММЫ И МЕТОДИКИ ИСПЫТАНИЙ СИСТЕМЫ	97
12.1. Тест кейс.....	97
12.1.1. Атрибуты тест кейса	97
12.1.2. Типичные ошибки при написании тест кейсов.....	98
12.2. Программа и методика испытаний	99
12.3. Задание	101

13. АНАЛИЗ И ОЦЕНКА РИСКОВ ПРИ РАЗРАБОТКЕ ПРОГРАММНОГО ПРОЕКТА	102
13.1. Риски проекта	102
13.2. Планирование способов реагирования.....	103
13.3. Виды и источники рисков.....	103
13.3.1. Самые распространенные риски.....	104
13.4. Риски семестрового проекта.....	108
13.5. Задание	111
14. АНАЛИЗ ЭКСПЛУАТАЦИИ ПРОЕКТА	113
14.1. Disaster Recovery.....	113
14.2. Disaster Recovery Plan	113
14.3. Риск = Угроза + Уязвимость + Актив.....	118
14.3.1. Технологические угрозы	119
14.3.2. Реагирование на риски	119
14.4. Задание	120
15. ЗАЩИТА ПРОЕКТА	122
15.1. Провальный проект	125
СПИСОК ИСТОЧНИКОВ И ЛИТЕРАТУРЫ.....	127

ВВЕДЕНИЕ

Системная инженерия – междисциплинарный подход, определяющий полный набор технических и управленческих усилий, необходимых для преобразования совокупности потребностей клиента, ожиданий и ограничений в решения и для поддержки этих решений на протяжении их жизни.

Программная инженерия – приложение систематического, дисциплинированного, измеримого подхода к разработке, функционированию и сопровождению программного обеспечения, а также исследованию этих подходов; то есть, приложение дисциплины инженерии к программному обеспечению.

Курс «Системная и программная инженерия» посвящен изучению процесса реализации программного обеспечения на всех стадиях его жизненного цикла, начиная от первичного обследования предметной области, заканчивая сопровождением, на примере собственного проекта. Ключевым моментом курса является формирование у студента навыков работы с требованиями на всех этапах жизненного цикла создаваемого программного обеспечения.

В данном учебном пособии содержится материал, предназначенный для самостоятельного освоения студентом данной дисциплины. Материал разбит на 15 разделов, отражающих отдельные аспекты работы с требованиями на каждом из этапов жизненного цикла программной системы: возникновение потребности в программной системе; формирование, уточнение и формализация требований в виде технического задания; описание эскизного и технического проекта и особенности организации прототипирования программных систем; проведение работ по тестированию и верификации полученного образца системы; проведение приемо-сдаточных испытаний системы; обеспечение эксплуатации разработанной системы.

Первые 4 раздела посвящены вопросам инженерии требований. В них рассмотрены методики сбора требований и приводится методика систематизации требований на основе матрицы требований.

Разделы с 5 по 8 посвящены вопросам проектирования архитектуры системы на основе собранных требований. В них рассматриваются различные типы архитектур, а также методы декомпозиции требований по компонентам исходя из потребности пользователя. Данный раздел заканчивается форматированием технического задания на автоматизированную систему по стандарту ГОСТ 34.602-2020 либо на приложение по стандарту ГОСТ 19.201-79

Разделы 9 и 10 посвящены вопросам документирования системы. В них рассмотрены методы формирования пользовательской и технической документации как в ручном режиме, так и с использованием специализированных

программных средств.

Разделы с 11 по 14 посвящены вопросам организации различных видов испытаний программных и автоматизированных систем. В данном разделе рассмотрены различные методики верификации и валидации программного обеспечения, оформления документации на процесс тестирования в виде Программы и методики испытаний. Также в данных разделах рассмотрены вопросы выявления и оценки рисков, связанных с эксплуатацией автоматизированной системы и организации ее эксплуатации с учетом непрерывности работы и доступности.

В данном учебном пособии содержатся теоретические сведения и практические задания для закрепления теоретического материала. Учебное пособие разработано для бакалавров направлений 01.03.04 «Прикладная математика», 09.03.01 «Информатика и вычислительная техника», 09.03.03 «Прикладная информатика», 09.03.04 «Программная инженерия».

1. ОБСЛЕДОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Выбор предметной области для разработки и распределение на команды

Командная работа подразумевает распределение обязанностей и ролей, каждая из которых включает в себя определенный набор задач. Один человек либо не может реализовывать крупный проект, либо реализация займет достаточно продолжительный период, в связи с большой нагрузкой. Для более грамотного распределения на команды, обозначим роли и функции, которые они в себя включают.

Возможные роли в команде:

- 1) руководитель группы/менеджер проекта;
- 2) разработчик;
- 3) тестировщик;
- 4) аналитик;
- 5) технический писатель;
- 6) дизайнер.

Руководитель группы (менеджер проекта) – промежуточное звено между командой и клиентом. Переводит пожелания клиента на язык программистов и наоборот. Координирует скорость и качество выполнения работы.

Менеджер должен отслеживать весь процесс разработки, решать возникающие проблемы в процессе разработки, разногласия, возникающие на всех этапах жизненного цикла.

В обязанности менеджера проекта также входит формирование списка задач в процессе реализации проекта, выставление сроков поставленных задач, а также отслеживание соблюдения данных сроков и их перенос при невозможности этого. Также менеджер проекта должен фиксировать причины возникающих переносов для формирования отчетности о проделанной работе.

В качестве трекеров для ведения задач можно использовать различные технологии на усмотрение участников команды, например, Trello. Требования к инструменту заключаются в следующем:

- возможность распределения задач по участникам команды;
- возможность выставления сроков на выполняемые задачи;
- возможность распределения задач между областями реализации проекта;
- возможность просмотра истории выполнения задач.

***Примечание:** Менеджер проекта должен быть в каждой команде обязательно.*

Роль разработчика подразделяется на Back-end разработчика, Front-end разработчика, мобильного разработчика.

Back-end разработчик – Отвечает за серверную часть проекта. Как правило, в студийных командах разработки является старшим программистом на проекте.

Front-end разработчик – Отвечает за внешнюю часть проекта: верстку, клиентское программирование, оптимизацию производительности.

Мобильный разработчик – Разработка под мобильные платформы (iOS, Android, Windows Phone), если проект подразумевает наличие мобильной части.

В рамках курса все данные роли обобщаются одним названием – разработчик.

***Примечание:** Роль разработчика не сочетается с ролью тестировщика, поскольку разработчик не может тестировать свой код (кроме Unit-тестов).*

Тестировщик – подключается по завершению этапа разработки, включающей в себя законченный набор функций, по завершению процесса разработки, и перед запуском релиза на широкую аудиторию.

Тестировщик должен принимать активное участие, тестируя завершенные модули на правильность их работы, таким образом снижая риски получить ошибки на более поздних этапах разработки.

***Примечание:** Роль тестировщика не сочетается с ролью разработчика, но сочетается с аналитиком.*

Аналитик – занимается описанием протекающих в выбранной предметной области процессов при помощи оговоренных аннотаций. Создает диаграммы информационных процессов, содержащие их участников и циркулирующую в процессах информацию.

Чем грамотнее аналитик опишет протекающие процессы, тем грамотнее будет реализована бизнес-логика приложения, с учетом всех возникающих в процессах нюансов и ограничений.

Технический писатель – занимается сбором требований, формированием технического задания, а также всей сопутствующей документацией к нему.

Технический писатель одним из первых вступает в процесс реализации программного обеспечения. На основании результатов его работы будет строиться дальнейший план разработки проекта.

Дизайнер – занимается созданием фирменного стиля, проектированием и отрисовкой интерфейсов на основе требований из основного ТЗ проекта.

Дизайнер должен принимать участие в формировании требований к интерфейсной части приложения. При этом в качестве требований могут использоваться макеты будущего интерфейса.

1.2. Задание

1. Разбиться на команды по 3-4 человека.
2. Распределить роли в команде.
3. Сформированной командой обсудите и выберите тему проекта, опираясь на тематики проектов:
 - веб-приложения;
 - мобильные приложения;
 - корпоративные информационные системы;
 - интернет Вещей;
 - компьютерные игры;
 - анализ данных;
 - своя тематика.

***Примечание:** обратите внимание, что это не темы проекта, а тематики. Темы проекта должны звучать следующим образом: Мобильное приложение «планировщик задач».*

2. ОПРЕДЕЛЕНИЕ ФУНКЦИОНАЛЬНЫХ И ПОЛЬЗОВАТЕЛЬСКИХ ТРЕБОВАНИЙ

2.1. Формирование назначения системы

Любая система или приложение должна иметь четкое **назначение**, которое обуславливает причину разработки, в противном случае, если назначение невозможно явно определить, реализация не имеет смысла, в связи с ненужностью приложения. Например, назначение системы или приложения может заключаться в автоматизации бизнес-процессов компании: процесса учета персонала, процесса работы с клиентами, процесса управления заказами. Или же цифровизация процесса продажи товаров или услуг компании/индивидуального предпринимателя: интернет-магазин, площадка для размещения вакансий работодателей/соискателей. Приложения или система могут ставить своим назначением упрощение и автоматизацию повседневных задач пользователей, как профессиональных (моделирование бизнес-процессов, разработка ПО и т.д.), так и бытовых (составление списка дел, управление умным домом и т.д.). Либо же, если говорить про GameDev, назначением может служить организация досуга пользователя, перевод игры в цифровой формат для более простого использования и т.д. В целом назначение зависит от типа системы/приложения и предметной области, в которой она/оно реализуется.

Назначение подкрепляется основными задачами, решаемыми предлагаемой автоматизированной системой/приложением. В качестве задач, решаемых системой/приложением, можно рассмотреть набор её/его функциональных возможностей, определяемый пользовательскими требованиями. Для более удобного формирования пользовательских требований удобнее всего использовать так называемые **пользовательские истории**.

2.1.1. Процесс определения пользовательских требований

Пользовательская история — это описание функциональной возможности ПО простыми, общими словами, составленное с точки зрения конечного пользователя. Она пишется с целью разъяснить, как именно функциональная возможность принесет пользу клиенту.

Пользовательские истории можно записывать в форме предложения, содержащего ответы на 3 вопроса. Кто пользователь? Какую задачу с помощью интерфейса он хочет решить? С какой целью?

1. Как пассажир я хочу зарегистрироваться на рейс, чтобы можно было сесть в самолёт.

2. Как редактор блога я хочу самостоятельно публиковать статьи, чтобы выкладывать тексты в день получения от копирайтера, не обращаясь за помощью к верстальщику.
3. Как посетитель интернет-магазина я хочу сравнивать товары друг с другом, чтобы выбрать подходящий.

Такие предложения можно записывать как в форме простого списка-перечня, так и в более структурированном виде. Структурированный вид представлен в табл. 2.1.

Таблица 2.1. Перечень user story

Кто?	Что хочет?	С какой целью?
Пассажир	Зарегистрироваться на рейс	Иметь возможность сесть в самолёт
Редактор блога	Самостоятельно публиковать статьи	Выкладывать тексты в день получения от копирайтера, не обращаясь за помощью к верстальщику
Посетитель	Сравнивать товары друг с другом	Иметь возможность выбора подходящего товара

2.1.2. Плохие user story

Как product owner, я хочу, чтобы в системе была возможность удалять объявления, чтобы юзеры могли удалять объявления.

Почему данная user story плохая? Вроде и компоненты все на месте, но все равно что-то не так. Данная история написана автором просто лишь бы написать user story. Подобное написание трактует взгляд с точки зрения product owner'а, а не с точки зрения нужд конечного пользователя, следовательно, итоговая система не будет соответствовать его потребностям.

Как developer, я хочу заменить виджет папок, чтобы у меня был лучший виджет папок.

Данная user story не представляют никакой ценности для пользователя и достаточно тяжело будет уговорить product owner'а принять ее. Такую историю лучше переписать с пользовательской точки зрения: «Как рекламодатель, я хочу, чтобы система позволяла создавать мне папки, чтобы я мог быстрее работать с большими списками объявлений»

Технические задачи к этой истории могут выглядеть так:

1. «Отрефакторить механизм добавления папок, чтобы позволять создавать вложенные папки вплоть до 3 уровня вложенности».

2. «Проапдейтить версию SDK для использования нового механизма локального хранения данных на устройстве».

При этом к такой истории гораздо проще написать критерии приемки:

3. «Пользователь может создавать папки с тремя уровнями вложенности».
4. «Пользователь не может создать больше 100 папок».

Такая запись позволяет более наглядно оценить реализуемый функционал, а также упростить формирование **функциональных требований** на основании пользовательских требований.

2.1.3. Процесс определения функциональных требований

Функциональные требования — это постановка задачи разработчику. Все, что не указано в требованиях, делается на усмотрение разработчика, что часто расходится с представлением product-менеджера об ожидаемом результате. Поэтому требования должны содержать ответы на все возможные вопросы по задаче.

Функциональные требования пишутся на основе пользовательских требований (пользовательских историй). Опишите как интерфейс взаимодействует с пользователем, что происходит в случае ошибки или успешного выполнения задачи. Это как написать инструкцию для интерфейса — какие элементы создать, чтобы пользователь достиг цели.

Функциональные требования удобно писать в форме утверждений со словами «должен» и «должна».

1. У пассажира должна быть возможность распечатать посадочные талоны на все рейсы, на которые он зарегистрировался.
2. Если в профиле пассажира не указаны предпочтения по выбору места, система резервирования должна сама назначить ему место.
3. У редактора блога должна быть возможность добавить новую статью через администраторскую панель блога.
4. Редактор должен форматировать текст при помощи панели инструментов.
5. У зарегистрированного покупателя должна быть возможность добавлять неограниченное количество товаров в избранное.
6. Если пользователь положил товары в избранное, он должен иметь возможность сравнивать их друг с другом.

Как уже было сказано выше, функциональные требования можно также представить рядом с user story в табличном виде (табл. 2.2.) для более наглядного соответствия.

Таблица 2.2. Перечень функциональных требований

Кто?	Что хочет?	С какой целью?	Функциональное требование
Пассажир	Зарегистрироваться на рейс	Иметь возможность сесть в самолёт	Печать посадочных талонов для всех рейсов, на которые зарегистрирован пассажир. Регистрация на рейс с учетом выбора конкретного места. Регистрация на рейс с резервированием места на основе указанных предпочтений пассажира. Регистрация на рейс с резервированием на основе автоматического выбора системы.
Редактор блога	Самостоятельно публиковать статьи	Выкладывать тексты в день получения от копирайтера, не обращаясь за помощью к верстальщику	Добавление новой статьи в блог через администраторскую панель. Форматирование текста статьи при помощи панели инструментов.
Посетитель	Сравнивать товары друг с другом	Иметь возможность выбора подходящего товара	Добавление неограниченного количества товаров в избранное для зарегистрированного покупателя. Сравнение добавленных в избранное товаров.

2.2. Задание

1. Определить назначение системы, основываясь на выбранной предметной области и тематике.
2. Написать User story (не менее 10) покрывающие весь функционал реализуемой системы/приложения.
3. На основе составленных User story опишите функциональные требования.

Примечание: User Story и функциональные требования могут быть описаны любым из предлагаемых вариантов. Зафиксировать их необходимо также в любом формате, например, при помощи документа в формате .docx, Google Doc, или же при помощи баг-трекеров или функционала любых git-сервисов - GitHub/GitHub Issues.

4. Руководитель проекта должен составить первоначальный план реализации проекта и распределить задачи между участниками в сервисе Trello/Jira/Miro или же им подобным.

3. НАЧАЛЬНОЕ МОДЕЛИРОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ ПРОЕКТА

3.1. Портрет пользователя

Портрет пользователя — это описание вымышленной личности, которое подробно характеризует идеального клиента. При построении портрета пользователя обычно включают следующие характеристики: возраст, пол, место проживания, семейное положение, количество детей, сфера занятости и уровень зарплаты, должность, связанные с ней проблемы, потребности, желания, фобии и т.д.

Портрет пользователя обычно составляют по характеристикам личности реального клиента и / или на основе обширных исследований.

Пример. Покупатель дорогого смартфона в интернет-магазине.

Олег, 27 лет.

Место проживания: Санкт-Петербург.

Семейное положение, количество детей: не женат.

Сфера занятости и уровень зарплаты: малый бизнес, 170 тысяч рублей.

Должность, связанные с ней проблемы: руководитель собственной компании по производству бюджетной мебели.

Потребности, желания, фобии: для переговоров с поставщиками и клиентами Олегу необходимо иметь статусный смартфон. Соответственно, он рассматривает только флагманы ведущих торговых марок. Готов приобретать дорогие аксессуары из натуральных материалов: кожи, полудрагоценных камней.

3.2. USE case

Цель разработки use-case диаграммы — формирование требований к системе, причем именно в таком виде, чтобы их мог легко понять заказчик.

Основные элементы диаграммы - участник (actor) и прецедент (вариант).

Участник — это множество логически связанных ролей, исполняемых при взаимодействии с прецедентами или сущностями (система, подсистема или класс). Участником может быть человек или другая система, подсистема или класс, которые представляют нечто вне сущности. Графически участник изображается “человечком”.

Прецедент (use case) - описание множества последовательных событий (включая варианты), выполняемых системой, которые приводят к наблюдаемому участником результату. Прецедент представляет поведение сущности, описывая взаимодействие между участниками и системой. Прецедент не показывает, “как” достигается некоторый результат, а только “что” именно выполняется.

Прецеденты обозначаются очень простым образом - в виде эллипса, внутри которого указано его название.

Диаграмму вариантов использования есть смысл строить во время *изучения технического задания*, она состоит из графической диаграммы, описывающей *действующие лица и прецеденты*, а также спецификации, представляющего собой текстовое описание конкретных *последовательностей действий (потока событий)*, которые выполняет пользователь при работе с системой. Спецификация затем станет основой для тестирования и документации, а на следующих этапах проектирования она дополняется и оформляется в виде диаграммы. Кроме того, use-case диаграмма достаточно проста, чтобы ее мог понять заказчик, следовательно вы можете использовать ее для согласования ТЗ (ведь диаграмма описывает функциональные требования к системе).

На диаграмме использования изображаются:

- акторы — группы лиц или систем, взаимодействующих с нашей системой;
- варианты использования (прецеденты) — сервисы, которые наша система предоставляет акторам;
- комментарии;
- отношения между элементами диаграммы.

На рис. 3.1 отображены основные элементы, используемые в USE case нотации:

Элемент/ Нотация	Предназначение
	Участник (Actor)
	Вариант (Use case)
	Граница (Boundary)
	Ненаправленная ассоциация (Undirected communication association)
	Направленная ассоциация (Directed communication association)
	Обобщение (Generalization)
	Зависимость (Dependency)
	Точка изгиба связей (Point)
	Комментарий (Note)
	Коннектор комментария (Note connector)

Рисунок 3.1. Элементы нотации USE CASE

Порядок построения диаграммы:

- 1) выделить группы действующих лиц (работающих с системой по-разному, часто из-за различных прав доступа);
- 2) идентифицировать как можно больше вариантов использования (процессов, которые могут выполнять пользователи). При этом не следует делить процессы слишком мелко, нужно выбирать лишь те, которые дадут пользователю значимый результат. Например, кассир может «продать товар» (это будет являться прецедентом), однако «ввод штрих-кода товара для получения цены» самостоятельным прецедентом не является;
- 3) дополнить прецеденты словесным описанием (сценарием):
 - 3.1) для каждого прецедента создать разделы: «главная последовательность» и «альтернативные последовательности»;
 - 3.2) при составлении сценария нужно упорно задавать заказчику вопросы «что происходит?», «что дальше?», «что еще может

происходить?» и записывать ответы на них.

3.3. Построения USE case

Рассмотрим пример системы для развития у детей математических навыков и построим для него use case диаграмму.

В данном случае для учеников функциональностью является: *выбор подготовленного учителем блока заданий, выполнение заданий*. В то время, как для учителя функциональностью является: *подготовка для учеников блоков заданий, добавление в систему ученика, просмотр отчетов*.

Как же должна вести себя данная система?

При первом запуске система должна позволять ввести пароль учителя. Задания представляют собой математические задачи на сложение, вычитание, умножение и деление. В блоке задач могут быть задачи различных типов (указывается количество). Помимо ввода типа выполняемой в примере операции необходимо указывать допустимые диапазоны чисел (или даже отдельные числа, т.к. при изучении таблицы умножения часто сначала учат умножение на 2, затем на 5, а только потом все остальное). Кроме того, для операции вычитания необходимо иметь возможность установить вычитаемое меньше уменьшаемого (т.к. в противном случае результат будет отрицательным, а отрицательные числа в школе проходят гораздо позже). На рис. 3.2 показано решение данной задачи:



Рисунок 3.2. Пример создания USE case диаграммы

3.4. Диаграмма взаимодействия пользователя

Диаграмма взаимодействия предназначена для моделирования отношений между объектами (ролями, классами, компонентами) системы в рамках одного прецедента.

Данный вид диаграмм отражает следующие аспекты проектируемой системы:

- 1) обмен сообщениями между объектами (в том числе в рамках обмена сообщениями со сторонними системами);
- 2) ограничения, накладываемые на взаимодействие объектов;
- 3) события, инициирующие взаимодействия объектов.

Диаграмма последовательности является одной из разновидностей диаграмм взаимодействия и предназначена для моделирования взаимодействия объектов Системы во времени, а также обмена сообщениями между ними.

Диаграмма последовательности объединяет диаграмму деятельности, диаграмму состояний и диаграмму классов.

Таким образом, на диаграмме последовательности мы можем увидеть следующие аспекты:

- 1) сообщения, побуждающие объект к действию;
- 2) действия, которые вызываются сообщениями (методы) – зачастую это передача сообщения следующему объекту или возвращение определенных данных объекта;
- 3) последовательность обмена сообщениями между объектами.

Диаграммы последовательности предназначены для моделирования взаимодействия между несколькими объектами. Зачастую диаграммы последовательности создаются для моделирования взаимодействия в рамках одного прецедента.

На диаграмме последовательности объекты в основном представляют экземпляры класса или сущности, обладающие поведением. В качестве объектов могут выступать пользователи, инициирующие взаимодействие, классы, обладающие поведением в системе или программные компоненты, а иногда и системы в целом.

Объекты располагаются слева направо таким образом, чтобы крайним слева был тот объект, который инициирует взаимодействие.

Неотъемлемой частью объекта на диаграмме последовательности является линия жизни объекта. Линия жизни показывает время, в течение которого объект существует в системе. Периоды активности объекта в момент взаимодействия

показываются с помощью фокуса управления. Временная шкала на диаграмме направлена сверху вниз.

В простом варианте на диаграмме последовательности присутствуют несколько видов обозначений:

1. **Объект** - обозначается прямоугольником с наименованием объекта.
2. **Сообщения** - линии со стрелками.
3. **Возвращаемые результаты** - пунктирные линии со стрелками.

Прямоугольники на вертикальных линиях под каждым из объектов показывают “**время жизни**” (фокус) объектов. (рис. 3.3)

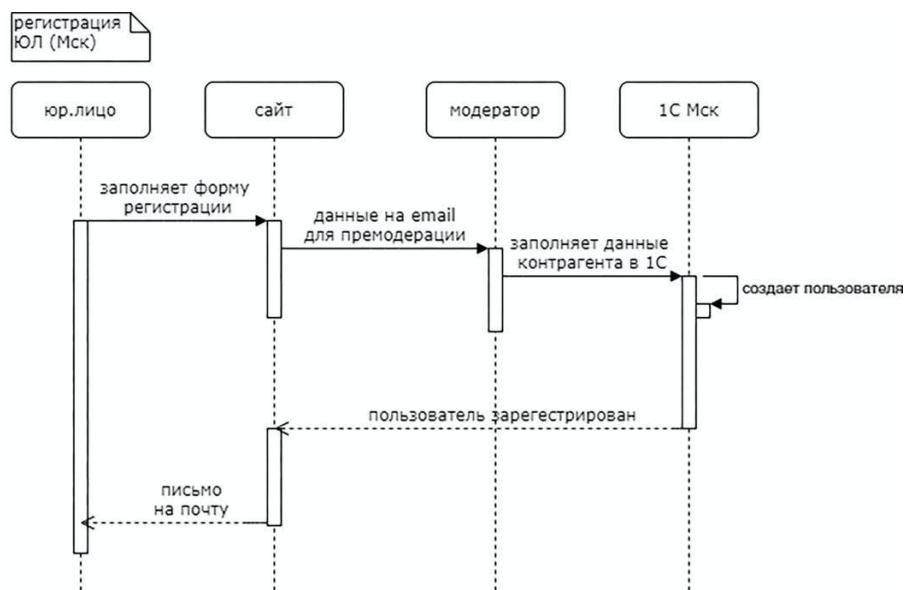


Рисунок 3.3. Пример диаграммы последовательности

В усложненном варианте (рис. 3.4) объекты могут подразделяться на несколько типов:

1. **Actor** – экземпляр участника процесса (роль на диаграмме прецедентов).
2. **Boundary** – Класс-Разграничитель – используется для классов, отделяющих внутреннюю структуру системы от внешней среды (экранная форма, пользовательский интерфейс, устройство ввода-вывода). Объект boundary показывает именно экранную форму, которая принимает и передает данные обработчику.
3. **Control** – Класс-контроллер - активный элемент, который используется для выполнения некоторых операций над объектами (программный компонент, модуль, обработчик).

4. **Entity** – Класс-сущность - обычно применяется для обозначения классов, которые хранят некую информацию о бизнес-объектах (соответствует таблице или элементу БД).

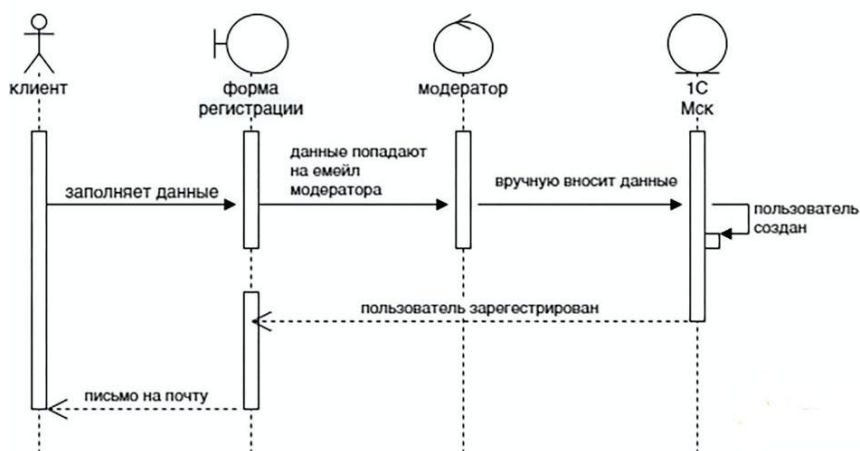


Рисунок 3.4. Пример усложненной диаграммы последовательности

Примечание: если тема проекта связана с разработкой игры, то вместо диаграммы последовательности необходимо построить событийную диаграмму.

3.5. Событийная диаграмма

Нотация EPC (Event-Driven Process Chain – событийная цепочка процессов) используется для описания процессов нижнего уровня. Диаграмма процесса в нотации EPC, представляет собой упорядоченную комбинацию событий и функций. Для каждой функции могут быть определены начальные и конечные события, участники, исполнители, материальные и документальные потоки, сопровождающие её, а также проведена декомпозиция на более низкие уровни. Декомпозиция может производиться в нотациях EPC или BPMN.

3.5.1. Обозначения, используемые в нотации EPC:

Блок (рис. 3.5) представляет собой функцию - действие или набор действий, выполняемых над исходным объектом (документом, ТМЦ и прочим) с целью получения заданного результата.

Внутри блока помещается наименование функции.

Временная последовательность выполнения функций задается расположением функций на диаграмме процесса сверху вниз.



Рисунок 3.5. Блок функции

Событие (рис. 3.6) – состояние, которое является существенным для целей управления бизнесом и оказывает влияние или контролирует дальнейшее развитие одного или более бизнес-процессов. Элемент отображает события, активизирующие функции или порождаемые функциями. Внутри блока помещается наименование события.

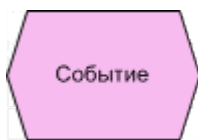


Рисунок 3.6. Блок функции

Стрелка (рис. 3.7) отображает связи элементов диаграммы процесса ЕРС между собой. Связь может быть направленной и ненаправленной в зависимости от соединяемых элементов и типа связи.



Рисунок 3.7. Стрелка связи элементов диаграммы

Элемент, изображенный на рис. 3.8, обозначает внешний (по отношению к текущей диаграмме) процесс или функцию. Используется для указания взаимосвязи процессов:

- 1) обозначает предыдущий или следующий процесс по отношению к диаграмме рассматриваемого процесса;
 - 2) обозначает процесс, откуда поступил или куда передается объект.
- Внутри блока помещается наименование внешнего процесса.

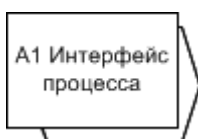


Рисунок 3.8. Элемент, обозначающий внешний процесс или функцию

На рис. 3.9 используется элемент для отображения на диаграмме организационных единиц (должности, подразделения, роли, внешнего субъекта) – исполнителей, владельцев или участников функций. Внутри блока помещается наименование организационной единицы.



Рисунок 3.9. Элемент, отображающий организационные единицы

На рис. 3.10 используется элемент для отображения на диаграмме бумажных документов, сопровождающих выполнение функции. Внутри блока помещается наименование бумажного документа.

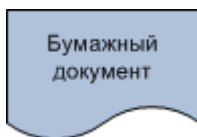


Рисунок 3.10. Элемент, отображающий бумажные документы

На рис. 3.11 используется элемент для отображения на диаграмме электронных документов, сопровождающих выполнение функции. Внутри блока помещается наименование электронного документа.



Рисунок 3.11. Элемент, отображающий электронные документы

Элемент на рис. 3.12 используется для отображения на диаграмме информационных потоков, сопровождающих выполнение функции. Внутри блока помещается наименование информационного потока.

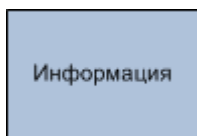


Рисунок 3.12. Элемент, отображающий информационные потоки

Элемент на рис. 3.13 используется для отображения на диаграмме информационной системы, поддерживающей выполнение функции. Внутри блока помещается наименование информационной системы.



Рисунок 3.13. Элемент, отображающий информационные системы

Элемент на рис. 3.14 используется для отображения на диаграмме модуля информационной системы, поддерживающего выполнение функции. Внутри блока помещается наименование модуля информационной системы.

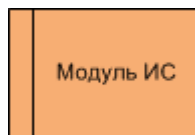


Рисунок 3.14. Элемент, отображающий модули информационной системы

Элемент на рис. 3.15 используется для отображения на диаграмме функции информационной системы, поддерживающей выполнение функции. Внутри блока помещается наименование функции информационной системы.

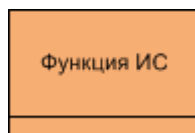


Рисунок 3.15. Элемент, отображающий функции информационной системы

На рис. 3.16 используется элемент для отображения на диаграмме базы данных, сопровождающей выполнение функции. Внутри блока помещается наименование базы данных.

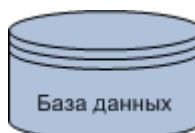


Рисунок 3.16. Элемент, отображающий базы данных

На рис. 3.17 используется для отображения на диаграмме объектов, сопровождающих выполнение функции. Наименования этих объектов - термины, используемые в организации. Внутри блока помещается наименование термина. Элемент может быть использован для обозначения данных, передаваемых между процессами или обрабатываемых при выполнении процессов. Элемент может быть также использован для обозначения статусов бумажных/электронных документов и других элементов справочника "Объекты деятельности".

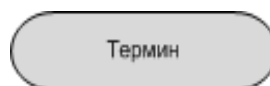


Рисунок 3.17. Элемент, сопровождающий выполнение функций

На рис. 3.18 представлен пример построения в нотации EPC:



Рисунок 3.18. Пример построения EPC

3.6. Задание

1. Определить портрет пользователя и виды пользователей (целевая аудитория).
2. Построить USE CASE диаграмму, в которой указаны основные требования к системе.
3. Построить диаграмму в зависимости от выбранной тематики:
 - 3.1. *Для тех, кто выбрал тематику игр необходимо построить событийную диаграмму.
 - 3.2. Для остальных тематик, необходимо построить диаграмму последовательности.

4. МАТРИЦА ТРЕБОВАНИЙ

4.1. Формирование нефункциональных требований

Нефункциональные требования — это условия, при которых продукт должен работать, и качества, которыми он должен обладать (например, производительность, надежность, масштабируемость).

Если совсем просто, то к нефункциональным относят те требования, которые не вошли в функциональные.

Нефункциональные требования должны быть измеримы. Их можно проверить и протестировать.

К нефункциональным требованиям относятся: Технические ограничения, бизнес-требования, локализация, доступность, производительность и масштабируемость, надежность, безопасность, удобство использования.

Представьте, что вы покупаете мотоцикл. Какие характеристики вам важны? Чтобы он мог ехать со скоростью 150 км в час и не развалиться на части? Или для вас важно, можно ли прикрепить к нему мотоколяску или прицеп? Наконец, он должен быть безопасным? Все эти требования не описывают напрямую основную функцию мотоцикла — доставку человека из пункта А в пункт Б. Это нефункциональные требования, но для водителей они тоже имеют значение. К сайтам, ПО, приложениям люди тоже предъявляют нефункциональные требования. Это качества, которые удовлетворяют пользователей.

4.1.1. Какие бывают нефункциональные требования

Нефункциональные требования описывают эксплуатационные качества к продукту. Например, ваш продукт собирает какие-либо данные пользователей и работает на территории ЕС. Значит, он должен по закону соответствовать правилам GDPR — Общий регламент по защите данных.

Технические ограничения. Операционные системы и их версии, сетевые особенности, браузеры и их версии, устройства и другие аппаратные требования. Например, разработка должна вестись на определенной платформе, пользователь входит по отпечаткам пальцев.

Бизнес-требования. Процессы и стандарты, принятые в компании. Продукт должен им следовать и разрабатываться в соответствии с ними.

Производительность и масштабируемость. Насколько быстро продукт реагирует на определенные действия пользователей при определенной рабочей нагрузке. Например, сколько пользователь должен ждать, чтобы прошла регистрация в личном кабинете, был обработан платеж с банковской карты. Требования к производительности могут описывать фоновые процессы, которые

пользователь не видит. Например, резервное копирование. Масштабируемость оценивает самые высокие рабочие нагрузки, при которых система все еще будет справляться.

Надежность, доступность, ремонтпригодность. Как часто в системе происходят критические сбои. Как быстро их можно устранить. Пример требований к доступности: сайт должен быть доступна для пользователей из США 98% времени каждый месяц в рабочие часы.

Безопасность. Как система и ее данные защищены от атак или несанкционированного доступа. Но здесь есть одна загвоздка. Львиная доля нефункциональных требований безопасности может быть переведена в конкретные функциональные требования.

Локализация. Соответствие системы особенностям страны, в которой ее будут использовать: местные языки, законы, валюты, культура, орфография и так далее. Чем больше продукт вписывается в контекст, тем больший успех он должен иметь у местной целевой аудитории. Чтобы документально подтвердить это требование, вы должны опираться на предварительное исследование рынка. Пример требования к локализации: формат даты должен быть следующим: месяц.число.год.

В подразделе 2.1.3 Вам было предложено описывать функциональные требования в виде обычных предложений или в виде таблицы. В данной работе предлагается оформить нефункциональные требования также.

Структурированная таблица для нефункциональных требований состоит из следующих столбцов: технические ограничения, производительность, масштабируемость, надежность, безопасность, локализация и т.д. (Можно добавить или убрать столбцы на свое усмотрение, поскольку не все требования могут быть в разработке).

В табл. 4.1 представлены примеры нефункциональных требований:

Таблица 4.1. Нефункциональные требования

Тип требования	Содержание требования
Технические ограничения	Веб-приложение должно быть кроссбраузерным и одинаково функционировать в всех популярных браузерах: Google Chrome, Mozilla Firefox, Opera, Yandex.Браузер, Microsoft Edge, Safari
	Мобильное приложение должно функционировать под версии Android 9.0 и старше

Тип требования	Содержание требования
Производительность	Полное резервное копирование базы данных клиентов должно производиться не менее чем 1 раз в неделю
	Snapshot базы данных заказов должен производиться не менее чем 1 раз в сутки
Масштабируемость	При нагрузке в 1000 запросов в секунду система должна выдавать пользователю ответ на запрос в пределах 500 мс
Надежность	Приложение должно иметь время uptime в год 99%
Безопасность	Для хэширования пароля необходимо использовать алгоритм SHA256

4.1.2. Формирование матрицы требований

Матрица требований – формализованный вид всех требований к разработке проекта. Матрица требований состоит из функциональных требований и нефункциональных требований.

Необходимые свойства к требованиям в матрице требований:

- 1) измеримость (SLI – индикатор/показатель (можем померить), SCO – критерий (порог, критическое значение), SLA – соглашение/договор);
- 2) полнота;
- 3) идентифицируемость (возможность четкого определения каждого требования без смешивания с другими требованиями);
- 4) прослеживаемость (отслеживание реализации требования в конечном продукте).

Источники требований в матрице требований:

1. Федеральное законодательство.*
2. Региональное и отраслевое регулирование.*
3. Стандарты.*
4. Документация на технологические решения.*
5. Работа с заказчиком (анкетирование, мозговой штурм).*
6. Моделирование (бизнес-процессы, математическое моделирование, имитационное моделирование).
7. Системы–аналоги и их анализ.

Примечание: пункт с * – хранилища готовых требований

Рассмотрим пример матрицы требований, который представлен в табл. 4.2, данная матрица требований составлена для приложения заказа такси.

Матрица требований подразделяется на составляющие приложения. В столбце “Требования” пишутся соответственно требования, которые предъявляются к данной составляющей. В столбце “Суть” необходимо подробно раскрыть смысл требования. В столбце “Автор” указывается кто придумал данное требование из команды. В столбце “Ссылки” необходимо указывать ссылку откуда было “почерпнуто” данное требование. В столбце “Критерий проверки” отражается как будет проверяться соответствие приложения данному требованию.

Таблица 4.2. Матрица требований

№	Требование	Суть	Автор	Ссылки	Критерий проверки
1	Мобильное приложение клиента				
1.1	Регистрация пользователя	“Приложение должно иметь функцию регистрации нового пользователя”	Иванов И.И.	https://habr.com/ru/company/infopulse/blog/346318/	Регистрация нового пользователя
1.2	Авторизация пользователя	“Приложение должно иметь функцию авторизации зарегистрированного пользователя”	Иванов И.И.	https://habr.com/ru/company/infopulse/blog/346318/	Авторизация зарегистрированного пользователя в приложении
1.3	Услуга “Вызов такси”	“Приложение должно иметь возможность заказа такси для авторизованного клиента”	Иванов И.И.	http://www.consultant.ru/document/cons_doc_LAW_85364/42005365849f53f10d570b0e31c04a1e6fb5080e/	Вызов такси
...	...	...	...	...	...
1.*	Расчет примерного времени маршрута	"Приложение должно иметь возможность примерного расчёта времени маршрута на базе сервиса Яндекс.Карты"	Иванов И.И.	https://yandex.ru/blog/mapsapi/38211	Возможность расчёта примерного времени маршрута
2	Веб-интерфейс				
2.1	Регистрация пользователя	“Приложение должно иметь функцию регистрации нового пользователя”	Петров А.А.	https://habr.com/ru/company/infopulse/blog/346318/	Регистрация нового пользователя

Продолжение табл. 4.2.

№	Требование	Суть	Автор	Ссылки	Критерий проверки
2.2	Авторизация пользователя	“Приложение должно иметь функцию авторизации зарегистрированного пользователя”	Петров А.А.	https://habr.com/ru/company/infopulse/blog/346318/	Авторизация зарегистрированного пользователя на сайте
...	...	...	...	...	...
2.*	Завершение маршрута и оценка	“В приложении должна быть функция подтверждения успешного выполнения услуги, а также оценка качества”	Петров А.А.	https://habr.com/ru/post/305374/	Завершение маршрута и оценка качества услуги
3	Приложение для водителя				
3.1	Авторизация водителя	“Приложение должно иметь функцию авторизации зарегистрированного (вручную модератором) водителя”	Сидоров С.С.	https://habr.com/ru/company/infopulse/blog/346318/	Авторизация водителя в приложении
3.2	Вывод контактной информации о клиенте	"Приложение должно иметь функцию вывода контактной информации о клиенте, для возможности связи"	Сидоров С.С.	http://www.consultant.ru/document/cons_doc_LAW_61801/	Вывод контактной информации выбранного клиента
...	...	...	...	...	...
3.*	Расчет оптимального маршрута заказа	"Приложение должно иметь функцию для прокладки оптимального маршрута для выполнения заказа с учетом транспортной ситуации на дорогах в текущий момент времени"	Сидоров С.С.	https://yandex.ru/blog/mapsapi/38211	Вывод оптимального маршрута следования
4	Правовые нормы регулирования деятельности компании				

№	Требование	Суть	Автор	Ссылки	Критерий проверки
4.1	Закон "О защите персональных данных"	Программная система, при получении, хранении и обработке персональных данных клиентов и сотрудников компании, должна руководствоваться нормами закона Федерального закона "О персональных данных" от 27.07.2006 N 152-ФЗ (последняя редакция)	Васильев В.В.	http://www.consultant.ru/document/cons_doc_LAW_61801/	Прохождение теста на невозможность получения закрытых персональных данных третьими лицами
...	...	...	...	...	...
4.*	Выполнение правил перевозки пассажиров и багажа	Система должна по возможности помогать исполнителю выполнять требования Постановления Правительства РФ от 14 февраля 2009 г. N 112 "Об утверждении Правил перевозок пассажиров и багажа автомобильным транспортом и городским наземным электрическим транспортом" (с изменениями и дополнениями)	Васильев В.В.	" http://77.rospotrebнадзор.ru/index.php/napravlenie/zpp/6673-pamyatka-potrebitelyu-perevozke-passazhira-legkovym-taksi https://base.garant.ru/195015/ "	Не должно быть выявлено нарушений

Примечание: обратите внимание на то, что матрица требований в данный момент неполная. Матрица требований будет дополняться на основании будущих разделов.

4.2. Задание

1. Определить нефункциональные требования для своей разработки. Нефункциональных требований должно быть не менее 3 для каждого из выбранных "разделов" и их составляют все члены команды.
2. Сформировать матрицу требований для своей разработки. За матрицу требований отвечает технический писатель.

5. СОЗДАНИЕ СТРУКТУРНОЙ ДИАГРАММЫ СИСТЕМЫ

5.1. Структурная диаграмма

Основное назначение структурных диаграмм заключается в графическом представлении состава статистических совокупностей, характеризующихся как соотношением различных частей каждой из совокупностей.

Структурная диаграмма – это инструмент модульного дизайна сверху вниз, построенный из квадратов, представляющих различные модули в системе и соединяющие их линии. Линии представляют связь и / или право собственности между видами деятельности и вспомогательными видами деятельности, как они используются в организационных диаграммах.

Структурная диаграмма отображает:

- 1) размер и сложность системы;
- 2) количество легко идентифицируемых функций и модулей в каждой функции;
- 3) является ли каждая идентифицируемая функция управляемым объектом или должна быть разбита на более мелкие компоненты.

Существует три основных типа структурных диаграмм:

1. Диаграмма классов.
2. Диаграмма развертывания.
3. Диаграмма объектов.

5.1.1. Диаграмма классов

Диаграмма классов (рис. 5.1) – структурная диаграмма языка моделирования UML, демонстрирующая общую структуру иерархии классов системы, их коопераций, атрибутов (полей), методов, интерфейсов и взаимосвязей между ними. Широко применяется не только для документирования и визуализации, но также для конструирования посредством прямого или обратного проектирования.



Рисунок 5.1. Пример заполнения в диаграмме классов

5.1.2. Диаграмма объектов

Диаграмма объектов в языке моделирования UML предназначена для демонстрации совокупности моделируемых объектов и связей между ними в фиксированный момент времени.

Диаграмма объектов (рис. 5.2) описывает конкретные экземпляры объектов и напрямую соотносится с диаграммой классов, которая дает общее представление о конфигурации системы. Она используется для документирования структур данных и создания статических снимков состояний объектов принимая во внимание реальные экземпляры или прототипы. Динамику поведения объектов обычно изображают в виде последовательности таких диаграмм.

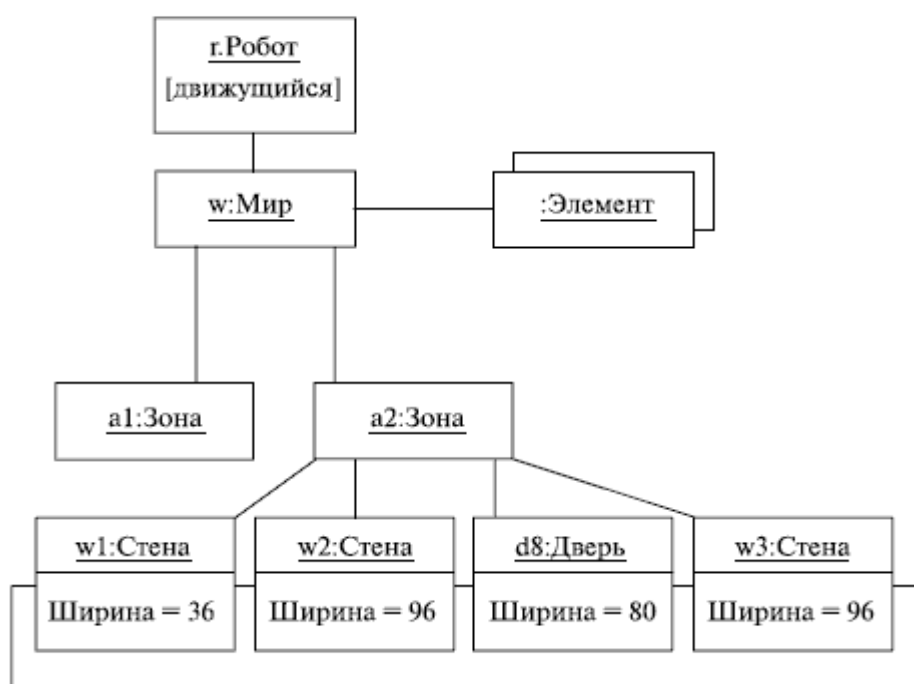


Рисунок 5.2. Пример заполнения в диаграмме объектов

5.1.3. Порядок построения структурной диаграммы

Структурная диаграмма может быть разработана, начиная с создания структуры, которая помещает корень перевернутого дерева, образующего структурную диаграмму. Следующим шагом является концептуализация основных подзадач, которые должны выполняться программой для решения проблемы. Затем программист сосредотачивается на каждой подзадаче индивидуально и концептуализирует, как каждую подзадачу можно разбить на еще более мелкие задачи. В конечном итоге, программа разбивается до точки, где листья дерева представляют простые методы, которые можно закодировать с помощью всего нескольких программных операторов.

5.1.4. Взаимосвязи в структурных диаграммах

Взаимосвязь (рис. 5.3) — это особый тип логических отношений между сущностями, показанных на диаграммах классов и объектов. В UML представлены следующие виды отношений.

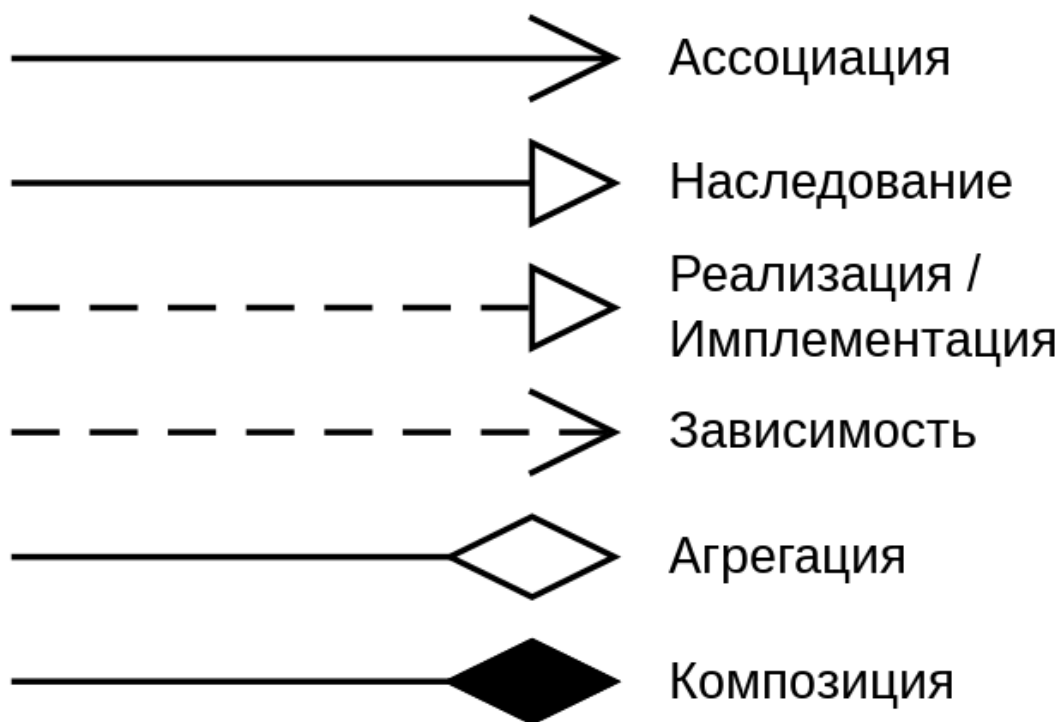


Рисунок 5.3. Взаимосвязи в структурных диаграммах

Зависимость означает такое отношение между классами, что изменение спецификации класса-поставщика может повлиять на работу зависимого класса, но не наоборот.

Ассоциация показывает, что объекты одной сущности (класса) связаны с объектами другой сущности таким образом, что можно перемещаться от объектов одного класса к другому. Является общим случаем композиции и агрегации.

Например, класс Человек и класс Школа имеют ассоциацию, так как человек может учиться в школе. Ассоциации можно присвоить имя «учится в».

Двойные ассоциации представляются линией без стрелочек на концах, соединяющей два классовых блока. Ассоциации более высокой степени имеют более двух концов и представляются линиями, один конец которых идёт к классовому блоку, а другой к общему ромбику. В представлении однонаправленной ассоциации добавляется стрелка, указывающая на направление ассоциации.

Ассоциация может быть именованной, и на концах представляющей её линии могут быть подписаны роли, принадлежности, индикаторы, мультипликаторы, видимости или другие свойства.

Агрегация — это разновидность ассоциации при отношении между целым и его частями. Как тип ассоциации агрегация может быть именованной. Одно отношение агрегации не может включать более двух классов (контейнер и содержимое).

Агрегация встречается, когда один класс является коллекцией или контейнером других. Причем по умолчанию, агрегацией называют агрегацию по ссылке, то есть, когда время существования содержащихся классов не зависит от времени существования содержащего их класса. Если контейнер будет уничтожен, то его содержимое — нет.

Графически агрегация представляется пустым ромбом на блоке класса, и линией, идущей от этого ромба к содержащемуся классу.

Композиция — более строгий вариант агрегации. Известна также как агрегация по значению.

Композиция имеет жесткую зависимость времени существования экземпляров класса контейнера и экземпляров содержащихся классов. Если контейнер будет уничтожен, то всё его содержимое будет также уничтожено.

Графически представляется, как и агрегация, но с закрашенным ромбиком.

5.1.5. Различия между взаимосвязями: композиция и агрегация

Приведем наглядный пример. Комната является частью квартиры, следовательно здесь подходит композиция, потому что комната без квартиры существовать не может. А, например, мебель не является неотъемлемой частью квартиры, но в то же время, квартира содержит мебель, поэтому следует использовать агрегацию.

5.2. Построение процессов проекта в нотациях IDEF0, BPMN 2.0, EPC

Построение процессов выполняются посредством построения диаграмм в различных нотациях. Нотации — графические модели, которые используются, чтобы фиксировать бизнес-процессы, анализировать их и оптимизировать. По сравнению с текстовыми описаниями, графические модели занимают меньше места, помогают увидеть алгоритм наглядно, представить, как он проходит от начала до конца. Однако, в отличие от текстового описания, графическая модель хуже передает детали.

Нотации применяются, чтобы сотрудники могли понять и запомнить схему, по которой они должны, к примеру, обрабатывать заявку на поставку партии

товара. А руководителю схема будет полезна, чтобы он мог найти проблемные или избыточные элементы (этапы, сотрудников), внести нужные корректировки. Часто это помогает ускорить или удешевить работу компании.

По аналогии с языками программирования, нотации называют языками моделирования бизнес-процессов.

Сегодня в мире наиболее популярны 3 нотации:

1. IDEF0.
2. EPC.
3. BPMN.

Для выполнения данного задания вам следует определить какой из процессов является основным и с какой стороны, а точнее с какой роли мы его рассматриваем.

Первая, IDEF0, возникла в армии, точнее – в ВВС США, произошло это в 1980-х годах. Целью была оптимизация работы предприятий, выпускающих военную продукцию.

Вторая, EPC (Event-driven Process Chain), появилась на 10 лет позднее. Её название (“цепочка событийных процессов”) дает понять, что фокус сделан именно на событие.

Нотация BPMN – часть концепции BPM (управления бизнес-процессами). Впервые она возникла в 2004 году (версия 1.0) и несколько раз модернизировалась в 2008, 2009, 2011 и 2013 годах. Самая последняя версия – BPMN 2.0.2.

Рассмотрим особенности каждой из нотаций и их отличия.

5.2.1. Нотация IDEF0

Она позволяет создать модель, которая будет отражать:

- структуру системы;
- функции;
- потоки ресурсов, информации.

Модель IDEF0 разворачивается одновременно слева направо и сверху вниз, по диагонали. Объекты, расположенные левее/выше, доминируют над теми, которые находятся правее/ниже. Доминирующие объекты могут включать в себя зависимые: например, доставка заказа – это элемент, входящий в состав более масштабного процесса управления заказами. Также доминирующие объекты могут являться предшествующими этапами для зависимых: получение заявки – согласование заявки.

Графические элементы:

- прямоугольники – действия или этапы;

- стрелки – ресурсы, исполнители, необходимые для совершения действия или прохождения этапа.

Главное достоинство IDEF0 – крайне высокая степень детализации, можно создать модель, которая будет учитывать на каждом этапе практически все ресурсы, сотрудников, которые потребуются даже для самых сложных алгоритмов. Недостатком является то, что графическая модель занимает очень много места, её тяжело читать, не имея специальных навыков.

Ещё один минус: с помощью IDEF0 лучше всего описываются модели, где бизнес-процесс представляет собой одну цепочку, без развилок. Если на пути он встречает множественные “или”, работать с IDEF0 становится очень сложно.

Пример построения в данной нотации можете ознакомиться ниже. Был рассмотрен процесс “работа с документами” (рис. 5.4-5.5) от лица “документоведа учебного отдела” с целью “оценка трудоемкости, организация информационного потока, определения функций документоведа учебного отдела”.

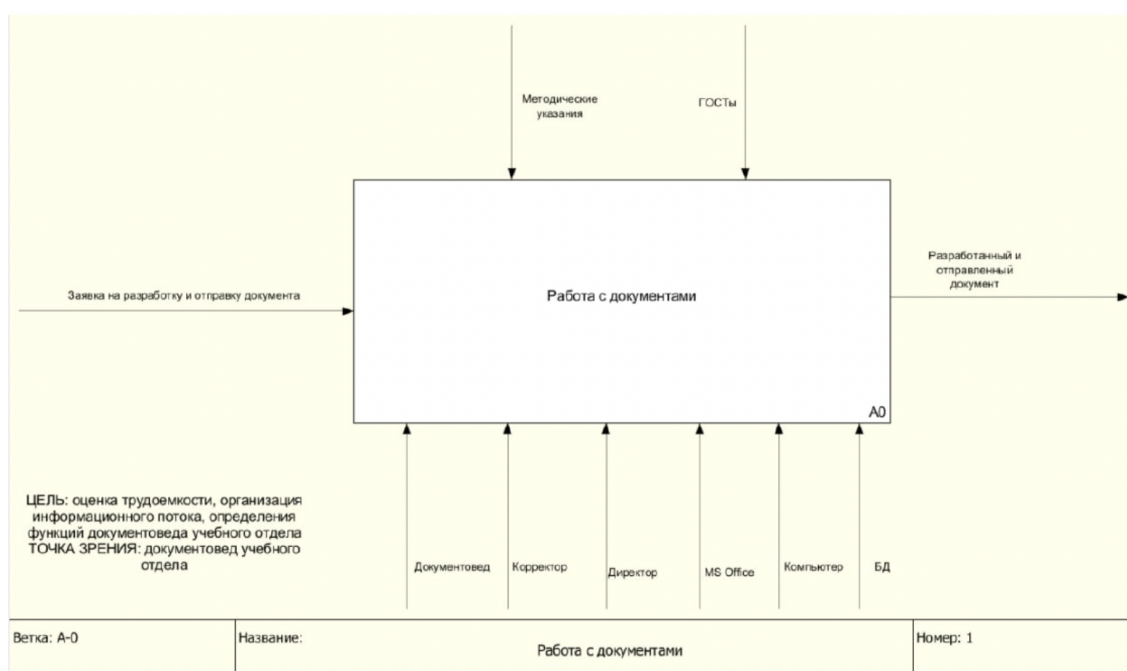


Рисунок 5.4. Процесс «Работа с документами» в Институте информационных технологий от лица документоведа

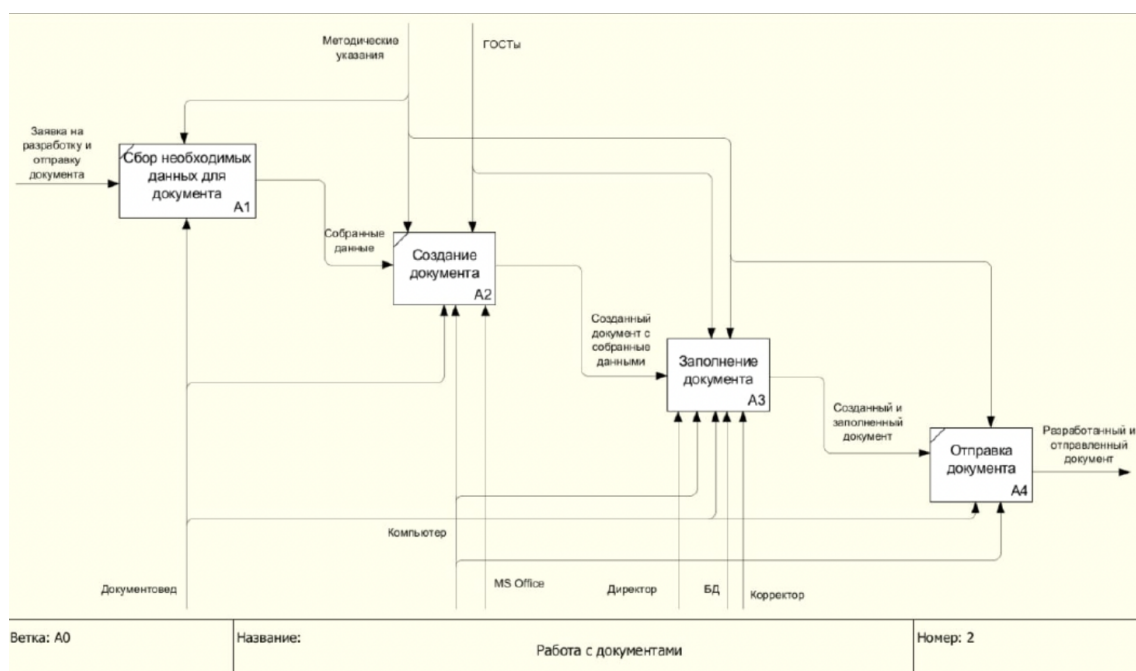


Рисунок 5.5. Декомпозиция процесса «Работа с документами» в Институте информационных технологий от лица документоведа

5.2.2. Нотация BPMN

Её центром является именно бизнес-процесс, и она используется, чтобы показать алгоритм его прохождения.

Основные элементы BPMN:

1. Задача (прямоугольник).
2. Событие (круг).
3. Шлюз, развилка (ромб).
4. Поток, ход (стрелка).
5. Базы данных, документы.
6. Сноски.
7. Пулы.

Базовая нотация BPMN включает не более 10 типов значков и помогает описать алгоритм в такой форме, которая будет понятна бизнес-пользователю, не прошедшему специального обучения. Расширенная BPMN содержит около 100 значков и позволяет сделать регламент машиночитаемым, причём не допуская разночтений.

Главное преимущество BPMN – она лучше всего подходит, если нужно описать именно бизнес-процесс, сделав его понятным даже для рядовых сотрудников. Сегодня BPMN пользуется популярностью: большинство вендоров, предлагающих системы BPM, предусматривают работу с BPMN: схему, созданную с её помощью, можно сделать исполняемой, подключив возможности информационной системы.

Недостаток BPMN в том, что она заиклена на бизнес-процессах и плохо подходит для описания структуры предприятия или дерева целей. При использовании расширенной версии схема усложняется, и понять её человеку без специальных навыков будет уже сложно.

В целом BPMN сегодня наиболее распространена, именно она пользуется наибольшим уважением в международной Ассоциации BPM-профессионалов (ABRMP). Выбор нотации для конкретного случая зависит от того, что именно будет описываться с её помощью, а также от информационных систем, которые планируется применять.

С примером в данной нотации можно ознакомиться ниже. В примере был приведен процесс «работа с документами» (рис. 5.6).

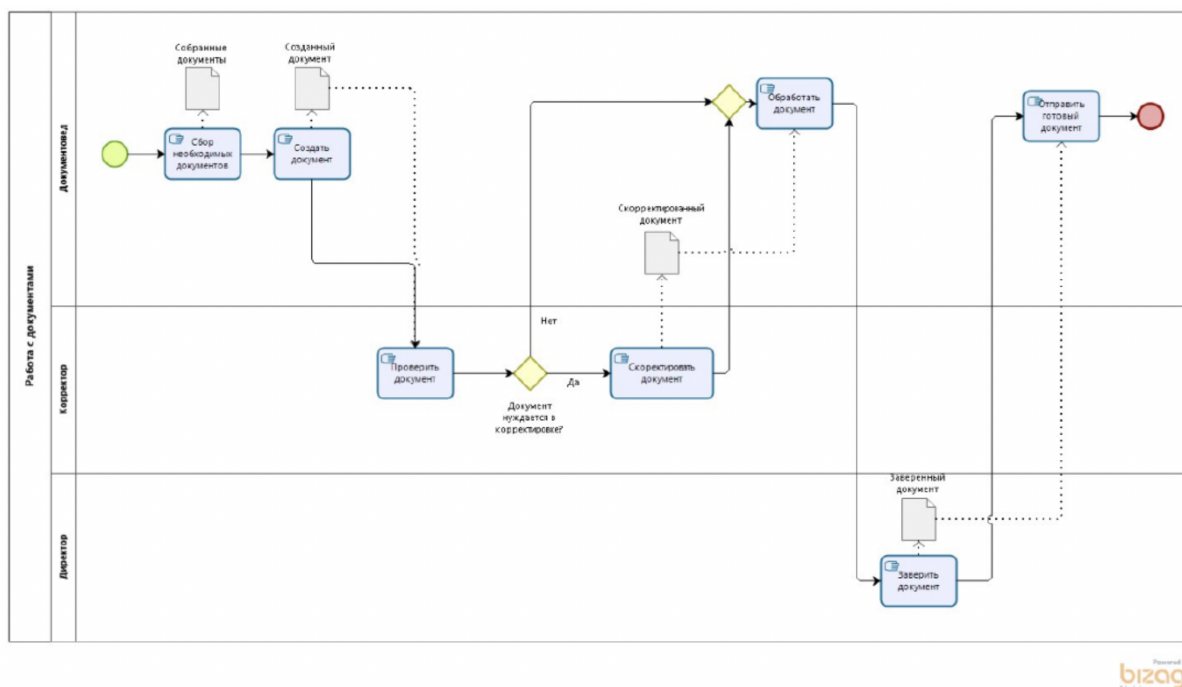


Рисунок 5.6. Бизнес-процесс «Работа с документами» в нотации BPMN 2.0

5.3. Задание

1. Построить структурные диаграммы своего проекта.
 - 1.1. Диаграмма классов.
 - 1.2. Диаграмма объектов.
2. Разобрать процесс работы внутри проекта в нотации по выбору, IDEF0 или BPMN 2.0. Построить собственный процесс в выбранной нотации.

6. СОЗДАНИЕ ИНФОРМАЦИОННОЙ ДИАГРАММЫ СИСТЕМЫ И ЛОГИЧЕСКОЙ МОДЕЛИ БАЗЫ ДАННЫХ

6.1. Диаграмма в нотации DFD

DFD — общепринятое сокращение от англ. data flow diagrams — диаграммы потоков данных. Так называется методология графического структурного анализа, описывающая внешние по отношению к системе, источники и адресаты данных, логические функции, потоки данных и хранилища данных, к которым осуществляется доступ. Диаграмма потоков данных (data flow diagram, DFD) — один из основных инструментов структурного анализа и проектирования информационных систем, существовавших до широкого распространения UML.

6.1.1. Различия внутри DFD

Исторически синтаксис этой нотации применяется в двух вариантах (рис. 6.1) — Йордана (Yourdon) и Гейна-Сарсона (Gane-Sarson). Различия между ними ниже:

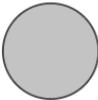
Нотация	Юрдан и Коад	Гейн и Сарсон
Внешняя сущность		
Процесс		
Хранилище данных		
Поток данных		

Рисунок 6.1. Различия синтаксиса внутри нотации DFD

В DFD нет жесткого синтаксиса, как, например, в BPMN. Здесь можно использовать разные варианты, главное, чтобы они были понятны вам и вашим

клиентам. Нотации DFD — удобный инструмент для создания нерегламентированных диаграмм.

Применяется этот вид нотации в случае, когда требуется описание системы как хранилища данных. Т.е. нотация должна наглядно ответить на вопросы:

- из чего состоит информационная система?
- что нужно, чтобы обработать информацию?

6.1.2. Элементы нотации DFD

Непосредственно DFD нотация состоит из следующих элементов:

- *процесс (англ. Process)*, т.е. функция или последовательность действий, которые нужно предпринять, чтобы данные были обработаны. Это может быть создание заказа, регистрация клиента и т.д. В названиях процессов принято использовать глаголы, т.е. «Создать клиента» (а не «создание клиента») или «обработать заказ» (а не «проведение заказа»). Здесь нет строгой системы требований, как, например, в IDEF0 или BPMN, где нотации имеют жестко определенный синтаксис, так как они могут быть исполняемыми. Но все же определенных правил стоит придерживаться, чтобы не вносить путаницу при чтении DFD другими людьми.
- *внешние сущности (англ. External Entity)*. Это любые объекты, которые не входят в саму систему, но являются для нее источником информации либо получателями какой-либо информации из системы после обработки данных. Это может быть человек, внешняя система, какие-либо носители информации и хранилища данных.
- *хранилище данных (англ. Data store)*. Внутреннее хранилище данных для процессов в системе. Поступившие данные перед обработкой и результат после обработки, а также промежуточные значения должны где-то храниться. Это и есть базы данных, таблицы или любой другой вариант организации и хранения данных. Здесь будут храниться данные о клиентах, заявки клиентов, расходные накладные и любые другие данные, которые поступили в систему или являются результатом обработки процессов.
- *поток данных (англ. Data flow)*. В нотации отображается в виде стрелок, которые показывают, какая информация входит, а какая исходит из того или иного блока на диаграмме.

6.1.3. Порядок построения диаграммы потоков данных

Давайте для примера рассмотрим нотацию автоматизации продаж. Допустим, у нас есть клиент, который делает заявку через сайт или по телефону. Есть менеджер, который регистрирует эту заявку. Таким образом, в системе появляются данные – клиент и его заказ. Работник склада должен это увидеть и произвести отгрузку товара с оформлением всех необходимых документов и передать документы клиенту.

Последовательность получается такая:

1. Клиент предоставляет свои данные и заявку.
2. Менеджер проверяет и вносит полученные данные в систему.
3. Работник склада формирует документы, например, расходную накладную, и отгружает товар.
4. Клиент получает товар и пакет документов к нему.

Эту последовательность действий нам необходимо увидеть с точки зрения хранения данных и работы с ними в IT-системе.

С точки зрения DFD у нас имеются:

- покупатель – это внешняя сущность, которая является источником данных и получением результата;
- процесс обработки заказа (подтверждение и проводка данных в системе менеджером);
- сбор заказа на складе (после получения заявки);
- оформление отгрузки (создание необходимых документов).

Какие правила необходимо знать, чтобы создать DFD диаграмму:

- *каждый процесс должен иметь хотя бы один вход и один выход.* Смысл процессов здесь заключается в обработке данных, а потому процесс должен получить данные (входящая стрелка) и отдать куда-то после обработки (исходящая стрелка);
- *процесс обработки данных должен иметь внешнюю входящую стрелку (данные от внешней сущности).* Для того, чтобы любой подобный процесс начал работать, мало использовать данные из хранилища, должна поступить новая информация для последующей обработки;
- *стрелки не могут связывать напрямую хранилища данных, все связи идут через процессы.* Нет смысла просто перемещать данные из одного места в другое, а именно так читается прямая связь двух хранилищ стрелкой. Данные поступают для того, чтобы производились

какие-то действия, в нашем примере – осуществлялся процесс продажи. А это возможно только посредством обработки (процесса);

- *все процессы должны быть связаны либо с другими процессами, либо с другими хранилищами данных.* Процессы не существуют сами по себе, а потому результат должен куда-то передаваться;
- *декомпозиция.* В DFD-диаграммах предусмотрена возможность создавать крупные процессы и декомпозировать их на подпроцессы с подробным описанием действий.

Например, мы можем создать процесс «создание заявки», который потом декомпозировать на последовательность действий, например, на получение заявки, отдельно – проверку и получение данных клиента, если товар в интернет-магазине продается под заказ, то также при формировании заявки потребуется получить данные от поставщика о наличии нужных наименований и т.д. И тогда на верхней диаграмме у нас будет блок «обработка заявки», а при декомпозировании мы получим диаграмму с подробной последовательностью действий на этом этапе. При этом ни на одном этапе у нас не будет условий и ветвления. Будет процесс и его декомпозиция глубиной до 3-4 уровней.

Как будет выглядеть диаграмма (без декомпозиции, верхний уровень, рис. 6.2):

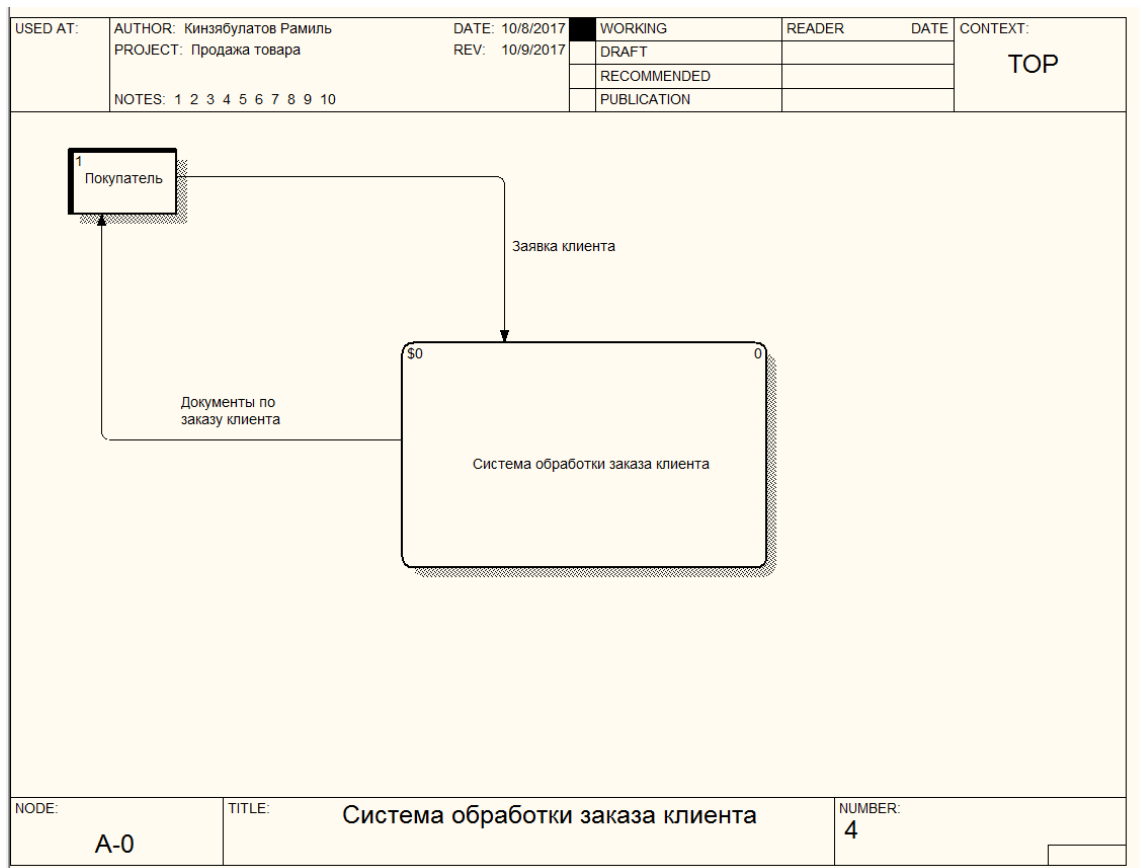


Рисунок 6.2. Диаграмма DFD. Верхний уровень

И декомпозиция основного элемента нашей диаграммы (рис. 6.3):

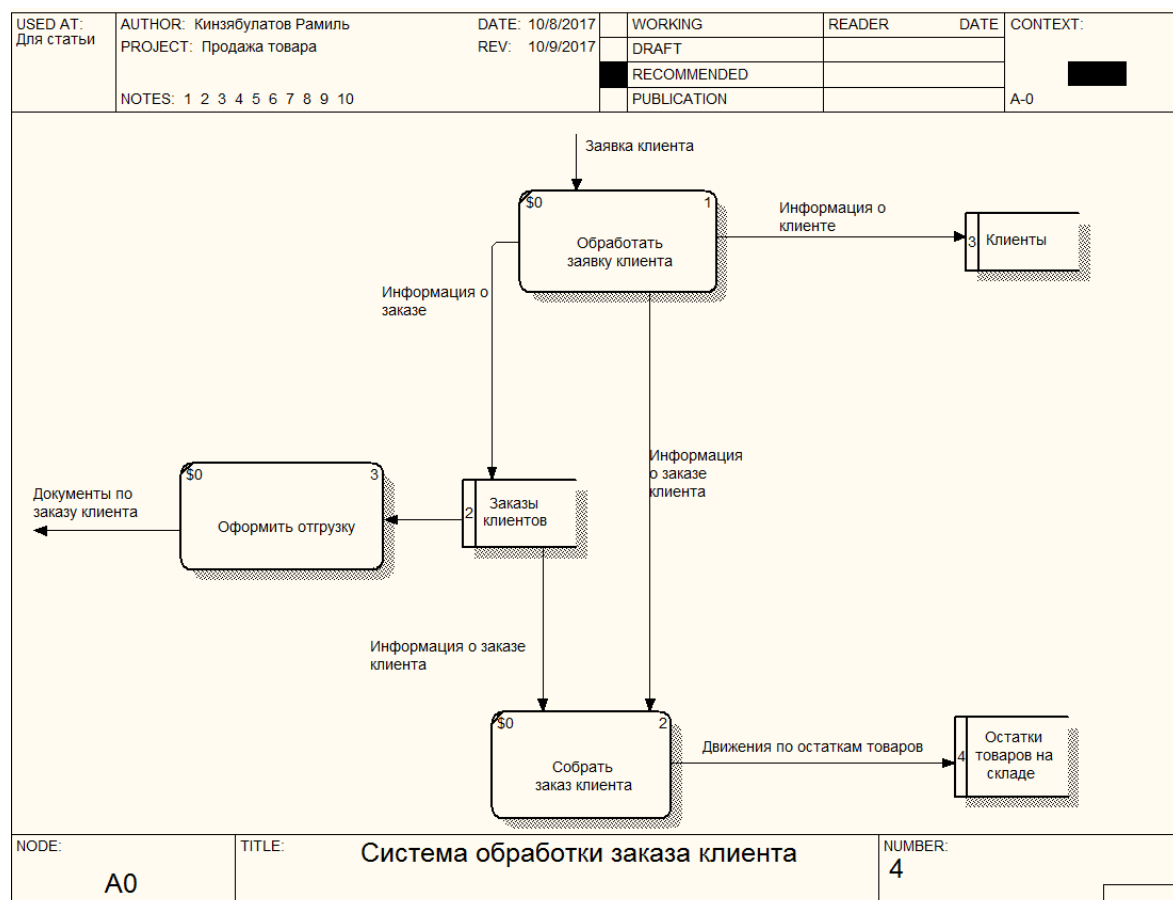


Рисунок 6.3. Диаграмма DFD. Декомпозиция

DFD-диаграммы активно применяются при разработке программного обеспечения. При этом:

- хранилища данных – это электронные таблицы и базы данных;
- внешние сущности – клиенты или другие базы данных, в том числе, из других программ (интеграция и обмен данными);
- процессы – это выполняемые функции и модули в системе.

6.2. Модель базы данных

Модель данных – это совокупность структур данных и операций их обработки.

6.2.1. Типы связей в модели базы данных

1. «Один к одному».

В этом виде отношений один объект связан с другим. Например, Менеджер -> Отдел. У каждого менеджера может быть только один отдел, и наоборот.

2. «Один ко многим».

В моделях данных отношение одного объекта с несколькими. Например, Сотрудник -> Отдел.

Каждый сотрудник может быть только в одном отделе, но в самом отделе может быть больше одного сотрудника.

3. «Многие ко многим».

В заданный момент времени объект может быть связан с любым другим. Например, Сотрудник -> Проект.

Сотрудник может участвовать в нескольких проектах, и каждый проект может объединять несколько сотрудников.

6.2.2. Реляционная модель базы данных

Реляционная модель базы данных - данные в базе данных представляют собой набор отношений. Отношения (таблицы) отвечают определенным условиям целостности. Реляционная модель данных поддерживает декларативные ограничения целостности уровня домена (типа данных), уровня отношения и уровня базы данных.

- в реляционной модели объекты и их отношения представлены двумерным массивом или таблицей;
- каждая таблица представляет объект;
- каждая таблица состоит из рядов и столбцов;
- отношения между объектами представлены столбцами;
- каждый столбец представляет атрибут объекта;
- значения столбцов выбираются из области или набора всех возможных значений;
- столбцы, которые используются для связи объектов, называются ключевыми. Есть два типа ключей — первичные и внешние;
- первичные служат для однозначного определения объекта. Внешний ключ — это первичный ключ одного объекта, существующий как атрибут в другой таблице.

Преимущества реляционной модели данных:

1. Простота использования.
2. Гибкость.
3. Независимость данных.
4. Безопасность.
5. Простота практического применения.
6. Слияние данных.
7. Целостность данных.

Таблицы в реляционной бд обозначаются прямоугольниками. В заголовке указывается имя таблицы.

Таблица состоит из трех столбцов со следующим содержанием:

- имя поля;
- тип поля;
- свойства поля.

6.2.3. Различия между логической и физической моделью БД

Логическая модель базы данных (рис. 6.4) — не строгое именование концептуальной схемы базы данных из архитектуры ANSI/SPARK. Физическая модель базы данных — не строгое именование внутренней схемы базы данных из архитектуры ANSI/SPARK.

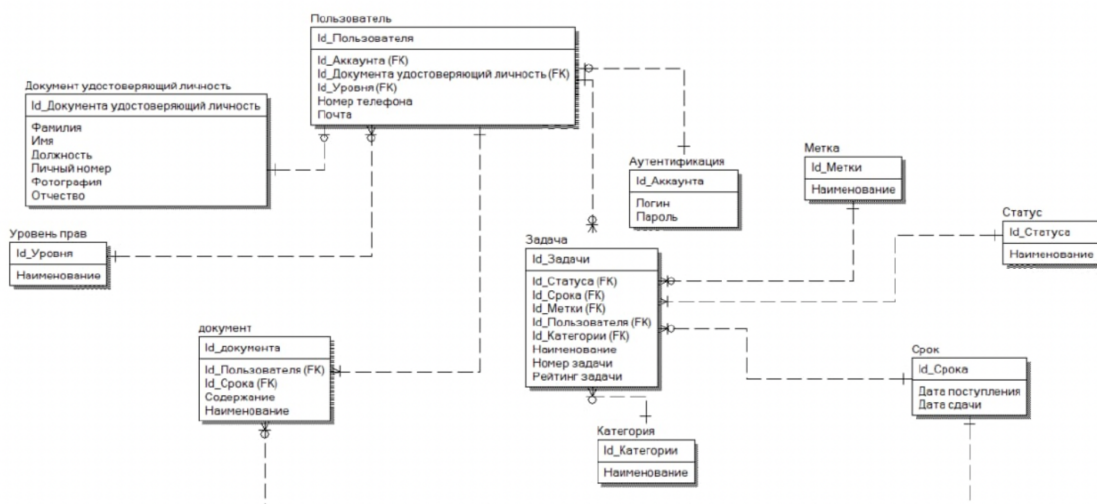


Рисунок 6.4. Пример логической модели базы данных

Если расписать более подробно, то получается следующая картина:

- физическая модель данных описывает физическую структуру базы данных. Логическая модель данных — это модель высокого уровня, которая не описывает физическую структуру базы данных.
- физическая модель данных зависит от используемой системы управления базой данных. Однако логическая модель данных не зависит от используемой системы управления базами данных.
- логическая модель данных включает сущности, атрибуты, отношения и ключи. Физическая модель данных включает таблицы, столбцы, типы данных, ограничения первичного и внешнего ключей, триггеры и хранимые процедуры.

- в логической модели данных для сущностей и атрибутов используются длинные неформальные имена. Однако в физических данных для имен таблиц и столбцов используются сокращенные формальные имена.
- логическая модель данных сначала выводится из описания. После этого выводится только физическая модель данных.
- логическая модель данных нормализована до четвертой нормальной формы. При необходимости физическая модель базы данных будет деформализована для соответствия требованиям.

6.3. Задание

1. Посредством анализа прошлых диаграмм и изучив материал практики, выполнить построение диаграммы в нотации DFD для своего проекта.
2. Словесно описать информационное взаимодействие компонентов системы.
3. Построить нормализованную логическую модель базы данных собственного проекта.

7. АРХИТЕКТУРА СИСТЕМЫ

Взявшись за написание небольшого, но реального и растущего проекта, мы «на собственной шкуре» убедились, насколько важно то, чтобы система не только хорошо работала, но и была хорошо организована. Не верьте, что продуманная архитектура нужна только большим проектам (просто для больших проектов «смертельность» отсутствия архитектуры очевидна). Сложность, как правило, растет гораздо быстрее размеров программы. И если не позаботиться об этом заранее, то довольно быстро наступает момент, когда ты перестаешь ее контролировать. Правильная архитектура экономит очень много сил, времени и денег. А нередко вообще определяет то, выживет ваш проект или нет. И даже если речь идет всего лишь о «построении табуретки» все равно вначале очень полезно ее спроектировать.

7.1. Критерии хорошей архитектуры

Обычно говорят, что не существует общепринятого термина «архитектура программного обеспечения». Тем не менее, когда дело касается практики, то для большинства разработчиков и так понятно какой код является хорошим, а какой плохим. *Хорошая архитектура* – это прежде всего *выгодная* архитектура, делающая процесс разработки и сопровождения программы более простым и эффективным. Программу с хорошей архитектурой легче расширять и изменять, а также тестировать, отлаживать и понимать. То есть, на самом деле можно сформулировать список вполне разумных и универсальных критериев:

Эффективность системы. В первую очередь программа, конечно же, должна решать поставленные задачи и хорошо выполнять свои функции, причем в различных условиях. Сюда можно отнести такие характеристики, как надежность, безопасность, производительность, способность справляться с увеличением нагрузки (масштабируемость) и т.п.

Гибкость системы. Любое приложение приходится менять со временем — изменяются требования, добавляются новые. Чем быстрее и удобнее можно внести изменения в существующий функционал, чем меньше проблем и ошибок это вызовет — тем *гибче* и конкурентоспособнее система. Поэтому в процессе разработки старайтесь оценивать то, что получается, на предмет того, как вам это потом, возможно, придется менять. Спросите у себя: «А что будет, если текущее архитектурное решение окажется неверным?», «Какое количество кода подвергнется при этом изменениям?». Изменение одного фрагмента системы не должно влиять на ее другие фрагменты. По возможности, архитектурные решения не должны «вырубаться в камне», и последствия архитектурных ошибок должны

быть в разумной степени ограничены. *"Хорошая архитектура позволяет ОТ-КЛАДЫВАТЬ принятие ключевых решений"* (Боб Мартин) и минимизирует «цену» ошибок.

Расширяемость системы. Возможность добавлять в систему новые сущности и функции, не нарушая ее основной структуры. На начальном этапе в систему имеет смысл закладывать лишь основной и самый необходимый функционал (принцип YAGNI — you ain't gonna need it, «Вам это не понадобится»). Но при этом архитектура должна позволять легко наращивать дополнительный функционал по мере необходимости. Причем так, чтобы внесение *наиболее вероятных* изменений требовало *наименьших усилий*.

Требование, чтобы архитектура системы обладала *гибкостью* и *расширяемостью* (то есть была способна к изменениям и эволюции) является настолько важным, что оно даже сформулировано в виде отдельного принципа — «Принципа открытости/закрытости» (Open-Closed Principle — второй из пяти принципов SOLID): *Программные сущности (классы, модули, функции и т.п.) должны быть открытыми для расширения, но закрытыми для модификации.*

Иными словами: *Должна быть возможность расширить/изменить поведение системы без изменения/переписывания уже существующих частей системы.*

Это означает, что приложение следует проектировать так, чтобы изменение его поведения и добавление новой функциональности достигалось бы за счет написания нового кода (расширения), и при этом не приходилось бы менять уже существующий код. В таком случае появление новых требований не повлечет за собой модификацию существующей логики, а может быть реализовано прежде всего за счет ее расширения. Именно этот принцип является основой «плагиной архитектуры» (Plugin Architecture).

Масштабируемость процесса разработки. Возможность сократить срок разработки за счёт добавления к проекту новых людей. Архитектура должна позволять распараллелить процесс разработки, так чтобы множество людей могли работать над программой одновременно.

Тестируемость. Код, который легче тестировать, будет содержать меньше ошибок и надежнее работать. Но тесты не только улучшают качество кода.

Существует целая методология разработки программ на основе тестов, которая так и называется — *Разработка через тестирование (Test-Driven Development, TDD).*

Возможность повторного использования. Систему желательно проектировать так, чтобы ее фрагменты можно было повторно использовать в других системах.

Хорошо структурированный, читаемый и понятный код. Сопровождаемость. Над программой, как правило, работает множество людей — одни уходят, приходят новые. После написания сопровождать программу тоже, как правило, приходится людям, не участвовавшим в ее разработке. Поэтому хорошая архитектура должна давать возможность относительно легко и быстро разобраться в системе новым людям. Проект должен быть хорошо структурирован, не содержать дублирования, иметь хорошо оформленный код и желательно документацию. И по возможности в системе лучше применять стандартные, общепринятые решения привычные для программистов. Чем экзотичнее система, тем сложнее ее понять другим (*Принцип наименьшего удивления — Principle of least astonishment*. Обычно, он используется в отношении пользовательского интерфейса, но применим и к написанию кода).

7.2. Монолитная архитектура

Монолит — это древнее слово, обозначающее огромный каменный блок. Хотя этот термин широко используется сегодня, представление остается одинаковым во всех областях. В программной инженерии монолитная модель относится к единой неделимой единице. Концепция монолитного программного обеспечения заключается в том, что различные компоненты приложения объединяются в одну программу на одной платформе. Обычно монолитное приложение состоит из базы данных, клиентского пользовательского интерфейса и серверного приложения. Все части программного обеспечения унифицированы, и все его функции управляются в одном месте. Давайте посмотрим на структуру монолитного программного обеспечения в деталях.

Монолитная архитектура (рис. 7.1) удобна для работы небольших групп, поэтому многие стартапы выбирают этот подход при создании приложения. Компоненты монолитного программного обеспечения взаимосвязаны и взаимозависимы, что помогает программному обеспечению быть самодостаточным. Эта архитектура является традиционным решением для создания приложений, но некоторые разработчики считают ее устаревшей.

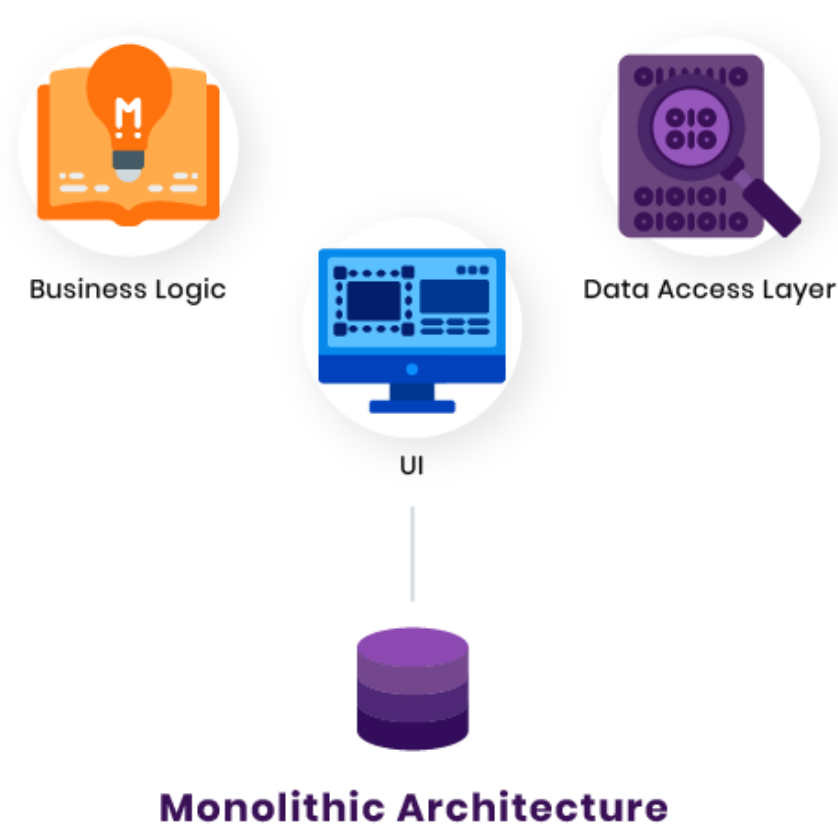


Рисунок 7.1. Монолитная архитектура

7.2.1. Плюсы монолитной архитектуры

Упрощенная разработка и развертывание. Есть много инструментов, которые вы можете интегрировать для облегчения разработки. Кроме того, все действия выполняются с одним каталогом, что упрощает развертывание. Благодаря монолитному ядру разработчикам не нужно развертывать изменения или обновления по отдельности, поскольку они могут сделать это сразу и сэкономить много времени.

Меньше сквозных проблем. Большинство приложений зависят от множества межкомпонентных задач, таких как контрольные журналы, ведение логов, ограничение скорости и т. д. Монолитные приложения гораздо легче учитывают эти вопросы благодаря своей единой кодовой базе. К этим задачам проще подключать компоненты, когда все работает в одном приложении.

Лучшая производительность. При правильной сборке монолитные приложения обычно более производительны, чем приложения на основе микросервисов. Например, приложению с микросервисной архитектурой может потребоваться выполнить 40 вызовов API для 40 различных микросервисов чтобы загрузить каждый экран, что, очевидно, приводит к снижению производительности. Монолитные приложения, в свою очередь, обеспечивают более быструю связь между программными компонентами благодаря общему коду и памяти.

7.2.2. Минусы монолитной архитектуры

Кодовая база со временем становится громоздкой. С течением времени большинство продуктов продолжают разрабатываться и увеличиваются в объеме, а их структура становится размытой. Кодовая база начинает выглядеть действительно громоздко и становится трудной для понимания и изменения, особенно для новых разработчиков. Также становится все труднее находить побочные эффекты и зависимости. С ростом кодовой базы ухудшается качество и перегружается IDE.

Сложно внедрять новые технологии. Если в ваше приложение необходимо добавить какую-то новую технологию, разработчики могут столкнуться с препятствиями на пути внедрения. Добавление новой технологии означает переписывание всего приложения, что является дорогостоящим и требует много времени.

Ограниченная гибкость. В монолитных приложениях каждое небольшое обновление требует полного повторного развертывания. Таким образом, все разработчики должны ждать, пока это не будет сделано. Когда несколько команд работают над одним проектом, гибкость может быть значительно снижена.

7.3. Трехуровневая архитектура

Трехуровневая архитектура (рис. 7.2) — это широко применяемая архитектура программного обеспечения, в которой приложения разделены на три логических и физических уровня: уровень представления (пользовательский интерфейс), уровень приложения, на котором осуществляется обработка данных, и уровень данных, предназначенный для хранения и управления данными, относящимися к приложению.

Основное преимущество трехуровневой архитектуры заключается в том, что поскольку каждый уровень имеет собственную инфраструктуру, разработкой каждого уровня может заниматься отдельная команда разработчиков. Кроме того, каждый уровень можно обновлять и масштабировать по мере необходимости, не затрагивая другие уровни.

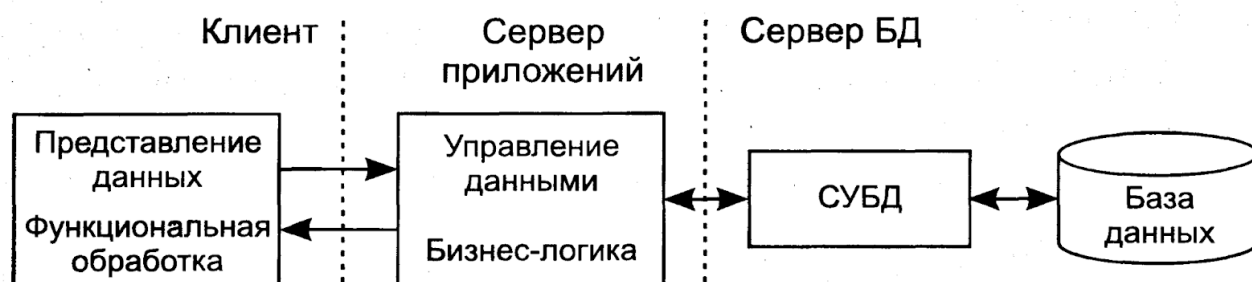


Рисунок 7.2. Трехуровневая архитектура

7.3.1. Уровень представления (Клиент)

На уровне представления обеспечивается взаимодействие с пользователем приложения — это пользовательский интерфейс и уровень обмена данными. Его основное предназначение состоит в отображении информации и получении информации от пользователя. Этот уровень может работать в веб-браузере или как графический пользовательский интерфейс компьютерного или мобильного приложения. Уровни представления веб-приложений обычно разрабатываются с помощью HTML, CSS и JavaScript. Компьютерные и мобильные приложения могут быть написаны на любом языке в зависимости от платформы.

7.3.2. Уровень приложений (Сервер приложений)

Уровень приложения, также известный как логический или промежуточный уровень, является центральным звеном приложения. На этом уровне обрабатывается информация, собранная на уровне представления — иногда с учетом другой информации из уровня данных — с помощью бизнес-логики, которая представляет собой набор бизнес-правил. Кроме того, уровень приложения может добавлять, изменять и удалять данные, расположенные на уровне данных.

Как правило, уровень приложения разрабатывается с помощью Python, Java, Perl, PHP или Ruby и взаимодействует с уровнем данных посредством вызовов API.

7.3.3. Уровень данных (Сервер БД)

Уровень данных, который также называется уровнем базы данных, уровнем доступа к данным или базовым уровнем, предназначен для хранения и управления информацией, обработанной приложением. Его роль может выполнять реляционная система управления базами данных, такая как PostgreSQL, MySQL, MariaDB, Oracle, DB2, Informix или Microsoft SQL Server, либо сервер базы данных NoSQL, такой как Cassandra, CouchDB или MongoDB.

В трехуровневом приложении обмен данными осуществляется только через уровень приложения. Уровень представления и уровень данных не могут взаимодействовать друг с другом напрямую.

7.3.4. Слои и уровни

В контексте трехуровневой архитектуры термины уровень (tier) и слой (layer) часто используются как взаимозаменяемые, однако это не совсем справедливо.

Между ними следует делать различие. «Слой» разделяет части программы по функциональному признаку, а «уровни» не только имеют разный

функционал, но и работают в независимой от других уровней инфраструктуре. Например, приложение Контакты на вашем телефоне представляет собой трех-слойное приложение, но вместе с тем оно является одноуровневым, поскольку все три слоя работают на телефоне.

Эта разница имеет важное значение, поскольку слои не позволяют получить те преимущества, которые дает разделение по уровням.

7.3.5. Преимущества трехуровневой архитектуры

Итак, главным преимуществом трехуровневой архитектуры является логическое и физическое разделение функциональных возможностей. Каждый уровень можно запустить в отдельной операционной системе и серверной платформе (веб-сервер, сервер приложений, сервер базы данных), наилучшим образом соответствующей его функциональным требованиям. Поскольку каждый уровень выполняется по крайней мере на одном выделенном аппаратном или виртуальном сервере, уровни можно настраивать и оптимизировать независимо друг от друга.

Другие преимущества (по сравнению с одноуровневой и двухуровневой архитектурой):

- более быстрая разработка: поскольку разработкой уровней одновременно занимаются разные команды, организация может быстрее вывести приложение на рынок, а программисты могут использовать наилучшим образом подходящие языки и инструменты для каждого уровня;
- улучшенная масштабируемость: каждый уровень можно масштабировать независимо от других в соответствии с потребностями;
- повышенная надежность: сбой одного из уровней не повлияет на доступность и производительность других уровней;
- высокий уровень безопасности: поскольку уровень представления и уровень данных не могут напрямую взаимодействовать друг с другом, хорошо спроектированный уровень приложения может выполнять роль своего рода внутреннего брандмауэра, предотвращая внедрение кода SQL и другие вредоносные действия.

7.3.6. Недостатки трехуровневой архитектуры

Недостатки вытекают из достоинств. По сравнению с клиент-серверной или файл-серверной архитектурой можно выделить следующие недостатки трехуровневой архитектуры:

- более высокая сложность создания приложений;

- сложнее в развертывании и администрировании;
- высокие требования к производительности серверов приложений и сервера базы данных, а, значит, и высокая стоимость серверного оборудования;
- высокие требования к скорости канала (сети) между сервером базы данных и серверами приложений.

7.4. Сервис-ориентированная архитектура (SOA)

Сервис-ориентированная архитектура (SOA — service-oriented architecture, рис. 7.3) — это стиль архитектуры программного обеспечения, который предполагает модульное приложение, состоящее из дискретных и слабосвязанных программных агентов, которые выполняют конкретные функции. SOA разделяет компоненты по двум основным ролям: поставщик и потребитель сервисов. Обе эти роли могут играть программные агенты. Концепция SOA заключается в следующем: приложение может быть спроектировано и построено таким образом, что его модули легко интегрируются и могут быть легко использованы повторно.

Consumers Layer

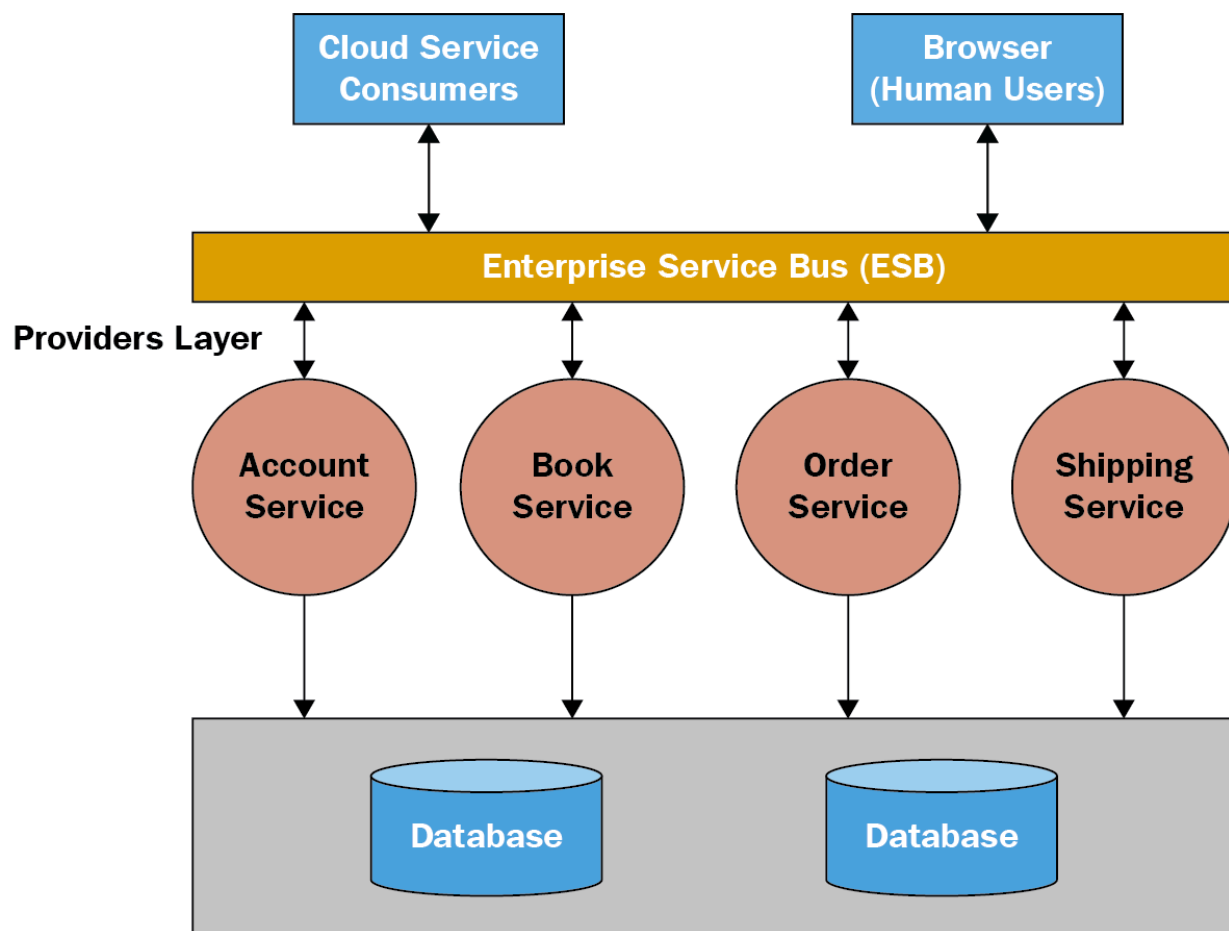


Рисунок 7.3. Сервис-ориентированная архитектура

7.4.1. Плюсы SOA

Повторное использование сервисов. Из-за автономной и слабо связанной природы функциональных компонентов в сервис-ориентированных приложениях эти компоненты можно повторно использовать в нескольких приложениях, без влияния на другие сервисы.

Легкость в сопровождении. Поскольку каждая служба программного обеспечения является независимой единицей, ее легко обновлять и поддерживать, не затрагивая другие службы. Например, крупными корпоративными приложениями легче управлять, когда они разбиты на службы.

Более высокая надежность. Службы легче отлаживать и тестировать, чем огромные куски кода, как в монолитах. Это, в свою очередь, делает продукты на основе SOA более надежными.

Параллельная разработка. Поскольку сервис-ориентированная архитектура разбита на прослойки, она поддерживает параллелизм в процессе разработки. Независимые сервисы могут разрабатываться параллельно и быть завершены одновременно.

7.4.2. Минусы SOA

Сложность в управлении. Основным недостатком сервис-ориентированной архитектуры является ее сложность. Каждый сервис должен обеспечивать своевременную доставку сообщений. Количество этих сообщений может превышать миллион за один раз, что затрудняет управление всеми службами.

Высокие инвестиционные затраты. Разработка SOA требует значительных предварительных инвестиций в человеческие ресурсы, технологии и разработку.

Дополнительная нагрузка. В SOA все входные данные проверяются до того, как один сервис взаимодействует с другим сервисом. При использовании нескольких сервисов это увеличивает время отклика и снижает общую производительность.

7.5. Микросервисная архитектура

Микросервисная архитектура — вариант сервис-ориентированной архитектуры программного обеспечения, распространенный подход к разработке программного обеспечения, когда приложение разбивается на небольшие автономные компоненты (микросервисы) с четко определенными интерфейсами. Именно эта архитектура характерна для cloud-native приложений, которые сейчас популярны благодаря преимуществам, что открывают для бизнеса облачные среды.

Микросервисную архитектуру часто сравнивают с устоявшимся стилем разработки приложений — монолитной архитектурой. И тот, и тот подход в разработке имеют свои плюсы и минусы. Однако очень важным свойством современного приложения является возможность оперативно применить нужные обновления. При монолитной архитектуре приложение выглядит, как единый, целый организм. Поэтому, когда нужно внести какое-то обновление или изменение в монолитную программу, то, фактически, нужно переписывать весь код программы, что является долгим по времени и финансово затратным мероприятием.

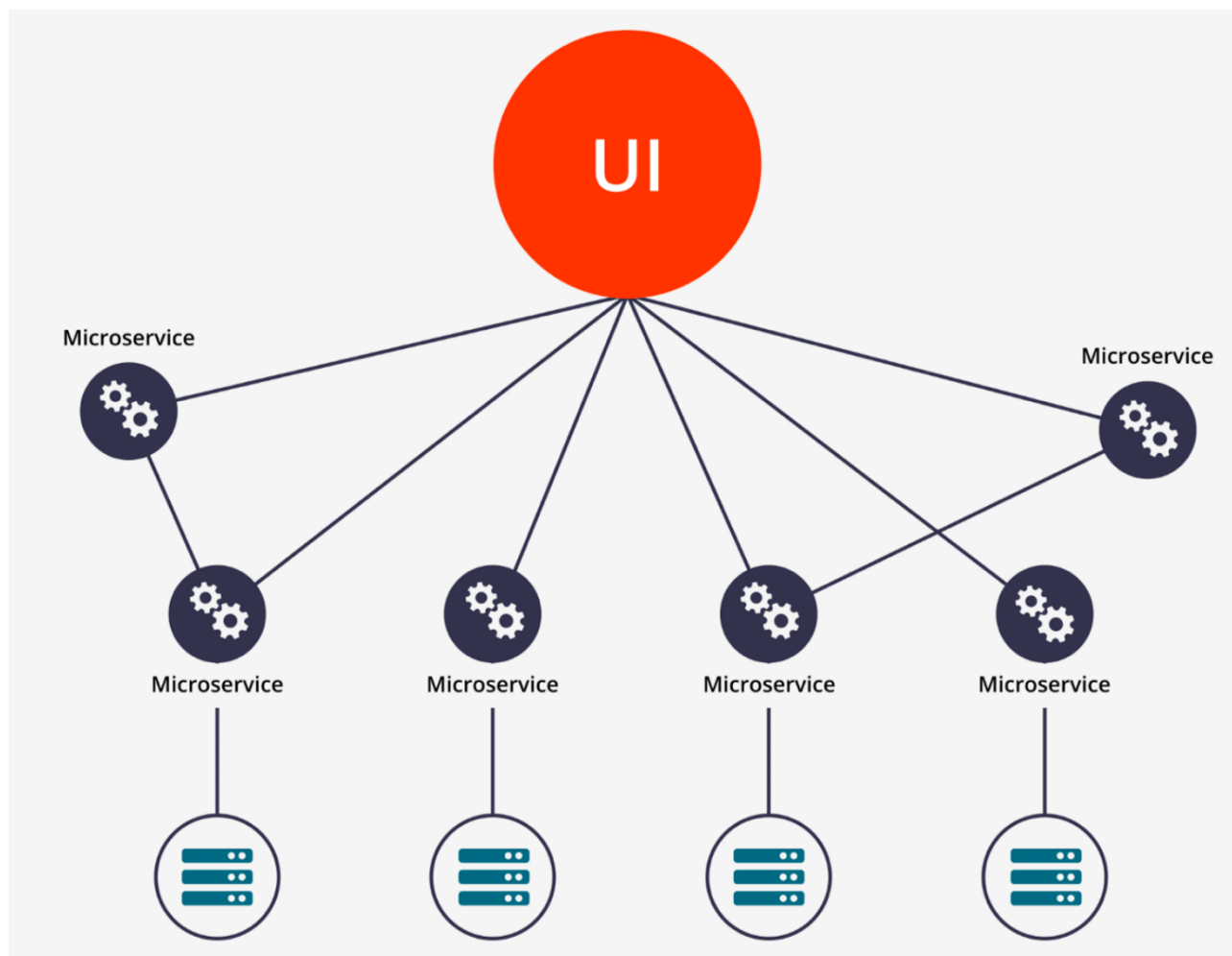


Рисунок 7.4. Микросервисная архитектура

В микросервисах все организовано по-другому. Там много разных уникальных модулей со своей системой связи между ними. Иногда один модуль — это одна какая-то отдельная функция программы. Поэтому, когда нужно обновить что-то в программе, то работа осуществляется с отдельным модулем, который отвечает за обновляемую функцию.

При монолитной архитектуре, если случайным образом удалить какую-то функцию приложения, то в результате мы получим «поломанное» и нерабочее приложение. При микросервисной архитектуре, если мы случайным образом

удалим модуль с какой-либо функцией приложения, то в результате получим стабильно работающее приложение, но без удаленной функции.

Благодаря такому подходу к разработке на 2021-й год микросервисная архитектура находится в фокусе внимания как у небольших разработчиков, так и у IT-гигантов наподобие:

- google;
- amazon;
- netflix;
- twitter и др.

7.5.1. Плюсы микросервисной архитектуры

Среди достоинств микросервисной архитектуры можно отметить такие, как:

Удобная масштабируемость проектов, которая требует меньше трудозатрат.

Улучшенная безопасность программы, за счет того, что каждый отдельный модуль — это самостоятельный элемент программы. В случае выхода из строя или атаки на какой-нибудь модуль, вся остальная часть программы будет работать и находиться в безопасности.

Отсутствие приверженности к определенному стеку технологий. Фактически, каждый отдельный модуль программы может быть разработан разными людьми и на разных языках программирования, что практически нереально при монолитном подходе.

Более легкое обслуживание и понимание проекта. К примеру, если приходит в проект новый разработчик, то ему нужно изучить лишь модуль, за который он будет отвечать и над которым будет работать. Ему вовсе не нужно изучать весь код программы.

Повторная реализация компонентов. То есть, разработав приложение на микросервисной архитектуре, вы в дальнейшем можете использовать его отдельные модули в других своих разработках.

7.5.2. Минусы микросервисной архитектуры

Из недостатков микросервисной архитектуры можно отметить такие, как:

Сложная распределенная система. Наличие множества отдельных модулей влечет за собой дополнительную сложность в организационном и архитектурном плане. То есть усложняется контроль над различными командами разработчиков, а также само развертывание программного продукта.

Усложненное тестирование. Потому что для начала нужно будет тестировать каждый модуль отдельно, а потом совместную функциональность всех модулей.

Приложения на микросервисной архитектуре менее производительны, чем их аналоги на монолитной архитектуре за счет наличия множества баз данных и множества более «длинных» путей связи между отдельными модулями.

7.6. Концептуальная архитектура (для игр)

Для разработки игры мы предлагаем сделать архитектуру на уровне представлений слоев будущей игры (рис. 7.5). Для примера была взята игра “2Д Змейка”.

- слой инициализации;
- слой интерфейса;
- слой логики*;
- слой контроллеров;
- слой управления.

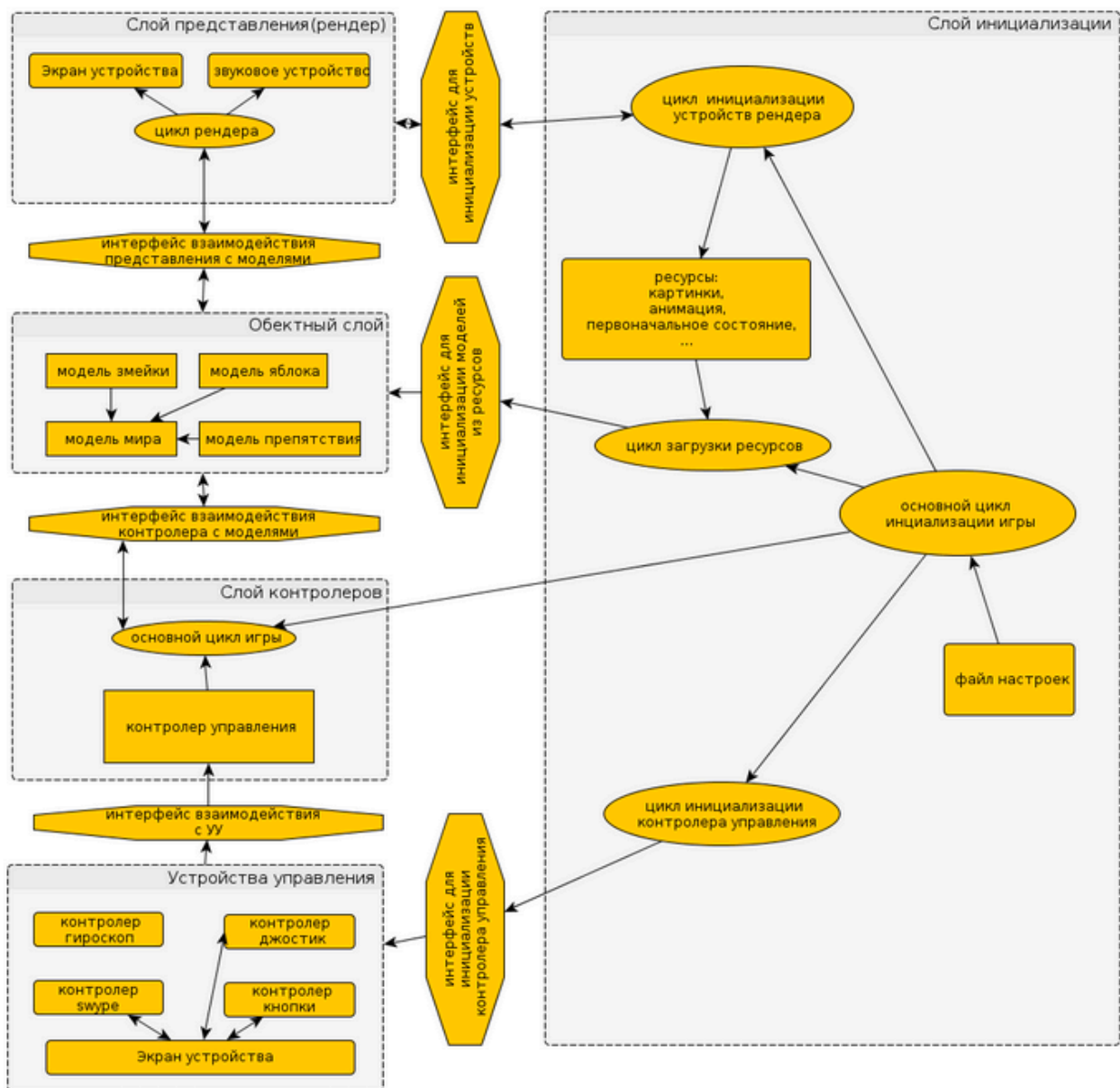


Рисунок 7.5. Концептуальная архитектура

Слой логики (рис. 7.6) лучше описывать в нотация ERD, однако, необходимо будет использовать не совсем стандартную версию нотации. На примере игры на рис. 7.5 представлены все объекты игровой логики и логические связи между ними. Каждый такой объект порождает один класс, обеспечивающий работу этого объекта.

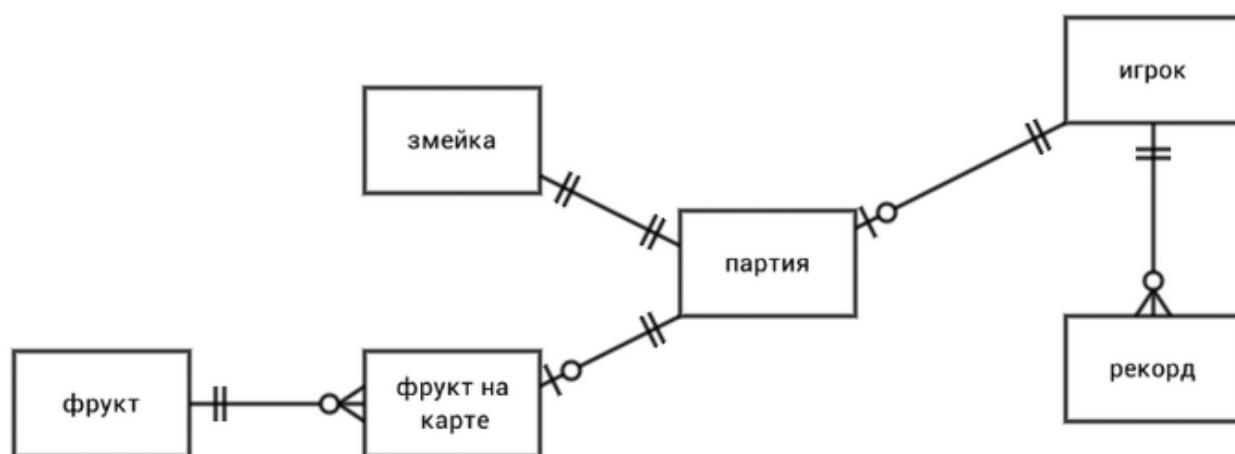


Рисунок 7.6. Слой логики игры

7.7. Дополнение матрицы требований

После того, как будет спроектирована архитектурная диаграмма, необходимо дополнить матрицу требований. В матрице (табл. 7.1) добавляется новый столбец с названием “Компоненты архитектуры”. Соответственно, необходимо указать в данном столбце при помощи каких компонентов системы реализовано то или иное требование.

Если после проектирования архитектурной диаграммы изменилось количество модулей, то необходимо также исправить матрицу требований под эти изменения.

Таблица 7.1. Матрица требований

№	Требование	Суть	Автор	Ссылки	Критерий проверки	Компоненты архитектуры
1	Мобильное приложение клиента					
1.1	Регистрация пользователя	“Приложение должно иметь функцию регистрации нового пользователя”	Иванов И.И.	https://habr.com/ru/company/inforpulse/blog/346318/	Регистрация нового пользователя	Ввод данных реализован на устройстве клиента, обработка происходит на сервере, хранение в единой базе данных
1.2	Авторизация пользователя	“Приложение должно иметь функцию авторизации зарегистрированного пользователя”	Иванов И.И.	https://habr.com/ru/company/inforpulse/blog/346318/	Авторизация зарегистрированного пользователя в приложении	Ввод данных реализован на устройстве клиента, обработка происходит на сервере, хранение в единой базе данных

Продолжение табл. 7.1.

№	Требование	Суть	Автор	Ссылки	Критерий проверки	Компоненты архитектуры
1.3	Услуга “Вызов такси”	“Приложение должно иметь возможность заказа такси для авторизованного клиента”	Иванов И.И.	http://www.consultant.ru/document/cons_doc_LAW_85364/42005365849f53f10d570b0e31c04a1e6fb5080e/	Вызов такси	Ввод данных реализован на устройстве клиента, обработка происходит на сервере, хранение в единой базе данных
...	...	...	...	...	...	
1.*	Расчет примерного времени маршрута	"Приложение должно иметь возможность примерного расчёта времени маршрута на базе сервиса Яндекс.Карты"	Иванов И.И.	https://yandex.ru/blog/mapsapi/38211	Возможность расчёта примерного времени маршрута	Вывод данных реализован на устройстве клиента, обработка происходит на сервере, хранение в единой базе данных, расчет производится на API Яндекс.Карты
2	Веб-интерфейс					
2.1	Регистрация пользователя	“Приложение должно иметь функцию регистрации нового пользователя”	Петров А.А.	https://habr.com/ru/company/inforpulse/blog/346318/	Регистрация нового пользователя	Ввод данных реализован на устройстве клиента в браузере, обработка происходит на сервере, хранение в единой базе данных.
2.2	Авторизация пользователя	“Приложение должно иметь функцию авторизации зарегистрированного пользователя”	Петров А.А.	https://habr.com/ru/company/inforpulse/blog/346318/	Авторизация зарегистрированного пользователя на сайте	Ввод данных реализован на устройстве клиента в браузере, обработка происходит на сервере, хранение в единой базе данных.

Продолжение табл. 7.1.

№	Требование	Суть	Автор	Ссылки	Критерий проверки	Компоненты архитектуры
...	...	...	...	...	...	...
2.*	Завершение маршрута и оценка	“В приложении должна быть функция подтверждения успешного выполнения услуги, а также оценка качества”	Петров А.А.	https://habr.com/ru/post/305374/	Завершение маршрута и оценка качества услуги	Ввод данных реализован на устройстве клиента в браузере, обработка происходит на сервере, хранение в единой базе данных.
3	Приложение для водителя					
3.1	Авторизация водителя	“Приложение должно иметь функцию авторизации зарегистрированного (вручную модератором) водителя”	Сидоров С.С.	https://habr.com/ru/company/infopulse/blog/346318/	Авторизация водителя в приложении	Ввод данных реализован на устройстве водителя, в специальном приложении, обработка происходит на сервере, хранение в единой базе данных.
3.2	Вывод контактной информации о клиенте	"Приложение должно иметь функцию вывода контактной информации о клиенте, для возможности связи"	Сидоров С.С.	http://www.consultant.ru/document/cons_doc_LAW_61801/	Вывод контактной информации выбранного клиента	При выборе данной опции из базы данных берется контактная информация клиента и выводится на устройство водителя.
...	...	...	...	...	...	
3.*	Расчет оптимального маршрута заказа	"Приложение должно иметь функцию для прокладки оптимального маршрута для выполнения заказа с учетом транспортной ситуации на дорогах в текущий момент времени"	Сидоров С.С.	https://yandex.ru/blog/mapsapi/38211	Вывод оптимального маршрута следования	Вывод данных на устройстве водителя о предстоящем заказе, получение данных о маршруте из API Яндекс.Карты и данных о клиенте.
4	Правовые нормы регулирования деятельности компании					

Продолжение табл. 7.1.

№	Требование	Суть	Автор	Ссылки	Критерий проверки	Компоненты архитектуры
4.1	Закон "О защите персональных данных"	Программная система, при получении, хранении и обработки персональных данных клиентов и сотрудников компании, должна руководствоваться нормами закона Федерального закона "О персональных данных" от 27.07.2006 N 152-ФЗ (последняя редакция)	Васильев В.В.	http://www.consultant.ru/document/cons_doc_LAW_61801/	Прохождение теста на невозможность получения закрытых персональных данных третьими лицами	Выполнение норм должны обеспечивать все компоненты программной системы.
...	...	...	...	...	...	
4.*	Выполнение правил перевозки пассажиров и багажа	Система должна по возможности помогать исполнителю выполнять требования Постановления Правительства РФ от 14 февраля 2009 г. N 112 "Об утверждении Правил перевозок пассажиров и багажа автомобильным транспортом и городским наземным электрическим транспортом" (с изменениями и дополнениями)	Васильев В.В.	http://77.rosпотребнадзор.ru/index.php/napravlenie/zpp/6673-pamyatka-potrebite-lyu-po-perevozke-passazhira-legkovym-taksi https://base.garant.ru/195015/	Не должно быть выявлено нарушений	Выполнение норм должны обеспечивать все компоненты программной системы.
5	Серверная часть и БД					

Окончание табл. 7.1.

№	Требование	Суть	Автор	Ссылки	Критерий проверки	Компоненты архитектуры
5.1	Выдерживание нагрузок на сервер	Серверная часть и БД должны выдерживать нагрузки до 1000 одновременно открытых заказов, до 50 одновременно работающих водителей. Данные взяты с запасом, в штате находится 8 водителей, оборот заказов в сутки не более 500	Иванов И.И.	https://firstvds.ru/technology/nagruzka-na-server-opredelenie-prichin	Система должна выдерживать тесты по пиковой нагрузке, которые соответствуют данным в описании требования	Увеличение допустимых пиковых нагрузок на сервер достигается с помощью улучшения характеристик серверного оборудования, а также улучшения методов работы с данными
...	...	...	...	...	...	
5.*	Стрессоустойчивость системы к аварийным ситуациям	Время восстановление системы после аварийной ситуации не должно превышать 30 минут	Иванов И.И.	https://m.habr.com/ru/company/dataline/blog/501848/	Восстановление серверной части системы не превышает 30 минут	Стрессоустойчивость системы достигается подведением системы к сертификации дата-центра по стандарту Tier от Uptime Institute (уровень не ниже Tier II)

7.8. Задание

1. Необходимо описать предлагаемую архитектуру системы и обосновать выбор определенных программных решений для реализации ее компонентов. Программные решения должны включать в себя языки реализации будущей системы, планируемые к применению фреймворки, движки, базы данных и т.д.
2. Построить архитектурную диаграмму своей разработки.
3. Отразить в матрице требований соответствие каждого требования компоненту архитектуры, в котором оно реализуется.

8. ФОРМИРОВАНИЕ ТЕХНИЧЕСКОГО ЗАДАНИЯ

Техническое задание (далее – ТЗ) – основной документ проекта, которым Заявитель устанавливает цели и задачи проекта, номенклатуру и назначение продуктов проекта, технические и иные значимые характеристики проектируемого производства и/или продукта проекта, порядок и последовательность необходимых стадий реализации проекта, создания продукта проекта (в том числе описание технологии) и контроля его качественных параметров.

Техническое задание должно давать исчерпывающее описание соответствия заявленного проекта требованиям, описанным в прошедших практиках собственного проекта.

8.1. ГОСТ 34.602-2020

ТЗ на АС содержит следующие разделы, которые могут быть разделены на подразделы:

- общие сведения;
- назначение и цели создания (развития) системы;
- характеристика объектов автоматизации;
- требования к системе;
- состав и содержание работ по созданию системы;
- порядок контроля и приемки системы;
- требования к составу и содержанию работ по подготовке объекта автоматизации к вводу системы в действие;
- требования к документированию;
- источники разработки.

В ТЗ на АС могут включаться приложения.

В зависимости от вида, назначения, специфических особенностей объекта автоматизации и условий функционирования системы допускается оформлять разделы ТЗ в виде приложений, вводить дополнительные, исключать или объединять подразделы ТЗ.

В ТЗ на части системы не включают разделы, дублирующие содержание разделов ТЗ на АС в целом.

8.1.1. Наполнение разделов в ГОСТ 34.602-2020

В разделе «Общие сведения» указывают:

- полное наименование системы и ее условное обозначение;
- шифр темы или шифр (номер) договора;

- наименование предприятий (объединений) разработчика и заказчика (пользователя) системы и их реквизиты;
- перечень документов, на основании которых создается система, кем и когда утверждены эти документы;
- плановые сроки начала и окончания работы по созданию системы;
- сведения об источниках и порядке финансирования работ;
- порядок оформления и предъявления заказчику результатов работ по созданию системы (ее частей), по изготовлению и наладке отдельных средств (технических, программных, информационных) и программно-технических (программно-методических) комплексов системы.

Раздел «Назначение и цели создания (развития) системы» состоит из подразделов:

- назначение системы;
- цели создания системы.

В подразделе «Назначение системы» указывают вид автоматизируемой деятельности (управление, проектирование и т. п.) и перечень объектов автоматизации (объектов), на которых предполагается ее использовать.

Для АСУ дополнительно указывают перечень автоматизируемых органов (пунктов) управления и управляемых объектов.

В подразделе «Цели создания системы» приводят наименования и требуемые значения технических, технологических, производственно-экономических или других показателей объекта автоматизации, которые должны быть достигнуты в результате создания АС, и указывают критерии оценки достижения целей создания системы.

В разделе «Характеристики объекта автоматизации» приводят:

- краткие сведения об объекте автоматизации или ссылки на документы, содержащие такую информацию;
- сведения об условиях эксплуатации объекта автоматизации и характеристиках окружающей среды.

***Примечание:** Для САПР в разделе дополнительно приводят основные параметры и характеристики объектов проектирования.*

Раздел «Требования к системе» состоит из следующих подразделов:

- требования к системе в целом;
- требования к функциям (задачам), выполняемым системой;
- требования к видам обеспечения.

Состав требований к системе, включаемых в данный раздел ТЗ на АС, устанавливают в зависимости от вида, назначения, специфических особенностей и условий функционирования конкретной системы. В каждом подразделе приводят ссылки на действующие НТД, определяющие требования к системам соответствующего вида.

В подразделе «Требования к системе в целом» указывают:

- требования к структуре и функционированию системы;
- требования к численности и квалификации персонала системы и режиму его работы;
- показатели назначения;
- требования к надежности;
- требования безопасности;
- требования к эргономике и технической эстетике;
- требования к транспортабельности для подвижных АС;
- требования к эксплуатации, техническому обслуживанию, ремонту и хранению компонентов системы;
- требования к защите информации от несанкционированного доступа;
- требования по сохранности информации при авариях;
- требования к защите от влияния внешних воздействий;
- требования к патентной чистоте;
- требования по стандартизации и унификации;
- дополнительные требования.

В требованиях к структуре и функционированию системы приводят:

- перечень подсистем, их назначение и основные характеристики, требования к числу уровней иерархии и степени централизации системы;
- требования к способам и средствам связи для информационного обмена между компонентами системы;
- требования к характеристикам взаимосвязей создаваемой системы со смежными системами, требования к ее совместимости, в том числе указания о способах обмена информацией (автоматически, пересылкой документов, по телефону и т. п.);
- требования к режимам функционирования системы;
- требования по диагностированию системы;
- перспективы развития, модернизации системы.

В требованиях к численности и квалификации персонала на АС приводят:

- требования к численности персонала (пользователей) АС;
- требования к квалификации персонала, порядку его подготовки и контроля знаний и навыков;
- требуемый режим работы персонала АС.

В требованиях к показателям назначения АС приводят значения параметров, характеризующие степень соответствия системы ее назначению.

Для АСУ указывают:

- степень приспособляемости системы к изменению процессов и методов управления, к отклонениям параметров объекта управления;
- допустимые пределы модернизации и развития системы;
- вероятностно-временные характеристики, при которых сохраняется целевое назначение системы.

В требования к надежности включают:

- состав и количественные значения показателей надежности для системы в целом или ее подсистем;
- перечень аварийных ситуаций, по которым должны быть регламентированы требования к надежности, и значения соответствующих показателей;
- требования к надежности технических средств и программного обеспечения;
- требования к методам оценки и контроля показателей надежности на разных стадиях создания системы в соответствии с действующими нормативно-техническими документами.

В требования по безопасности включают требования по обеспечению безопасности при монтаже, наладке, эксплуатации, обслуживании и ремонте технических средств системы (защита от воздействий электрического тока, электромагнитных полей, акустических шумов и т. п.), по допустимым уровням освещенности, вибрационных и шумовых нагрузок.

В требования по эргономике и технической эстетике включают показатели АС, задающие необходимое качество взаимодействия человека с машиной и комфортность условий работы персонала.

Для подвижных АС в требования к транспортабельности включают конструктивные требования, обеспечивающие транспортабельность технических средств системы, а также требования к транспортным средствам.

В требования к эксплуатации, техническому обслуживанию, ремонту и хранению включают:

- условия и регламент (режим) эксплуатации, которые должны обеспечивать использование технических средств (ТС) системы с заданными техническими показателями, в том числе виды и периодичность обслуживания ТС системы или допустимость работы без обслуживания;
- предварительные требования к допустимым площадям для размещения персонала и ТС системы, к параметрам сетей энергоснабжения и т. п.;
- требования по количеству, квалификации обслуживающего персонала и режимам его работы;
- требования к составу, размещению и условиям хранения комплекта запасных изделий и приборов;
- требования к регламенту обслуживания.

В требования к защите информации от несанкционированного доступа включают требования, установленные в НТД, действующей в отрасли (ведомстве) заказчика.

В требованиях по сохранности информации приводят перечень событий: аварий, отказов технических средств (в том числе - потеря питания) и т. п., при которых должна быть обеспечена сохранность информации в системе.

В требованиях к средствам защиты от внешних воздействий приводят:

- требования к радиоэлектронной защите средств АС;
- требования по стойкости, устойчивости и прочности к внешним воздействиям (среде применения).

В требованиях по патентной чистоте указывают перечень стран, в отношении которых должна быть обеспечена патентная чистота системы и ее частей.

В требования к стандартизации и унификации включают: показатели, устанавливающие требуемую степень использования стандартных, унифицированных методов реализации функций (задач) системы, поставляемых программных средств, типовых математических методов и моделей, типовых проектных решений, унифицированных форм управленческих документов, установленных ГОСТ 6.10.1, общесоюзных классификаторов технико-экономической информации и классификаторов других категорий в соответствии с областью их применения, требования к использованию типовых автоматизированных рабочих мест, компонентов и комплексов.

В дополнительные требования включают:

- требования к оснащению системы устройствами для обучения персонала (тренажерами, другими устройствами аналогичного назначения) и документацией на них;
- требования к сервисной аппаратуре, стендам для проверки элементов системы;
- требования к системе, связанные с особыми условиями эксплуатации;
- специальные требования по усмотрению разработчика или заказчика системы.

В подразделе «Требование к функциям (задачам)», выполняемым системой, приводят:

- по каждой подсистеме перечень функций, задач или их комплексов (в том числе обеспечивающих взаимодействие частей системы), подлежащих автоматизации;
- при создании системы в две или более очереди - перечень функциональных подсистем, отдельных функций или задач, вводимых в действие в 1-й и последующих очередях;
- временной регламент реализации каждой функции, задачи (или комплекса задач);
- требования к качеству реализации каждой функции (задачи или комплекса задач), к форме представления выходной информации, характеристики необходимой точности и времени выполнения, требования одновременности выполнения группы функций, достоверности выдачи результатов;
- перечень и критерии отказов для каждой функции, по которой задаются требования по надежности.

В подразделе «Требования к видам обеспечения» в зависимости от вида системы приводят требования к математическому, информационному, лингвистическому, программному, техническому, метрологическому, организационному, методическому и другие видам обеспечения системы.

Для математического обеспечения системы приводят требования к составу, области применения (ограничения) и способам, использования в системе математических методов и моделей, типовых алгоритмов и алгоритмов, подлежащих разработке.

Для информационного обеспечения системы приводят требования:

- к составу, структуре и способам организации данных в системе;
- к информационному обмену между компонентами системы;

- к информационной совместимости со смежными системами;
- по использованию общесоюзных и зарегистрированных республиканских, отраслевых классификаторов, унифицированных документов и классификаторов, действующих на данном предприятии;
- по применению систем управления базами данных;
- к структуре процесса сбора, обработки, передачи данных в системе и представлению данных;
- к защите данных от разрушений при авариях и сбоях в электропитании системы;
- к контролю, хранению, обновлению и восстановлению данных;
- к процедуре придания юридической силы документам, продуцируемым техническими средствами автоматизированных систем.

Для лингвистического обеспечения системы приводят требования к применению в системе языков программирования высокого уровня, языков взаимодействия пользователей и технических средств системы, а также требования к кодированию и декодированию данных, к языкам ввода-вывода данных, языкам манипулирования данными, средствам описания предметной области (объекта автоматизации), к способам организации диалога.

Для программного обеспечения системы приводят перечень покупных программных средств, а также требования:

- к независимости программных средств от используемых СВТ и операционной среды;
- к качеству программных средств, а также к способам его обеспечения и контроля;
- по необходимости согласования вновь разрабатываемых программных средств с фондом алгоритмов и программ.

Для технического обеспечения системы приводят требования:

- к видам технических средств, в том числе к видам комплексов технических средств, программно-технических комплексов и других комплектующих изделий, допустимых к использованию в системе;
- к функциональным, конструктивным и эксплуатационным характеристикам средств технического обеспечения системы.

В требованиях к метрологическому обеспечению приводят:

- предварительный перечень измерительных каналов;
- требования к точности измерений параметров и (или) к метрологическим характеристикам измерительных каналов;

- требования к метрологической совместимости технических средств системы;
- перечень управляющих и вычислительных каналов системы, для которых необходимо оценивать точностные характеристики;
- требования к метрологическому обеспечению технических и программных средств, входящих в состав измерительных каналов системы, средств, встроенного контроля, метрологической пригодности измерительных каналов и средств измерений, используемых при наладке и испытаниях системы;
- вид метрологической аттестации (государственная или ведомственная) с указанием порядка ее выполнения и организаций, проводящих аттестацию.

Для организационного обеспечения приводят требования:

- к структуре и функциям подразделений, участвующих в функционировании системы или обеспечивающих эксплуатацию;
- к организации функционирования системы и порядку взаимодействия персонала АС и персонала объекта автоматизации;
- к защите от ошибочных действий персонала системы.

Для методического обеспечения САПР приводят требования к составу нормативно-технической документации системы (перечень применяемых при ее функционировании стандартов, нормативов, методик и т. п.).

Раздел «Состав и содержание работ по созданию (развитию) системы» должен содержать перечень стадий и этапов работ по созданию системы, сроки их выполнения, перечень организаций - исполнителей работ, ссылки на документы, подтверждающие согласие этих организаций на участие в создании системы, или запись, определяющую ответственного (заказчик или разработчик) за проведение этих работ.

В данном разделе также приводят:

- перечень документов, предъявляемых по окончании соответствующих стадий и этапов работ;
- вид и порядок проведения экспертизы технической документации (стадия, этап, объем проверяемой документации, организация-эксперт);
- программу работ, направленных на обеспечение требуемого уровня надежности разрабатываемой системы (при необходимости);

- перечень работ по метрологическому обеспечению на всех стадиях создания системы с указанием их сроков выполнения и организаций-исполнителей (при необходимости).

В разделе «Порядок контроля и приемки системы» указывают:

- виды, состав, объем и методы испытаний системы и ее составных частей (виды испытаний в соответствии с действующими нормами, распространяющимися на разрабатываемую систему);
- общие требования к приемке работ по стадиям (перечень участвующих предприятий и организаций, место и сроки проведения), порядок согласования и утверждения приемочной документации;
- статус приемочной комиссии (государственная, межведомственная, ведомственная).

В разделе «Требования к составу и содержанию работ по подготовке объекта автоматизации к вводу системы в действие» необходимо привести перечень основных мероприятий и их исполнителей, которые следует выполнить при подготовке объекта автоматизации к вводу АС в действие.

В перечень основных мероприятий включают:

- приведение поступающей в систему информации (в соответствии с требованиями к информационному и лингвистическому обеспечению) к виду, пригодному для обработки с помощью ЭВМ;
- изменения, которые необходимо осуществить в объекте автоматизации;
- создание условий функционирования объекта автоматизации, при которых гарантируется соответствие создаваемой системы требованиям, содержащимся в ТЗ;
- создание необходимых для функционирования системы подразделений и служб;
- сроки и порядок комплектования штатов и обучения персонала.

Например, для АСУ приводят:

- изменения применяемых методов управления;
- создание условий для работы компонентов АСУ, при которых гарантируется соответствие системы требованиям, содержащимся в ТЗ.

В разделе «Требования к документированию» приводят:

- согласованный разработчиком и Заказчиком системы перечень подлежащих разработке комплектов и видов документов, соответствующих требованиям ГОСТ 34.201-89 и НТД отрасли заказчика;

перечень документов, выпускаемых на машинных носителях; требования к микрофильмированию документации;

- требования по документированию комплектующих элементов межотраслевого применения в соответствии с требованиями ЕСКД и ЕСПД;
- при отсутствии государственных стандартов, определяющих требования к документированию элементов системы, дополнительно включают требования к составу и содержанию таких документов.

В разделе «Источники разработки» должны быть перечислены документы и информационные материалы (техико-экономическое обоснование, отчеты о законченных научно-исследовательских работах, информационные материалы на отечественные, зарубежные системы-аналоги и др.), на основании которых разрабатывалось ТЗ и которые должны быть использованы при создании системы.

В состав ТЗ на АС при наличии утвержденных методик включают приложения, содержащие:

- расчет ожидаемой эффективности системы;
- оценку научно-технического уровня системы.

Приложения включают в состав ТЗ на АС по согласованию между разработчиком и заказчиком системы.

8.2. ГОСТ 19.201-78

Техническое задание должно содержать следующие разделы:

- введение;
- основания для разработки;
- назначение разработки;
- требования к программе или программному изделию;
- требования к программной документации;
- технико-экономические показатели;
- стадии и этапы разработки;
- порядок контроля и приемки;
- в техническое задание допускается включать приложения.

В зависимости от особенностей программы или программного изделия допускается уточнять содержание разделов, вводить новые разделы или объединять отдельные из них.

В разделе "Введение" указывают наименование, краткую характеристику области применения программы или программного изделия и объекта, в котором используют программу или программное изделие.

В разделе "Основание для разработки" должны быть указаны:

- документ (документы), на основании которых ведется разработка;
- организация, утвердившая этот документ, и дата его утверждения;
- наименование и (или) условное обозначение темы разработки.

В разделе "Назначение разработки" должно быть указано функциональное и эксплуатационное назначение программы или программного изделия.

Раздел "Требования к программе или программному изделию" должен содержать следующие подразделы:

- требования к функциональным характеристикам;
- требования к надежности;
- условия эксплуатации;
- требования к составу и параметрам технических средств;
- требования к информационной и программной совместимости;
- требования к маркировке и упаковке;
- требования к транспортированию и хранению;
- специальные требования.

В подразделе "Требования к функциональным характеристикам" должны быть указаны требования к составу выполняемых функций, организации входных и выходных данных, временным характеристикам и т.п.

В подразделе "Требования к надежности" должны быть указаны требования к обеспечению надежного функционирования (обеспечение устойчивого функционирования, контроль входной и выходной информации, время восстановления после отказа и т.п.).

В подразделе "Условия эксплуатации" должны быть указаны условия эксплуатации (температура окружающего воздуха, относительная влажность и т.п. для выбранных типов носителей данных), при которых должны обеспечиваться заданные характеристики, а также вид обслуживания, необходимое количество и квалификация персонала.

В подразделе "Требования к составу и параметрам технических средств" указывают необходимый состав технических средств с указанием их основных технических характеристик.

В подразделе "Требования к информационной и программной совместимости" должны быть указаны требования к информационным структурам на входе и выходе и методам решения, исходным кодам, языкам программирования и программным средствам, используемым программой.

При необходимости должна обеспечиваться защита информации и программ.

В подразделе "Требования к маркировке и упаковке" в общем случае указывают требования к маркировке программного изделия, варианты и способы упаковки.

В подразделе "Требования к транспортированию и хранению" должны быть указаны для программного изделия условия транспортирования, места хранения, условия хранения, условия складирования, сроки хранения в различных условиях.

В разделе "Требования к программной документации" должны быть указаны предварительный состав программной документации и, при необходимости, специальные требования к ней.

В разделе "Технико-экономические показатели" должны быть указаны: ориентировочная экономическая эффективность, предполагаемая годовая потребность, экономические преимущества разработки по сравнению с лучшими отечественными и зарубежными образцами или аналогами.

В разделе "Стадии и этапы разработки" устанавливают необходимые стадии разработки, этапы и содержание работ (перечень программных документов, которые должны быть разработаны, согласованы и утверждены), а также, как правило, сроки разработки и определяют исполнителей.

В разделе "Порядок контроля и приемки" должны быть указаны виды испытаний и общие требования к приемке работы.

В приложениях к техническому заданию, при необходимости, приводят:

- перечень научно-исследовательских и других работ, обосновывающих разработку;
- схемы алгоритмов, таблицы, описания, обоснования, расчеты и другие документы, которые могут быть использованы при разработке;
- другие источники разработки.

8.3. Задание

1. Необходимо обосновать выбор ГОСТа для разработки технического задания, почему был выбран тот или иной стандарт, по каким разделам данный ГОСТ подходит больше и т.д.
2. Составить в соответствии с выбором техническое задание по ГОСТу.

9. ОРГАНИЗАЦИЯ РАЗРАБОТКИ

9.1. Описание выбора методологии управления процессом разработки

Командная разработка подразумевает взаимодействие нескольких человек при реализации программного продукта. Чтобы организовать управление процессом разработки были придуманы различные методы. Применение данных методов зависит от входного состояния проекта и подготовленности команды к процессу разработки.

В целом среди методологий можно выделить следующих представителей:

1. Каскадная модель (она же “водопадная” – waterfall).
2. Итерационные модели.
3. Инкрементная модель.
4. Спиральная модель.

По большому счету все методологии можно разделить на две больших группы: последовательные и итерационные.

9.2. Waterfall (каскадная модель)

Основная суть модели **Waterfall** (рис. 9.1) в том, что этапы зависят друг от друга и следующий начинается, когда закончен предыдущий, образуя таким образом поступательное (каскадное) движение вперед.

Параллелизм этапов в каскадной модели, хоть и ограничен, но возможен для абсолютно независимых между собой работ. При этом интеграция параллельных кусков все равно происходит на каком-то следующем этапе, а не в рамках одного.

Команды разных этапов между собой не коммуницируют, каждая команда отвечает четко за свой этап.

Недостатками этой модели являются получение результата по прохождению всех этапов и сложность выявления ошибок. Возвращаться назад трудно. Не понятно, что возвращать: если произошел сбой на каком-то этапе, его последствия видны только в конце.

Для заказчиков данная модель выглядит линейно и со стороны достаточно просто: из завершеного этапа проектирования следует программирование, а затем тестирование – и так шаг за шагом пока не будет достигнута финальная точка и цель, ради которой ведется разработка.

Данная модель понятно и чисто укладывается в документы, например в договора и роадмапы при наличии четко обозначенных контрольных точек. В любой момент времени можно легко понять была ли пройдена та или иная точка контроля или нет, и соблюдены ли сроки. По этим причинам долговременные и особо крупные проекты, рассчитанные на десятилетия и вовлечение большого

числа организаций-участников, руководствуются преимущественно waterfall.

Однако представление о простоте каскадной модели является иллюзорным. Оно появляется из-за ограниченного видения клиентом всего процесса, ведь данная модель не подразумевает вовлечение заказчика в детали процессов разработки, и демонстрирует понятный и конечный результат работы только на контрольных точках и в конце проекта.

В реальности каскадную модель нельзя назвать простой, на практике ею сложно управлять. Внесение заказчиком значительных изменений в процессе разработки по waterfall или срабатывание серьезных не предусмотренных проектом рисков несут разрушительный характер для всего процесса - модель приходится перестраивать, графики перепланировать.

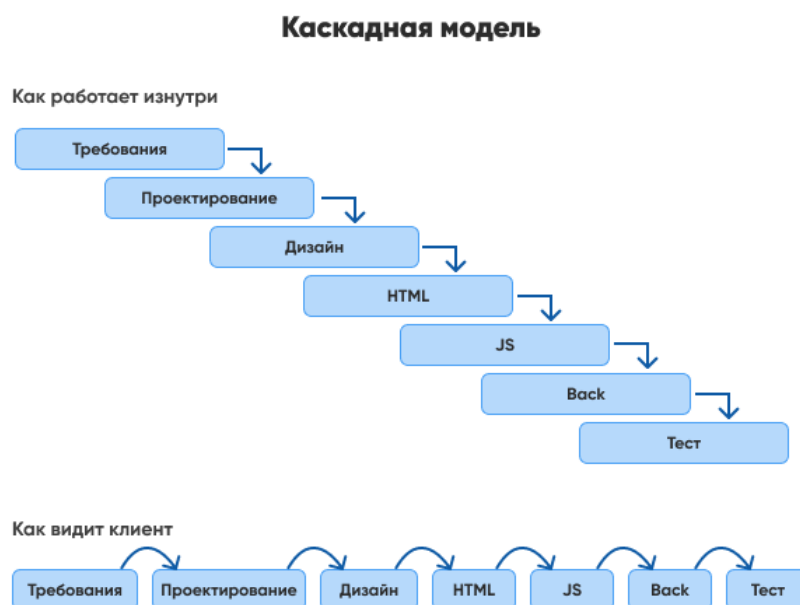


Рисунок 9.1. Каскадная модель

9.3. Итерационная, спиральная и инкрементная модели

Итерационная модель (рис. 9.2) предполагает разбиение проекта на части (этапы, итерации) и прохождение этапов жизненного цикла на каждом из них. Каждый этап является законченным сам по себе, совокупность этапов формирует конечный результат.

Поясним разбиение на этапы на бытовом примере. Допустим, нам нужен стол в гостиную.

- на первом этапе мы сделаем столешницу, ножки и скрепим их так, чтобы стол стоял;
- на втором, укрепим и покрасим его;

- а на третьем, накроем скатертью и купим подходящие к нему стулья.

На каждой итерации мы работали с одним и тем же продуктом и в конце каждой итерации получали результат, которым можно пользоваться (естественно, с определенными ограничениями).

С каждым этапом разработка приближается к конечному желаемому результату или уточняются требования к результату по ходу разработки, и соответственно в любой момент текущая итерация может оказаться последней или очередной на пути к завершению.

Данный подход позволяет бороться с неопределенностью, снимая ее этап за этапом, и проверять правильность технического, маркетингового или любого другого решения на ранних стадиях.

Использование итерационной модели снижает риски глобального провала и растраты всего бюджета, получение несинхронизированных ожиданий и ошибочного понимания процессов как клиентом, так и каждым участником команды разработки. Оно также дает возможность завершения разработки в конце любой итерации (в каскадной модели вы должны прежде завершить все этапы).

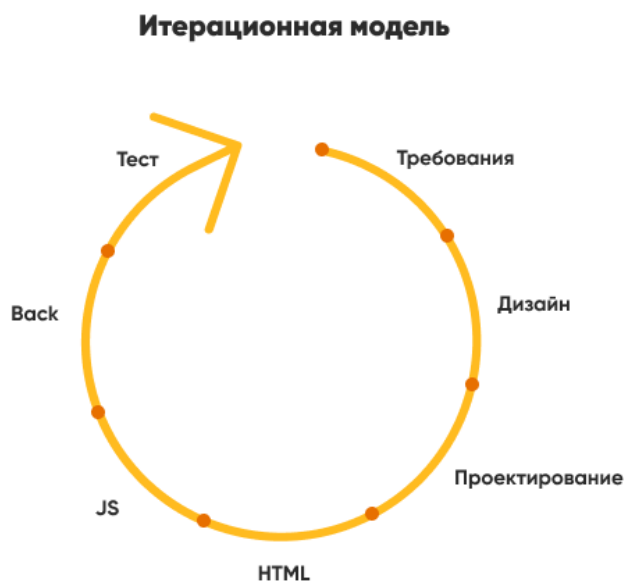


Рисунок 9.2. Итерационная модель

Спиральная и инкрементная модели являются видами итерационной модели жизненного цикла.

9.3.1. Спиральная модель

Все этапы жизненного цикла при спиральной модели (рис. 9.3) идут витками, на каждом из которых происходят проектирование, кодирование, дизайн,

тестирование и т. д. Такой процесс отражает суть названия: поднимаясь, проходит один виток (цикл) спирали для достижения конечного результата. Причем не обязательно, что один и тот же набор процессов будет повторяться от витка к витку. Но результаты каждого из витков ведут к главной цели.

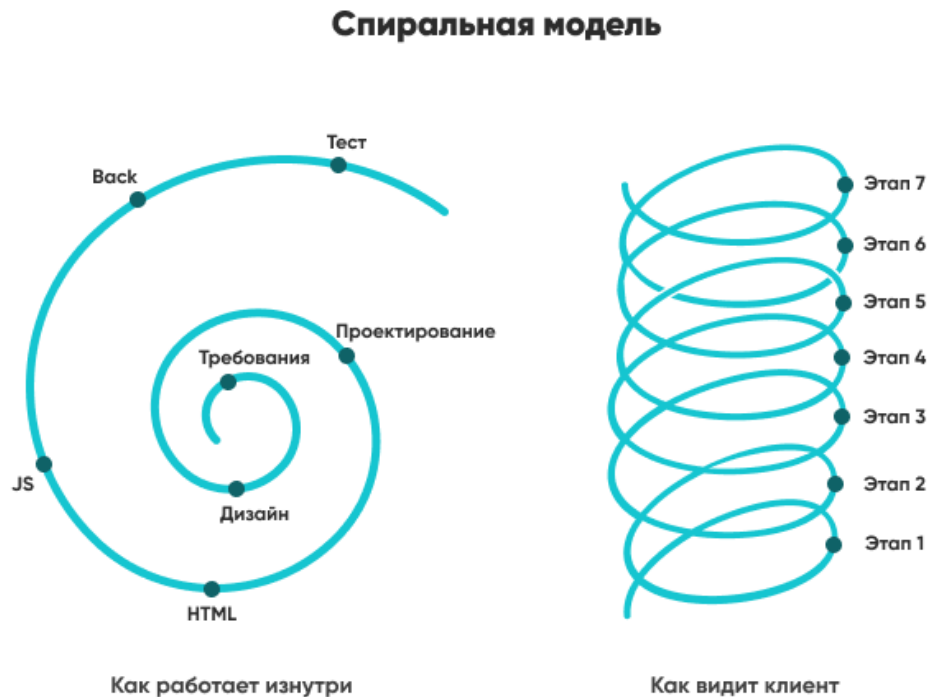


Рисунок 9.3. Спиральная модель

9.3.2. Инкрементная модели

Принцип, который лежит в основе инкрементной модели, подразумевает расширение возможностей, достраивание модулей и функций приложения. Буквальный перевод слова инкремент: «увеличение на один». Это «увеличение на один» применяется в том числе для обозначения версий продукта.

Если в каскадной модели, по сути, есть два состояния продукта: «ничего» и «готовый продукт», то с появлением итерационных моделей стало применяться версионирование продукта. Каждая итерация обозначается цифрой: 1,2,3 и соответственно продукт после каждой итерации имеет версию с соответствующим номером: v.1, v.2, v.3. Числами после слова версия обозначают масштабные изменения в ядро продукта.

А когда одна из версий эксплуатируется, следующая, учитывая недочеты предыдущей, только планируется или уже разрабатывается, а улучшения заказчику и пользователю хочется доставить прямо сейчас, тогда появляются минорные версии. Туда попадают изменения, которые не влияют на ядро разработки и

представлены как под-версии 1.1,1.2,1.3 или релизы 1.1.1, 1.1.2 и т.п.

В совокупности такие поэтапные релизы приводят к полноценной версии 2.0.

9.4. Agile и Lean: принципы разработки ПО

9.4.1. Agile

Agile — набор принципов гибкой разработки (всего их 12) и идей. Основные идеи Agile:

- люди и взаимодействие важнее процессов и инструментов;
- работающий продукт важнее исчерпывающей документации;
- сотрудничество с заказчиком важнее согласования условий контракта;
- готовность к изменениям важнее следования первоначальному плану.

Один из принципов - взаимодействие - подразумевает, что заказчик взаимодействует с командой, команда с заказчиком - все между собой. Это позволяет обмениваться опытом между участниками команды и клиентом и участвовать каждому из них в принятие решений. За счет такого подхода снижаются риски потери времени и денег и повышается способность команды решать сложные нестандартные задачи с высокой степенью неопределенности.

Однако взаимодействия всех и со всеми может вылиться в хаос, что влияет на все сферы разработки. Поэтому используя Agile нужно понимать ограничения: команды должны быть небольшие, участники должны быть компетентные и мотивированные, итерации короткие с максимально понятными целями, установлены четкие ограничения по времени и конечный результат должен быть очевидным.

Agile прекрасно управляется с неопределенностью, предопределяя будущее на более короткий период. Правило такое: чем выше неопределенность - тем короче итерация, причем ее длительность может быть даже в рамках 24 часов, как и происходит на хакатонах. Вначале каждой итерации неизбежно выполняется контроль, ретроспектива, оценка и анализ результатов, планирование следующей итерации.

Во внутреннем планировании и в продуктовой разработке без этого принципа и элементов Agile не обойтись.

9.4.2. Lean

Идея подхода **Lean** в том, что мы бережливо относимся к ресурсам (в том

числе времени) и решаем задачи самым простым способом. Например: мы не делаем весь продукт, чтобы понять, что он никому не нужен – мы делаем лендинг и форму подписки и даем на него рекламу, чтобы проверить что этот продукт вызывает интерес клиентов и принять осознанное решение о необходимости разработки.

Когда доходит до разработки продукта, или делается какое-то улучшение, производственное или инженерное, мы сначала делаем его MVP (minimum viable product). Термин MVP сейчас широко распространён и применяется повсеместно, но он родился именно из Lean подхода. MVP это такая версия продукта, которая выполняет свою главную функцию и при этом её не отторгают клиенты и признают её полезность. Конечно, её можно улучшать и улучшать, но в целом продукт на стадии MVP должен быть полезным, понятным клиенту и таким, чтобы можно было принять решение о его дальнейших улучшениях или признать эксперимент неудачным и тестировать новую гипотезу, потратив при этом как можно меньше ресурсов.

В целом, Lean подход ориентируется на тестирование потребностей и ценностей и попадание в ожидание рынка самыми минимальными средствами. Lean-подход это не о “технических” проблемах и их решениях инженерными средствами, а о предпринимательском подходе к решению задач. Lean предполагает, что ты ищешь самое простое решение для достижения результата: технически, организационно и т.п. упрощая при этом всё что не является действительно важным.

В упрощении сила и главная “ловушка” Lean: стремление всё упростить иногда приводит к ситуациям, в которых продукт упрощают настолько, что теряются действительно важные функции и продукт, по сути, оказывается ненужным, поскольку не несёт ценности пользователю.

В заключение о Lean хочется сказать такое: часто, когда говорят о Lean то говорят, что “Lean это о том, чтобы вместо сервиса сделать лендинг с формой контактов”. Нет, это не Lean. Такое тестирование не противоречит Lean, но Lean-методология предлагает гораздо больше приемов чем этот, и стоит внимательного изучения.

9.5. Основные методы разработки ПО: гибкие методологии

Рассмотрим основные методологии, применяющие гибкий подход к разработке программного обеспечения.

Scrum методология основывается на понятии спринта (sprint), в течении которого выполняется работа над продуктом. Перед началом каждого спринта

проводится планирование (Sprint Planning), на котором производится оценка содержимого списка задач по развитию продукта (Product Backlog) и формирование бэклога на спринт (Sprint Backlog), в рамках которых и действует команда. Для спринта всегда существуют ограничения по времени, обычно от недели до месяца. Жизнь продукта таким образом разбита на равные по продолжительности спринты.

По **Kanban** методологии проект делится на этапы, которые визуализируются в виде канбан-доски. Этапы, это например: планирование, разработка, тестирование, релиз и т.п. Задачи в виде “карточек” перемещаются с этапа на этап. Новые задачи можно добавлять в любое время. Задача закрывается не по истечению конкретного времени, а по смене статуса на “завершено”. Канбан это методология из концепции “бережливого производства”.

RUP (Rational Unified Process) — разработка продукта при данном методе состоит из четырех фаз (начальная стадия, уточнение, построение, внедрение), каждая из которых включает в себя одну или несколько итераций. RUP огромная методология, которую трудно уложить в абзац текста, но методы, рекомендуемые RUP основаны на статистике коммерчески успешных проектов.

DSDM (Dynamic Systems Development Model) — методология, которая демонстрирует набор принципов, predetermined типов ролей и техник.

Принципы направлены на главную цель – сдать готовый проект вовремя и уложиться в бюджет, с возможностью регулировать требования во время разработки. DSDM входит в семейство гибкой методологии разработки программного обеспечения, а также разработок не входящих в сферу информационных технологий.

RAD (Rapid Application Development) — методология быстрой разработки приложений, которая предполагает применение инструментальных средств визуального моделирования (прототипирования) и разработки. RAD предусматривает небольшие команды разработки, сроки до 4 месяцев и активное привлечение заказчика с ранних этапов. Данная методология опирается на требования, но также существует возможность их изменений в период разработки системы. Такой подход позволяет сократить расходы и свести время разработки к минимуму.

XP (Extreme Programming) — методология, которая ориентируется на постоянное изменение требований к продукту и предлагает 12 подходов для достижения эффективных результатов в подобных условиях. Среди них:

- быстрый план и его постоянное изменение;
- простой дизайн архитектуры;
- частое тестирование;

- участие одновременно двух разработчиков в одной задаче или даже за одним рабочим местом;
- постоянная интеграция и частые небольшие релизы.

9.6. Организация процесса разработки

Определившись с методологией управления процессом разработки необходимо также организовать процесс разработки с технической точки зрения. Основой современного подхода к разработке является использование системы контроля версий для управления версионированием ПО, а также для организации взаимодействия между несколькими разработчиками. Одной из самых распространенных систем контроля версий является git, который обладает большим количеством веб-сервисов для хранения совместных репозиторий: GitHub, GitLab, BitBucket и т.д.

Также современная разработка подразумевает использование более сложных, чем обычный Блокнот, инструментов написания кода. Подбор таких инструментов обуславливается предпочтениями конкретного разработчика или же требованиями разрабатываемого проекта.

9.7. Задание

1. Выбрать методологию управления процессом разработки выбранного проекта исходя из потребностей своей команды.
2. Создать удаленный git репозиторий на одном из популярных сервисов (можно использовать другую систему контроля версий при желании).
3. Описать выбранные инструменты разработки программного обеспечения.

10. ДОКУМЕНТИРОВАНИЕ РАЗРАБОТКИ ПО

10.1. Документация разработчика

Сопровождение недокументированного кода является достаточно сложной задачей, затрачивающей большое количество сил разработчика. Документированный программный продукт облегчает задачу доработки программного обеспечения, поскольку сразу становится ясно, какой участок кода за что отвечает без необходимости изучения каждой его строчки.

Несколько основных причин для разработки документации для кода:

- обслуживать код становится достаточно проблематично, если к нему не применялся должный уровень документирования. Особенно это актуально для поддержки использования кодов другими программами, что работают над проектом.
- если потребуется задействовать других программистов посредством технологии открытого кода. При начале масштабной коллективной работы документирование кода становится безотлагательным делом.
- при наличии документирования кода логика заметно проясняется, а код улучшается по своей структуре.
- код, не прошедший документирование, неудобен в работе не только при передаче на сопровождение, но также непросто в самостоятельном сопровождении.

Написание документации к программному обеспечению является достаточно трудоемким процессом, для облегчения которого разрабатываются различные программные средства, позволяющие собрать документацию разработчика из комментариев исходного кода, примером такого ПО может послужить Doxygen.

10.1.1. Doxygen

Вероятнее всего, многие сталкивались с результатами работы различных генераторов документации. Общий принцип их работы, следующий: на вход такого генератора поступает специальным образом комментированный исходный код, а иногда и другие компоненты программы, а на выходе создаётся готовая документация для распространения и использования.

Рассматриваемая система Doxygen как раз и выполняет эту задачу: она позволяет генерировать на основе исходного кода, содержащего комментарии специального вида, читабельного вида документацию, содержащую в себе ссылки, диаграммы классов, вызовов и т.п. в различных форматах: HTML, LaTeX, CHM, RTF, PostScript, PDF, man-страницы.

Данная система поддерживает много различных языков, например: C++, C, Objective-C, C#, PHP, Java, Python, IDL, Fortran, VHDL, Tcl, и частично D.

Doxygen позволяет создавать документацию типа HTML и представляет собой консольную программу наподобие классики в стиле Unix. Принцип работы напоминает компилятор, который анализирует исходные тексты с созданием документации.

Основное преимущество в процессе использования Doxygen подразумевает определённую последовательность документирования исходного кода. Оказывается эффективная помощь при создании структуры кода с применением недокументированного исходного файла. Главное, осуществить надлежащие настройки.

10.1.2. Sphinx

Выступает в роли популярного инструмента для создания документации в текстовом виде и преобразования в различные форматы. Очень удобен при использовании версиями в системах управления, отслеживающими возможные изменения. Доступ возможно получить согласно лицензии BSD. Совместимыми языками программирования являются Python, C, C++. Наравне документацией проектного типа может документироваться код и происходит избавление от рутины, а типовые проблемы разрешаются автоматически. К ним относятся индексирование заголовков и особое выделение кода, к примеру, при присутствии фрагмента кода в документе. При этом определённым образом выделяется синтаксис.

Если система разрабатывается на языке, не входящем в список doxygen или sphinx, можно воспользоваться другими генераторами документации, к примеру, Jazzy для языка Swift.

В итоге, в данном разделе должны быть приведены примеры написанной документации разработчика, показывающие её реализацию. Помимо этого, к работе должен быть приложен архив со сгенерированной документацией.

10.2. Документация пользователя

Документация разработчика позволяет подробно изучить то, как устроено приложение с точки зрения программиста, однако у пользователя также могут возникать вопросы в процессе эксплуатации ПО. Для ответов на эти вопросы должна быть разработана документация пользователя, содержащая инструкции по различным функциям приложения. Достаточно удобным форматом документации является написание Wiki по проекту, которое освещает каждую функцию приложения в отдельности.

10.2.1. FANDOM

Фэндом (англ. Fandom, Fandom powered by Wikia), ранее известный как Викия (англ. Wikia) — бесплатный сервис хостинга, предоставляющий возможность любому желающему создать свой собственный тематический вики-проект или принять участие в коллективной работе над уже существующими проектами.

Уровень разработки содержания в проектах Фэндом разный. На сайте каждый зарегистрированный участник может создать новую вики. Но поскольку желание или возможность их развивать с нуля есть не у всех, большинство из них забрасываются. Проекты-пустышки сразу не удаляются (если длительное время в вики не наблюдается активности, то замораживается база данных, а если никто не заинтересуется дальнейшей судьбой вики, вики удаляется с сохранением истории правок в архиве, если такие были), поэтому на один действующий и обновляющийся проект приходится несколько десятков заброшенных.

Также множество вики были созданы самими участниками для различных тестов, что уже гарантирует тот факт, что развиваться они не будут.

Ранее на сайте было множество проектов-дубликатов, созданных по обыкновению участниками, которые не смогли наладить диалог с администрацией основного проекта. Это создавало путаницу для новопришедших людей.

В 2020 году инструмент создания вики был переработан: теперь он автоматически блокирует создание дубликата, если находит вики на данную тематику.

10.2.2. Mediawiki

MediaWiki — это распространенная wiki-система. Документы в wiki хранятся в виде плоского текста с использованием некоторого языка разметки. Текст можно править «по месту» в процессе чтения материала и немедленно публиковать. Следующий момент — облегченная схема построения гиперссылок. В wiki-системе соблюдается цепочка Идентификаторы=Названия=Заголовки и используется адаптивная линковка, заключающаяся в перенаправлениях и даже «опережающих» ссылках на несуществующие еще статьи. Наконец, wiki-система обеспечивает централизованное хранение, что для большинства пользователей удобнее работы со множеством файлов размеченной (HTML, XML, LaTeX, SGML) документации. Не требуется одновременно знать и файловую структуру проекта, и идентификаторы разделов (для ссылок) и иметь систему синхронизации изменений от различных пользователей. Совместное редактирование влечет совместную ответственность за результат, вырабатывается культура обсуждений и поиска правильного решения, легкость редактирования ведет к многократным итерациям, что улучшает качество текста. Кроме того, легкость порождения статей

способствует фиксации больших объемов знаний. В wiki-системе на основе MediaWiki не дублируются знания, доступные в Сети, а содержит исключительно внутреннюю уникальную информацию: описания внутренних технологий, регламентов, инструкций, FAQ-списки, библиотеку, личные странички сотрудников с квалификационными профилями и т.п.

10.3. Задание

1. Создать документацию разработчика при помощи любого инструмента для генерации документации кода.
2. Создать документацию пользователя при помощи FANDOM или MediaWiki (можно предложить что-то свое или сделать вики на GitHub).

11. ПОДГОТОВКА К ТЕСТИРОВАНИЮ ПРОДУКТА

11.1. Тестирование программного продукта

Все более распространенным становится заблуждение, согласно которому тестировщики занимаются тем, что просто нажимают на кнопки и вводят рандомную информацию в разные поля программы. На самом деле это не так, если бы тестировщики хаотично нажимали на кнопки и вводили случайные данные, то результаты тестирования никакой ценности для разработчика не принесли бы. Результаты представляли бы собой неструктурированную информацию, из которой невозможно получить представление о том, насколько качественным получился продукт и насколько удобен он для пользователей. У тестировщиков всегда есть стратегия работы, план, который позволяет получить объективное описание актуального состояния продукта.

Также считается, что тестировщики ответственны за качество ПО. На самом деле, ответственность за качество разработки продукта несет вся команда. Тестировщики же помогают улучшать качество разработки, а также выявляют проблемы на ранних стадиях.

Тестирование программного обеспечения является неотъемлемой частью создания программного продукта. От того, насколько досконально проведены тесты, зависит то, как скоро проект будет сдан окончательно, и будет ли необходимость впоследствии устранять ошибки. Тестирование программного продукта на разных стадиях создания — залог качественного выполнения заказа.

11.2. Тестовый случай или тестовый сценарий (test case)

Тестовый случай или тестовый сценарий (test case) — артефакт, описывающий совокупность этапов, конкретных условий и параметров, необходимых для проверки реализации тестируемой функции или её части.

У стандартного тестового случая есть 5 частей, то есть 5 атрибутов (рис. 11.1):

1. **Порядковый номер тестового сценария**
2. **Название тестового сценария.** Из него должно быть понятно, в чем суть тест кейса.
3. **Предусловия тестового сценария.** Это условия, которые необходимы для проведения тест кейса. Они должны быть выполнены еще до запуска тест кейса. Допустим: компания сдает самокаты в поминутную аренду. Нужно провести тест кейс функции, которая уведомляет пользователя о том, что заряд аккумулятора самоката. Предусловием тест кейса будет то, что самокат должен находиться в состоянии аренды.

4. **Порядок действий** в тест кейсе и описания действий в тестовом сценарии.

5. **Ожидаемый результат тестового сценария**

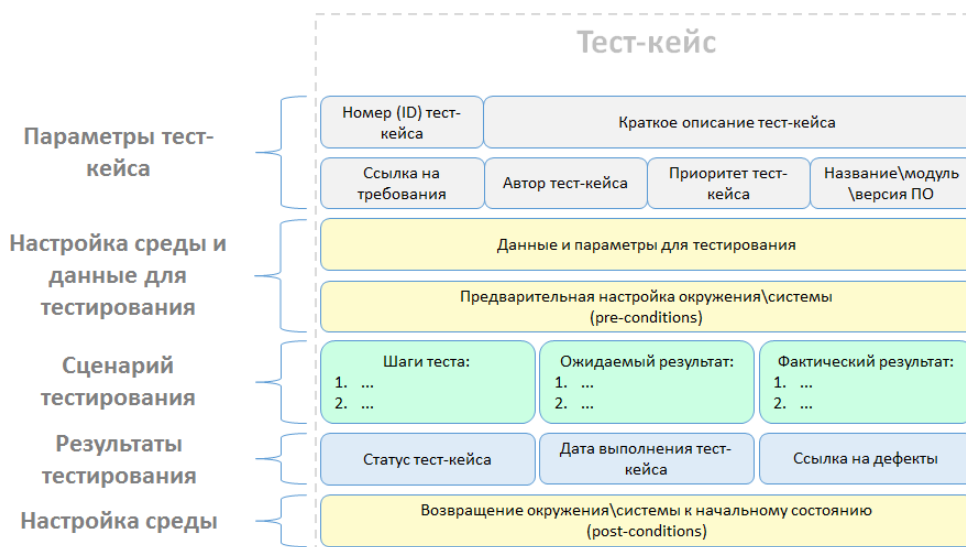


Рисунок 11.1. Атрибуты тестового сценария

Пример заполненного тестового сценария представлен в табл. 11.1.

Таблица 11.1. Пример заполненного тестового сценария

Название:	Кейс – прохождение тренажера	
Функция:	Прохождение теста, основанного на разработанной методологии “Компания”	
Действие	Ожидаемый результат	Результат теста: · пройден · провален · заблокирован
Предусловие:		
Авторизация в ЛК	ЛК – открыт и функционирует согласно с ТЗ	пройден
Шаги теста:		
Запустить тренажер	Выводится окно приветствия	пройден
Нажать на “Начать”	Выводится окно “Приветствие”	пройден
Нажать кнопку “Начать”	Выводится тестовое задание	пройден
Выбрать ответ	Ответ подсвечивается, а кнопка “Далее” становится активной	пройден
Нажать “Далее”	Выводится следующий вопрос с оформлением	пройден
Завершить тест	Сообщается о прохождении и высылаются результаты на почту	провален
Нажать на кнопку “Вернуться в ЛК”	Переход на страницу с тренажерами	пройден

Тестовые сценарии разделяются по ожидаемому результату на **позитивные** и **негативные**:

- **позитивный тест кейс** использует только корректные данные и проверяет, что приложение правильно выполнило вызываемую функцию;
- **негативный тест кейс** оперирует как корректными, так и некорректными данными (минимум 1 некорректный параметр) и ставит целью проверку исключительных ситуаций (срабатывание валидаторов), а также проверяет, что вызываемая приложением функция не выполняется при срабатывании валидатора.

11.3. Этапы тестирования

В работе важно соблюдать все этапы тестирования, чтобы выпустить качественный и полностью функционирующий продукт. Этапы тестирования программного обеспечения включают:

1. **Проектирование тестирования:** Определение цели и объема тестирования, разработка тестового плана, определение ресурсов, формирование расписания и списка требований для тестирования.
2. **Анализ требований и создание тестовых случаев:** Анализ требований к ПО и разработка тестовых случаев. Тестовые случаи описывают шаги, выполняющиеся для проверки функциональных и нефункциональных требований.
3. **Подготовка тестового окружения:** Создание необходимых условий для проведения тестирования, включая установку ПО, настройку тестовых данных, тестовых средств и инструментов. (Виды тестовых сред: среда разработчика, среда тестирования, интеграционная среда, предпрод и продакшн среда)
4. **Выполнение тестов:** Проведение тестирования согласно разработанным тестовым случаям. Формирование чек-листа на основе выполнения тестирования.
5. **Регистрация и отслеживание поломок:** При выявлении ошибок происходит формирование дефектного отчета. В отчете указывается описание проблемы, шаги для воспроизведения, приоритет и серьезность. Дефекты отслеживаются и передаются разработчикам для исправления.
6. **Анализ результатов тестирования:** Оценка выполнения требований как функциональных, так и нефункциональных, обнаружения дефектов, покрытия тестами и эффективность тестирования.
7. **Завершение и отчетность.** Формирование отчета о выполненном тестировании, включающим информацию о проведенных тестах, обнаруженных

дефектах, выполнении требований и других важных аспектах и передача отчета заинтересованным сторонам, например руководителю проекта или заказчику.

11.4. Типы тестирования

Существует большое количество различных типов тестирования, которые представлены на рисунке 11.2.

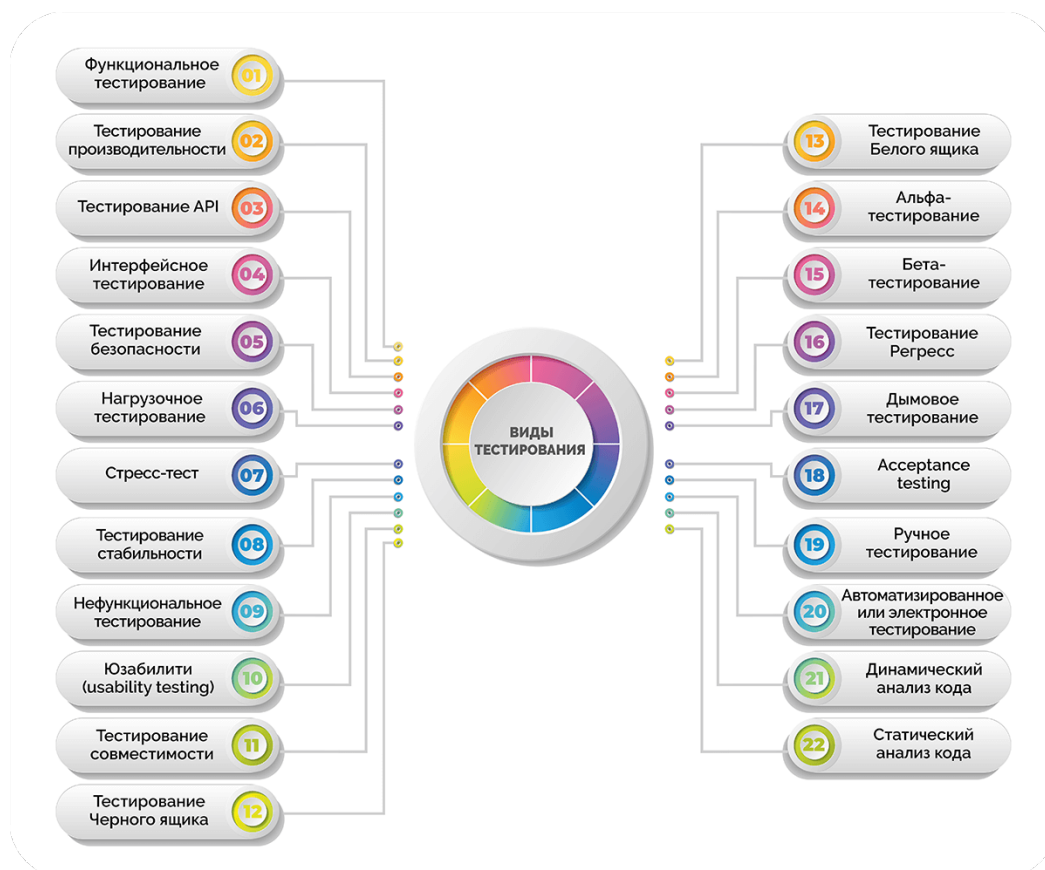


Рисунок 11.2. Виды тестирования

Для тестирования важно различать уровни тестирования, то есть различные ступени или подходы к тестированию ПО, которые обычно выполняются последовательно. Основные уровни тестирования:

1. Модульное тестирование: Проверяются отдельные модули или компонентное наполнение ПО на корректность их функционирования. Тестовые случаи проводятся над отдельными функциями, процедурами или классами. (*Примечание:* в данном случае тестовый случай состоит из одного шага, то есть самой функции (модуля), который подвергается проверки).

2. Интеграционное тестирование: Проверяется взаимодействие и взаимосвязь между различными модулями или компонентами ПО.

3. Системное тестирование: Проводится на интегрированной системе в целом. Проходит проверка функциональности, производительности и поведения системы в соответствии с требованиями и ожиданиями.

4. **Приемочное тестирование:** Проверка соответствия ПО конечным требованиям заказчика или пользователя. Для подтверждения готовности продукта к использованию от заказчика.

5. **Регрессионное тестирование:** Тестирование после внесения изменений в готовое ПО для обнаружения новых дефектов или нежелательных побочных эффектов, возникших в результате внесенных изменений.

6. **Альфа- и бета-тестирование:** Тестирование перед релизом продукта и выполняются в контролируемой среде разработчика (альфа-тестирование) и в реальной среде пользователя (бета-тестирование).

11.5. Check-list (контрольный список задач)

Check-list (контрольный список задач) – список тестовых проверок, которые необходимо выполнить в тестируемой системе. Простыми словами — это перечисление тест-кейсов без деталей (без последовательных шагов и их результатов), главная цель которого запомнить, что мы проверяем и не дать уйти с намеченного пути. Также чек-лист может выступать в роли отчета по проведенному тестированию.

Основные характеристики чек-листа:

1. **Структурированность:** Чек-лист должен состоят из упорядоченных тестовых случаев (как для функциональных, так и для нефункциональных требований), которые необходимо выполнить для полной проверки программного продукта.

2. **Полнота и покрытие:** Чек-лист включает в себя все необходимые аспекты, функции или случаи использования, которые требуется протестировать, чтобы обеспечить высокое качество продукта.

3. **Конкретность:** Каждый элемент в чек-листе должен быть конкретным и понятным, чтобы тестировщик мог точно выполнить задачу или проверить критерий.

4. **Отслеживаемость:** Чек-лист позволяет отслеживать выполнение задач и контролировать прогресс тестирования.

5. **Адаптируемость:** Чек-лист может быть адаптирован под конкретные потребности проекта, включая новые функции, изменения в требованиях или особенности окружения.

Обычно чек-лист включает в себя следующие столбцы:

1. **№:** Этот столбец предназначен для нумерации конкретных тестовых сценариев. Нумерация помогает легко идентифицировать каждый тест и отслеживать его выполнение.

2. **Наименование тестового случая:** В этом столбце указывается краткое и информативное наименование каждого тестового случая. Наименование должно четко отражать то, что будет тестироваться или проверяться в рамках данного теста.

3. **Действия/Входные данные:** В этом столбце указываются действия, которые необходимо выполнить, либо входные данные, которые необходимо использовать для проведения теста. Это может включать в себя ввод определенных данных, выполнение конкретных действий в системе, навигацию по интерфейсу и т.д.

4. **Ожидаемый результат:** Здесь указывается ожидаемый исход тестового сценария или шага. Это то, что должно произойти, если функциональность работает корректно.

5. **Фактический результат:** В этом столбце регистрируется реальный результат выполнения теста. Тестировщик должен записать фактический результат и сравнить его с ожидаемым результатом.

6. **Примечание:** Этот столбец предоставляет возможность добавить дополнительную информацию или комментарии по результатам тестирования. Здесь можно указать, например, какие-то особенности или проблемы, которые были замечены в процессе выполнения теста, требования к конфигурации окружения, использованные тестовые данные и так далее.

Примеры того, что можно добавить в примечание:

- Описание окружения, на котором проводилось тестирование (например, версии операционных систем, браузеров, разрешение экрана и т.д.).
- Ссылки на баг-репорты в системе управления ошибками.
- Список использованных тестовых данных и тестовых случаев.
- Краткие инструкции по воспроизведению найденных проблем.
- Список выполненных действий перед тестированием (например, очистка кэша, обновление страницы и т.д.).

Пример чек-листа представлен в табл. 11.2.

Таблица 11.2. Пример чек-листа тестирования

№	Наименование тестового случая	Действия или Входные данные	Ожидаемый результат	Фактический результат	Примечание
1	Вход в систему (позитивный)	Логин: user@example.com, Пароль: 123456	Успешный вход в систему	Успешный вход в систему	Применение инструментов автоматизированного тестирования веб-интерфейса, например Selenium. Создание и запуск модульных тестов, написанных с использованием подходящего фреймворка.

2	Вход в систему (негативный)	Логин: user@example.com, Пароль: неправильный	Вывод сообщения «Ошибка аутентификации»	Выведено сообщение «Ошибка аутентификации»	Применение инструментов автоматизированного тестирования веб-интерфейса, например Selenium.
3	Оформление заказа (позитивный)	Выбранный товар, Адрес: ул. Пушкина, 10, Способ оплаты: Кредитная карта	Заказ успешно оформлен	Заказ успешно оформлен	Создание и запуск модульных тестов, проверяющих корректность адреса доставки, наличие товара в корзине, есть способ оплаты, корректность суммы заказа. Выполнение ручного тестирования для веб-интерфейса.
4	Оформление заказа (негативный)	Пустая корзина	Ошибка: корзина пуста. Выведено сообщение/изменение интерфейса для пользователя.	Ошибка: корзина пуста. Выведено сообщение/изменение интерфейса для пользователя.	Выполнение автоматизированного тестирования веб-интерфейса системы, например Selenium IDE.
5	Фильтрация товаров (позитивный)	Категория: Электроника, Цена: от \$100 до \$500	Товары отфильтрованы в соответствии с выбранными критериями	Товары отфильтрованы в соответствии с выбранными критериями	Проведение авторизованного/ручного тестирования, проверяющего корректность фильтрации данных относительно запроса, например модульное тестирование, интеграционное и т.д.
6	Время доступа к системе	Вход в систему разных устройств и мест	Доступ к системе в течение X секунд	Результаты измерений времени доступа	Применение следующих инструментов, таких как Apache JMeter, LoadRunner, Gatling, Locust и др.
7	Отказоустойчивость	Нагрузочное тестирование, имитация сбоев	Система выдерживает нагрузку и сбои без потери функциональности	Результаты нагрузочного тестирования	Проведение нагрузочного тестирования и имитации сбоев при использовании специальных инструментов (Apache JMeter, LoadRunner, Gatling, Locust и др.)

8	Безопасность	Тестирование на проникновение, проверка стандартов безопасности	Система защищена от угроз безопасности соответствует стандартам	Результаты тестирования на проникновение и проверки безопасности	Применение автоматизированных инструментов для обнаружения уязвимостей (OWASP ZAP или Burp Suite). Проведение ручного тестирования специалистом по информационной безопасности.
9	Масштабируемость	Добавление большого количества товаров/пользователей	Система масштабируется без потери производительности	Результаты тестирования масштабируемости	Результаты тестирования масштабируемости должны включать данные о производительности системы при различных объемах данных или нагрузке, а также оценку ее способности масштабироваться без потери производительности. Применение инструментов Apache JMeter или Locust.
10	Совместимость	Проверка работоспособности на различных платформах и устройствах	Система корректно работает на различных платформах и устройствах	Результаты тестирования совместимости	Применение инструментов таких как BrowserStack, CrossBrowserTesting или Appium. Проведение ручного тестирования в виде запуска системы в различных браузерах, ОС.

Примечание: в чек-листе необходимо добавить тестовые случаи для всех функциональных и нефункциональных требований, сформированных в первом блоке заданий.

11.6. Дополнение матрицы требований

После того, как будет проведено тестирование необходимо занести итоговые результаты в матрицу требований (табл. 11.3). В матрице требований добавляется столбец “Результат тестирования”.

Если в ходе разработки изменились функциональные или нефункциональные требования, то необходимо внести изменения в матрицу требований.

Таблица 11.3. Дополненная матрица требований

№	Требование	Суть	Критерий проверки	Компоненты архитектуры	Результат тестирования
1	Мобильное приложение клиента				
1.1	Регистрация пользователя	Приложение должно иметь функцию регистрации нового пользователя	Регистрация нового пользователя	Ввод данных реализован на устройстве клиента, обработка происходит на сервере, хранение в единой базе данных	Пользователь зарегистрирован успешно (Требование выполнено)
...	...	...	...	...	...

Примечание: для внесения большой матрицы требований (со всеми дополнениями) в отчет, лучше будет перевернуть страницу в альбомный формат.

11.7. Задание

1. Создать не менее 10 тестовых случаев и оформить согласно таблице 11.1.
2. Описать выбранные инструменты для проведения тестирования.
3. Провести тестирование всех функциональных и нефункциональных требований, сформированных в первом блоке.
4. Результаты проведенного тестирования оформить в виде чек-листа согласно таблице 11.2.
5. Дополнить матрицу требований столбцом “Результат тестирования” и заполнить данными.

11.8. Список литературы о проведении тестирования

1. <https://habr.com/ru/companies/itsumma/articles/682022/>
2. <https://www.intervolga.ru/blog/support/test-surka-testiruem-s-selenium-ide/>
3. <https://habr.com/ru/articles/456090/>
4. <https://habr.com/ru/companies/eurostudio/articles/109010/>
5. <https://habr.com/ru/companies/otus/articles/790780/>
6. <https://hackr.io/blog/top-10-open-source-security-testing-tools-for-web-applications>
7. <https://habr.com/ru/companies/simbirsoft/articles/566862/>

12. ФОРМИРОВАНИЕ ПРОГРАММЫ И МЕТОДИКИ ИСПЫТАНИЙ СИСТЕМЫ

12.1. Программа и методика испытаний

После формирования технического предложения и разработки эскизного и технического проектов наступает момент, когда необходимо четко определить, как будет решаться вопрос о соответствии системы всем требованиям, описанным в проектной документации, как будет определяться степень ее надежности, а также, что самое главное, уровень соответствия системы своему назначению. Программа и методика испытаний как раз призвана решить весь перечень вопросов.

Программа и методика испытаний – это документ, входящий в комплект конструкторской документации, составляемой на автоматизированную систему или программу на стадии разработки. Программа методики испытаний призвана установить технические данные, которые подлежат проверке во время испытаний всей системы в целом или ее отдельных компонентов. Этим документом устанавливается порядок опытов и способы контроля их результатов.

Согласно межгосударственному стандарту ГОСТ 19.301-79, документ «Программа и методика испытаний» должен иметь следующую структуру:

- раздел «Объект испытаний» – название продукта, область его применения.
- раздел «Цель испытаний» – цели ПСИ, которые мы обозначили выше.
- раздел «Требования к программе» – требования, подлежащие проверке во время испытаний и перечисленные в «Техническом задании» (ТЗ) на программу. Обратите внимание, требования, которые указываются в данном разделе, должны быть продемонстрированы во время испытаний (или хотя бы запланированы). Если в продукте реализовано 100% требований, но продемонстрировать по тем или иным причинам вы можете только 70%, значит, в этот раздел включаются только 70% требований, которые будут продемонстрированы заказчику. Оставшиеся 30% заказчик будет считать нереализованными до тех пор, пока не убедится в обратном.
- раздел «Требования к программной документации» – состав программной документации, предъявляемой на испытании, а также специальные требования к документации, если они заданы в ТЗ.
- раздел «Средства и порядок испытаний» – перечень технических и программных средств, используемых во время испытаний, а также порядок проведения испытаний. Рекомендуем максимально подробно описать все

средства, которые вы планируете использовать для демонстрации. Базы данных, программы для чтения содержимого очередей, консоли, логи, автотесты – всё это должно быть зафиксировано в данном разделе, иначе засчитано и допущено к демонстрации не будет.

- раздел «Методы испытаний» – указывается, как именно проводится демонстрация, какие методы используются для проверки реализации того или иного требования и ожидаемый результат по каждой проверке.

ГОСТ 34.603-92 "Программная документация. Программа и методика испытаний. Требования к содержанию и оформлению" устанавливает требования к содержанию и оформлению программной документации, включая программу и методику испытаний. Согласно ГОСТ 34.603-92, который описывает программную документацию, основные разделы документа могут быть организованы следующим образом:

1. Общие положения: Этот раздел может содержать общие принципы и цели программы и методики испытаний программного обеспечения, а также определение терминов и область их применения.

2. Предварительные испытания:

- Автономные испытания: Здесь могут быть описаны испытания, проводимые независимо от других компонентов системы для проверки их работоспособности и соответствия требованиям.

- Комплексные испытания: Этот подраздел может включать в себя описание тестовых сценариев и процедур, которые охватывают работу системы в целом и взаимодействие ее компонентов.

3. Опытная эксплуатация: Здесь можно описать процедуры, связанные с тестированием системы на реальных условиях использования, например, в тестовой среде или на пилотных объектах.

4. Приемочные испытания: Этот раздел может содержать информацию о тестировании, проводимом для подтверждения соответствия программного обеспечения установленным требованиям заказчика или стандартам.

Для составления ПМИ необходимо воспользоваться государственными стандартами, такими как: ГОСТ 19.301-79 «ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ», ГОСТ 19.105-78 «ОБЩИЕ ТРЕБОВАНИЯ К ПРОГРАММНЫМ ДОКУМЕНТАМ», ГОСТ 34.603.92 «ВИДЫ ИСПЫТАНИЙ АВТОМАТИЗИРОВАННЫХ СИСТЕМ».

12.2. Задание

1. Составить программу и методику испытаний по вашему программному продукту в соответствии с ГОСТами.

13. АНАЛИЗ И ОЦЕНКА РИСКОВ ПРИ РАЗРАБОТКЕ ПРОГРАММНОГО ПРОЕКТА

13.1. Риски проекта

Вряд ли вам встречался проект, который волшебным образом шел в точности, как его запланировали в самом начале. Любой проект подвержен рискам — всевозможным событиям, которые на него влияют и обычно создают проектному менеджеру головную боль. Вообще, если не считать планирования, главная головная боль проектного менеджера как раз и заключается в разрешении рисков. Чтобы уменьшить ее, можно подумать об угрозах заранее. Оно того стоит — управлять рисками куда дешевле и проще, чем управлять реальными проблемами.

В широком смысле рисками реализации проекта называют условия или события, которые оказывают влияние на результат проекта. Такие влияния могут сопровождаться положительным эффектом, «нулевым» или отрицательным. В более узком значении проектные риски определяются как потенциально неблагоприятные влияния, влекущие за собой потери и убытки, поскольку связанная с рисками природа неопределенности рассматривается как элемент непредсказуемого ухудшения ситуации из-за внутренних и внешних обстоятельств.

Возможные риски проекта и реагирование на них зависят от параметров вероятности, величины рисков, значимости последствий, толерантности к риску, наличия резервов (в том числе — управленческих) на случай возникновения рисковых ситуаций.

Риски проекта проявляют эффект накопления вероятностей событий, влияющих на проект. При этом само событие может принести как выгоду, так и ущерб, иметь разную степень неопределенности, различные причины и последствия (изменение трудозатрат, финансовых затрат, сбой плана действий).

Вероятность риска — это возможность реализации угрозы в диапазоне от 0 до 100 процентов. Крайние значения рисками не считаются, поскольку нулевой предел означает невозможность наступления события, а 100-процентная гарантированность должна быть предусмотрена в проекте как факт. Событие, которое имеет очень высокую степень вероятности (например, гарантированное повышение цен поставщиком), часто вообще выносится за пределы рассмотрения в контексте темы проектных рисков. Вероятность определяется двумя типами методов:

- объективными, когда вероятность результата, полученного в аналогичных условиях, вычисляется со статистической достоверностью на

основе частоты события;

- субъективными, основанными на предположении о возможном продолжении или исходе, а само предположение здесь базируется на понимании логики процесса субъектом, принимающим решения, и его опыта, которые субъект представляет в числовом выражении.

Если информации о потенциальных издержках недостаточно (например, уже после запуска проекта произошло неожиданное изменение налогового законодательства), то по таким неизвестным рискам закладывается специальный резерв, а процедуры управления не реализовываются. Резерв для непредвиденных событий может выражаться как в дополнительной сумме, так и в дополнительном времени, и его следует вносить в базовый план проектной стоимости.

Если об изменениях можно судить заранее, то выстраивается план реагирования, направленный на минимизацию рисков. Как правило, границы управления рисками частично захватывают информационное поле, по которому нет никакой информации (полная неопределенность), и частично – поле с полной определенностью, по которому есть исчерпывающая информация. Внутри этих границ располагаются известные и неизвестные факторы, составляющие общую и специфическую неопределённость.

13.2. Планирование способов реагирования

При выборе способов реагирования ориентируются на 4 основные типа стратегии:

- уклонение (избежание) – устранение источников риска;
- страхование (передача) – привлечение третьей стороны, принимающей риски на себя;
- минимизация (снижение) – уменьшение вероятности воплощения угрозы;
- принятие – пассивная форма предполагает осознанную готовность к угрозе, а активная форма – согласование плана действия при наступлении непредвиденных, но принимаемых обстоятельств.

Каждый способ может применяться для своего типа рисков как оптимальный.

13.3. Виды и источники рисков

Говоря об управлении рисками IT проекта, важно обозначить возможные источники этих рисков. Когда мы можем распределить возможные риски по категориям, то их поиск и управление становится намного проще. Тем не менее,

категории должны отражать природу взятого проекта. В риск-менеджменте в контексте IT, можно выделить три больших вида рисков:

- внешний риск – результат влияния клиента;
- внутренний риск – результат самого процесса разработки ПО;
- персональный риск – результат усилий, качества и вовлеченности отдельных участников команды в проект.

13.3.1. Самые распространенные риски

Изменение требований и приоритетов (внешний риск).

Во время проекта происходит много изменений – концепции, требования, количество задач в спринте, приоритеты. Иногда клиент может передумать, когда задание уже наполовину выполнено – это повлечет за собой увеличение стоимости проекта.

Последствия:

- перегруженные (или недогруженные) спринты;
- брошенные незавершенные задачи;
- полная или частичная переделка приложения;
- изменения в расписании;
- незавершенные или растянутые спринты;
- внезапная необходимость добавления большего количества людей в команду.

Решение:

Привнося изменения, необходимо сначала проанализировать как это отразится на текущем состоянии проекта, сколько усилий потребует и существует ли риск задержки. Благодаря анализу, вы сможете грамотно разделить обязанности, внести изменения в приоритеты и предоставить клиенту точную информацию о том, что может (или не может) быть выполнено.

Безответная реакция в таком случае будет означать ваше безоговорочное принятие тех или иных изменений. Это распространенная практика в процессе разработки ПО. Именно поэтому очень важно всем участникам проекта осознавать последствия изменений и совместно идти на некоторые компромиссы, если они необходимы.

Недостаточная вовлеченность (персональный риск).

Вовлеченность всех членов команды является обязательным условием для успеха каждого проекта. Поэтому очень важно, чтобы каждый участник процесса был предан общей цели, понимал свою роль и поддерживал других членов команды.

Слово “команда” относится к каждому участнику проекта – клиенту, проджект-менеджерам, разработчикам, тестировщикам, дизайнерам, аналитикам.

Последствия:

- задержки сроков выполнения,
- негативное влияние на мотивацию других членов команды.

Решение:

Уделяйте внимание другим членам команды и постарайтесь понять, что может усилить их вовлеченность. Они должны быть в хорошем расположении духа, но в то же время, должны уметь фокусироваться на своей работе. Дайте им возможность личностного роста, разговаривайте с ними и хвалите. Убедитесь, что предоставляете им полноценную информацию о проектах – так они смогут почувствовать себя важной частью чего-то большего.

Нехватка коммуникации (персональный риск).

Коммуникация необходима для эффективной работы во время проекта по разработке ПО. Какие последствия влечет за собой слабая коммуникация?

Последствия:

- пробелы в знаниях;
- задвоенные задачи;
- сниженная продуктивность.

Решение:

Регулярные собрания всех членов команды ради завершения задач и обмена знаниями существуют как часть проекта. Собрания должны проходить в здоровой атмосфере, где у каждого есть шанс высказаться. Никогда не оставляйте чей-либо вопрос без ответа. Если вы не знаете, как ответить, то сообщите человеку, что вы попытаетесь найти ответ и вернетесь с ним позже.

Очень важно, чтобы каждый понимал свою роль в проекте. Встреча в нерабочей обстановке может также положительно сказаться на командном духе.

Плохая документация (внутренний + внешний риски).

Что такое проектная документация? Продукт с минимальным функционалом (MVP), описания задач в JIRA и выделенное место для проекта в Confluence – всё это очень важно для успеха проекта.

Последствия:

- хаос;
- команда, зря тратящая время на повторяющиеся вопросы касаясь базовой информации о проекте;
- отсутствие хороших контрольных показателей для использования членами команды как во время проекта, так и после него;

- недостаточные знания членов команды, присоединившихся к проекту на пол пути.

Решение:

Даже минимальная проектная документация может сыграть большую роль в предотвращении последствий.

Согласно лучшим практикам Agile: “работающее ПО важнее детальной документации”. Тем не менее, не надо считать документацию малозначительным элементом. Как же лучше решить эту проблему?

Отведите некоторое время на написание документации с самого начала. В таком случае, у вас не будет никаких отговорок. Используйте такие инструменты, как JIRA, Confluence или QA Touch – они действительно облегчают работу. Также существует много более специализированных инструментов, которые помогут вам написать документацию для ИПП и других отчетных материалов по проекту. Определите какая информация должна всегда быть доступной. Хорошее место для ее хранения – Confluence. Это система, позволяющая найти всю базовую проектную документацию, членов команды, их роли и другую важную информацию по свойствам проекта, его среды, описания пользователей и перечня функций.

Постоянно используйте специальные условные обозначения для обозначения и описания задач. Помните, документация не должна быть объемной. Ее задача – всестороннее описание проекта простым и понятным языком.

Незапланированное отсутствие члена команды (персональный риск).

Каждое незапланированное отсутствие члена команды служит основанием для беспокойства. Осенью и зимой шансы заболеть значительно увеличиваются.

Последствия:

- дезорганизация;
- задержки сроков выполнения задач;
- нехватка знаний о проекте, если данный специалист был главным членом команды (опять же, становится очевидной важность хорошей документации!);
- демотивация команды.

Решение:

Важно, чтобы все члены команды обладали одинаковыми основополагающими знаниями о проекте. В зависимости от того, как долго отсутствует сотрудник и на каком этапе находится проект, проджект-менеджер должен принять решение о необходимости замены. Новичку будет проще приступить к работе если с ним поделится информацией о проекте и предоставят документацию.

Слабая коммуникация с заказчиком (внешний риск).

Заказчик не отвечает? Вы уже пытались с ним связаться несколько раз? Скорее всего клиент в отпуске, но не удосужился оповестить других членов команды.

Последствия:

- задержки проекта по срокам из-за молчания клиента;
- демотивация других членов команды.

Решение:

Покажите клиенту как важно для вас поддерживать хорошие отношения с ним с самой первой встречи. Обозначьте какие решения всегда должны быть приняты совместно и какие – разработчиками/проджект-менеджерами самостоятельно. Когда вы отправляете email с запросом к клиенту, обозначьте почему задержка с ответом может повлечь за собой проблемы (например, трудности с оказанием услуги в срок). Если ни один из способов достучаться до клиента не работает, проджект-менеджер должен принять меры для улучшения коммуникации с заказчиком.

Невыполнение проекта в срок (персональный риск).

Изменения требований во время внедрения, плохо оцененные задачи, неудавшееся тестирование, незапланированное отсутствие, слабая коммуникация с заказчиком и т.д. Большинство этих проблем можно предотвратить при хорошем планировании. Поэтому вы должны принять во внимание эти риски с самого начала.

Последствия:

- задержки реализации проекта;
- незавершенная задача, мешающая выполнению остальных задач;
- неудовлетворенность клиента;
- плохая рабочая атмосфера.

Решение:

При планировании дэдлайнов проекта и/или спринта, примите во внимание все факторы. Проанализируйте возможные риски и проинформируйте о них клиента. Всегда распределяйте задачи с учетом количества доступных членов команды, а также их навыков, сильных и слабых сторон. Всегда отчитывайтесь о своем прогрессе и решайте проблемы во время ежедневных планерок. Это лучший способ контроля качества в разработке ПО.

При невозможности выполнения дэдлайна необходимо уведомить клиента об этом как можно скорее. Хороший способ справиться со всем – это разделить одну большую задачу на несколько маленьких. Лучше выполнить пару

небольших задач, чем вообще ничего. Для клиента это также будет показателем вовлеченности команды.

Подобные проблемы также могут встретиться в реализуемом семестровом проекте.

13.4. Риски семестрового проекта

Во-первых, необходимо описать риски в формате «причина-риск-эффект» (пример представлен в табл. 13.1).

Таблица 13.1. Пример рисков в формате «причина-риск-эффект»

#	Риск (причина-риск-эффект)
1	– Мы тратим много времени на сборку и выкладывание билдов. – Можем опоздать с разработкой, так как девелоперы будут тратить время на конфигурацию. – Потеряем время.
2	– Не можем провести тестирование проекта под Мас, так как его у нас нет. – Проблемы с Мас могут все-таки появиться. – Пострадает качество продукта.
3	– В багтрекере много тикетов, статус которых устарел. – Мы можем заниматься некритичными багами, которые отмечены как критичные, и наоборот, упустить важные. – Потеряем во времени и в качестве.
4	– Наши сервера нестабильны, и падают примерно раз в месяц. – Тестовый сервер может упасть в момент сдачи итерации. – Задержим сдачу.
5	– В работе над проектом также участвуют разработчики со стороны заказчика. – Они могут делать существенные изменения в коде и архитектуре проекта не предупреждая нас. – Потратим время на разбирательства и откаты на предыдущие версии.
6	– Заказчик часто пропадает на 2-3 дня. – Мы можем не успеть одобрить спецификации следующей фазы вовремя. – Начало разработки сдвинется на 2-3 дня.
7	– Наш тестовый сервер недостаточно мощный. – Результаты перформанс тестирования на нашем оборудовании могут серьезно отличаться от продакшна. – Потратим лишнее время на поиск и починку проблем перформанса, которые не воспроизведутся на серверах заказчика.
8	– Текущая версия продукта уже на продакшне. – Срочные изменения, приходящие со стороны конечных пользователей, могут повлиять на наши текущие планы. – Опоздаем с разработкой новой фазы.
9	– SVN проекта находится не у нас, а на стороне заказчика. – SVN сервер может упасть, и мы не сможем поднять его сами. – Задержка в разработке.
10	– Ручное тестирование проекта забирает все больше времени с каждой пройденной фазой. – Со временем мы можем прийти к тому, что будем тратить недели ручного труда на регресс тестирование. – Задержки в сдаче проекта, потенциальные проблемы с качеством.

После этого необходимо провести оценку **Вероятности (1-10)** и **Силы последствий (1-10)** для каждого риска, и на основании этих показателей посчитать **Важность (Вероятность * Последствия)** каждого из выявленных рисков (пример представлен в табл. 13.2).

Таблица 13.2. Пример таблицы для определения важности рисков»

#	Риск (причина-риск-эффект)	Вероятность (1-10)	Последствия (1-10)	Важность (В*П)
7	– Наш тестовый сервер недостаточно мощный. – Результаты перформанс тестирования на нашем оборудовании могут серьезно отличаться от продакшна. – Потратим лишнее время на поиск и починку проблем перформанса, которые не воспроизведутся на серверах заказчика.	9	9	81
8	– Текущая версия продукта уже на продакшне. – Срочные изменения, приходящие со стороны конечных пользователей, могут повлиять на наши текущие планы. – Опоздаем с разработкой новой фазы.	9	8	72
10	– Ручное тестирование проекта забирает все больше времени с каждой пройденной фазой. – Со временем мы можем прийти к тому, что будем тратить недели ручного труда на регрессию тестирование. – Задержки в сдаче проекта, потенциальные проблемы с качеством.	8	9	72
5	– В работе над проектом также участвуют разработчики со стороны заказчика. – Они могут делать существенные изменения в коде и архитектуре проекта не предупреждая нас. – Потратим время на разбирательства и откаты на предыдущие версии.	8	8	64
1	– Мы тратим много времени на сборку и выкладывание билдов. – Можем опоздать с разработкой, так как девелоперы будут тратить время на конфигурацию. – Потеряем время.	7	8	56
6	– Заказчик часто пропадает на 2-3 дня. – Мы можем не успеть одобрить спецификации следующей фазы вовремя. – Начало разработки сдвинется на 2-3 дня.	8	7	56

Для повышения наглядности того, как сильно каждый из рисков может повлиять на проект, необходимо вписать номера рисков из табл. 13.1 в **матрицу рисков** (табл. 13.3).

Таблица 13.3. Пример матрицы рисков

ВЕРОЯТНОСТЬ	УРОВЕНЬ УЩЕРБА				
	Несущественные (1-2)	Низкие (3-4)	Средние (5-6)	Существенные (7-8)	Катастрофические (9-10)
Весьма вероятно (9-10)				8	7
Вероятно (7-8)				1, 5, 6	10
Возможно (5-6)					
Маловероятно (3-4)					
Крайне маловероятно (1-2)					

После составления матрицы рисков необходимо описать **планы реагирования** (табл. 13.4) на самые значимые из выявленных рисков. Следует описать основной план управления каждым из рисков и, при возможности, дополнительный план, реализуемый при невозможности выполнения основного. На основании предложенных планов реагирования необходимо определить случится ли каждый из рисков или нет.

Таблица 13.4. Пример таблицы планов реагирования

#	Риск (причина-риск-эффект)	Стратегия	Основной план	Отходной план
7	– Наш тестовый сервер недостаточно мощный. – Результаты перформанс тестирования на нашем оборудовании могут серьезно отличаться от продакшна. – Потратим лишнее время на поиск и починку проблем перформанса, которые не воспроизведутся на серверах заказчика.	Mitigate	Организовать перформанс-тестирование удаленно на серверах заказчика.	Тестировать на своем оборудовании, но не исправлять проблемы до тех пор, пока их не утвердит заказчик.
8	– Текущая версия продукта уже на продакшне. – Срочные изменения, приходящие со стороны конечных пользователей, могут повлиять на наши текущие планы. – Опоздаем с разработкой новой фазы.	Ассерт	Вряд ли тут что-то можно поделывать, поэтому просто заложим в план 5 дней резерва.	–

Продолжение Табл. 13.4.

#	Риск (причина-риск-эффект)	Стратегия	Основной план	Отходной план
10	– Ручное тестирование проекта забирает все больше времени с каждой пройденной фазой. – Со временем мы можем прийти к тому, что будем тратить недели ручного труда на регрессионное тестирование. – Задержки в сдаче проекта, потенциальные проблемы с качеством.	Mitigate	Добавить в план задачу на исследование инструмента автотестирования и внедрить его в наши процессы.	Сделать несколько ревью наших тест кейсов, с целью уменьшить их объем, не потеряв в покрытии.
5	– В работе над проектом также участвуют разработчики со стороны заказчика. – Они могут делать существенные изменения в коде и архитектуре проекта не предупреждая нас. – Потратим время на разбирательства и откаты на предыдущие версии.	Transfer	Оповестить заказчика, что ответственность за любые задержки, причиненные действиями его разработчиков, лежит на нем.	–
1	– Мы тратим много времени на сборку и выкладывание билдов. – Можем опоздать с разработкой, так как девелоперы будут тратить время на конфигурацию. – Потеряем время.	Mitigate	Добавить в план задачу на полную интеграцию нашего проекта с автосборкой и разверткой билдов.	Максимально формализовать процесс сборки, написать мануал в одну страницу, по которому собрать и выложить билд сможет любой.
6	– Заказчик часто пропадает на 2-3 дня. – Мы можем не успеть одобрить спецификации следующей фазы вовремя. – Начало разработки сдвинется на 2-3 дня.	Transfer	Оповестить заказчика, что ответственность за любые задержки, причиненные действиями его разработчиков, лежит на нем.	–

13.5. Задание

1. Описать риски в формате «причина-риск-эффект», провести оценку вероятности и силы последствий для всех рисков (самих рисков должно быть не менее 10).
2. Составить матрицу рисков.

3. Дополнить таблицу “причина-риск-эффект” планами реагирования (основными и дополнительными) на выявленные риски.

14. АНАЛИЗ ЭКСПЛУАТАЦИИ ПРОЕКТА

14.1. Disaster Recovery

Disaster Recovery (DR) — катастрофоустойчивость или аварийное восстановление, инструмент реагирования на критические сбои в ИТ-инфраструктуре компании, направленный на быстрое восстановление работы всех систем. То есть отказ серверной или даже всего ЦОДа не должен надолго (в отдельных случаях речь о секундах) прерывать бизнес-процессы.

Реализация DR основана на управлении рисками (анализ, оценка, нужен ли вообще план восстановления, планирование). Один из вариантов — дублирующая инфраструктура, способная взять на себя всю нагрузку, если основная система откажет. В идеале ключевые узлы инфраструктуры должны работать по принципу георезервирования:

- *георезерв инфраструктуры* — запасную инфраструктуру разворачивают в другой геолокации, не подверженной или менее подверженной аналогичным природным или техногенным ЧС. Да, от взрыва Солнца даже ЦОД на Марсе не спасёт, но уберечься от наводнения, развернув дублирующую инфраструктуру в разных городах, получится;
- *георезерв электросети* — зачастую используют другие магистральные каналы. Например, выходит из строя местная электростанция, основной ЦОД переходит на дизельные генераторы, а запасной ЦОД продолжит питаться от ГЭС. Так как второй вариант дешевле, можно остановить работу первого ЦОДа, пока аварийные службы восстанавливают электропитание. Такой подход также используют и в рамках одного ЦОДа;
- *георезерв интернет-каналов* — другие магистральные интернет-провайдеры, каналы которых не проходят в одном месте. Если экскаватор пробьет один из каналов, когда будет копать рядом с ЦОДом, это не остановит работу.

14.2. Disaster Recovery Plan

Всем без исключения нужен план аварийного восстановления **DRP (Disaster Recovery Plan)**. Его составлением нужно заниматься комплексно: со стороны бизнеса и со стороны ИТ-инфраструктуры и специалистов. Даже малому бизнесу с одним сервером неплохо бы иметь план на салфетке, что делать, когда тот выйдет из строя, чтобы минимизировать финансовые потери.

Пункты, которые нужно проработать при составлении DRP:

- состав штатной DR-команды и уровни доступа у сотрудников;
- предварительная оценка рисков;
- определение критически важных бизнес-процессов и данных;
- бюджет;
- необходимость в подрядчиках и поставщиках (услуг в том числе), а также их надежность;
- используемые технологии;
- необходимые уровни доступности;
- нужен ли SLA (Service-Level Agreement);
- быстрое восстановление или параллельная инфраструктура.

1. Состав штатной DR-команды.

Во-первых, кто-то должен спланировать, утвердить, реализовать и обслуживать инфраструктуру DR. Во-вторых, должен быть регламент конкретных действий каждого сотрудника, чтобы как можно быстрее начать процедуру аварийного восстановления. Полностью автоматизировать весь процесс невозможно, поэтому нужно выделить и/или нанять сотрудников под эти задачи. Если компания и инфраструктура небольшая, то хватит одного человека, если речь о дата-центрах Amazon, то нужна целая команда или департамент.

При этом нужно чётко определить необходимые уровни доступа для каждого специалиста, чтобы не случилось сюрпризов, когда у члена DR-команды нет доступа к северу или ЦОДу. Или, наоборот, когда новый член команды будет иметь слишком много прав.

2. Предварительная оценка рисков.

Оценку рисков можно провести самостоятельно — силами штатных сотрудников, исходя из собственного опыта. Или можно отдать эту задачу на аутсорс. Методик много, универсальной модели не существует, но есть общие фундаментальные принципы, от которых нужно отталкиваться.

Внешние риски

Этот вид рисков требует особого внимания и анализа, так как включает в себя политическую, экономическую, экологическую и другие повестки. Всё — от наводнения до тестирования автономного рунета и санкций — может повлиять на ваши бизнес-процессы.

Другие риски

Обстоятельства непреодолимой силы не получится противостоять, но вы сможете подготовиться к ним. К счастью, некоторые риски уже проанализированы и оценены. Нужно только правильно воспользоваться

общедоступной информацией.

Внутренние риски

С определением внутренних рисков ситуация обстоит одновременно проще и сложнее, так как они напрямую связаны с деятельностью компании. С одной стороны — ими легче управлять и легче предвидеть, а с другой — их может быть значительно больше, а чтобы провести точный анализ, нужно много данных и времени. Например:

Риски человеческого фактора.

Невозможно на 100% процентов быть уверенным в ком-то. Человек может недобросовестно работать, устать, допустить ошибку, обидеться, внезапно уволиться, заболеть, взять отгул по семейным обстоятельствам, переоценить/недооценить свои возможности и т.д. Поэтому человеческий фактор всегда стоит закладывать в оценку рисков. Это не значит, что у вас будут точные цифры. Скорее вы будете готовы к тому, что даже бульдозер могут перевернуть на ровном месте, а значит было бы неплохо иметь второй.

Технические риски.

Если составить технические регламенты для сотрудников и строго следовать им, то можно минимизировать риск ошибок. Однако всё равно остаётся вероятность, что ошибку допустят на этапе написания инструкции, или сотрудник, читающий её, что-то не так поймёт. Это может быть опасно как для оборудования, так и для сотрудников. Поэтому нужно со вниманием отнестись к соблюдению регламентов, инструкций, техники безопасности и т.д., а также учесть, что и они могут быть не идеальны и требуют доработок со временем.

Технологические риски.

Технологические риски связаны с типом используемого оборудования, ПО, интенсивностью нагрузок т.д. Стоит учесть, что продукция крупных вендоров-пионеров заслуживает бóльшего доверия: она проверена временем, оснащена проприетарными разработками, да и технологии тестирования и производства более совершенны, так как на отладку всех процессов ушли десятилетия.

Оценивание внутренних рисков

Вряд ли вам удастся с точностью до процента определить, какова вероятность возникновения человеческого фактора; насколько сервер Dell или HPE будет отличаться надежностью от серверов Atos, NEC или Supermicro; как часто выходит из строя оборудование, обслуживаемое по регламенту и без. Разве что вы целенаправленно не собирали статистику лет 5-10 по всем направлениям.

В противном случае вы можете поискать общедоступную информацию в смежных или аналогичных областях, а все точки отказа, которые кажутся

наиболее уязвимыми, защитить. Что касается IT-инфраструктуры, Disaster Recovery закрывает множество вопросов — спокойный сон в их числе.

3. Определите критически важные бизнес-процессы и уровень доступности для них.

Чтобы IT-инфраструктура не стоила чудовищных денег, нужно расставить приоритеты. Если взять критерий непрерывной работы, то бизнес-процессам можно присвоить уровни важности: высокий, средний или низкий:

- *высокий*. Даже несколько минут простоя могут привести к большим и(или) необратимым потерям и негативным последствиям.
- *средний*. Бизнесу требуется быстрое восстановление, но он может пережить простой от нескольких минут до часа без существенных потерь.
- *низкий*. Это могут быть любые компании, которым больше важна сохранность данных, чем их непрерывная доступность. Простой от двух до нескольких часов не станет большой проблемой.

Когда вы закончите с приоритетностью бизнес-процессов, можно приступить к определению уровня доступности системы:

- *постоянная доступность (continuous availability)*. Если речь, например, об электронном движении капитала, то инфраструктура должна быть готова к запланированным и незапланированным простоям, а также непрерывно работать 24/7 годами.
- *непрерывный режим работы (continuous operations)*. Чуть менее доступная инфраструктура, не готовая к авариям на всех узлах, но по-прежнему работающая 24/7 годами, если не возникает сбоев.
- *высокая доступность (high availability)*. Режим работы инфраструктуры, при котором система периодически останавливается в нерабочие дни или на запланированное обслуживание с предварительным уведомлением пользователей. Режим работы определяется работой организации: 8/5, 8/7, 12/5, 12/7 и т.д.
- *обычная доступность*. Если высокой доступности и(или) сохранности данных не требуется, IT-системам присваивают статус Office Production. Инфраструктура этого уровня может не резервироваться или резервироваться на самых простых уровнях под конкретные задачи. Время восстановления от одного до нескольких дней.

4. Заключите SLA — соглашение об уровне обслуживания.

SLA (Service-Level Agreement) — это единый документ, договор между поставщиком и клиентом, в котором закрепляются конкретные услуги и их объем, ожидания сторон, их обязанности, точки контактов для решения проблем, метрики эффективности, а также время исправления инцидента (Время реакции на аварию + Время решения проблемы = Время исправления инцидента).

5. Определите RPO и RTO.

Важными параметрами аварийного восстановления являются RPO (Recovery Point Objective) и RTO (Recovery Time Objective). Требования по ним идут от бизнеса — директоров, руководителей и менеджеров, а IT-команда пытается максимально приблизиться к требуемым показателям, учитывая бюджет и реальное положение дел:

1. *RPO* — это целевая точка восстановления, которая устанавливается после определения точки безубыточности: затраты на сохранность актуальных данных не должны превышать ущерба от их потери. Если RPO равен 1-ой минуте, значит резервная копия будет обновляться каждую минуту. При этом копия будет только одна.
2. *RTO* — это целевое время восстановления. Устанавливается, чтобы понимать, за какой отрезок времени бизнес-процесс должен вернуться к полноценной работе. Если RTO установили в полчаса, значит через 30 минут система должна работать в штатном режиме — ни минутой позже.

6. Выбор конкретного решения DR.

Все предыдущие оценки позволят вам определиться, какой уровень DR подойдет под конкретный бизнес-процесс и бюджет.

1) бэкапы и быстрое восстановление из облака (DRaaS Backup & Restore).

Это компромиссный и относительно недорогой вариант, поскольку не приходится покупать, настраивать и обслуживать дублирующую инфраструктуру; стоимость такого решения минимальна, а платить нужно только за объем данных в облаке и за временное использование виртуальных машин в случае ЧС. Быстрое восстановление допускает непродолжительную остановку бизнес-процессов.

Существуют готовые решения DR в облаке:

1. Veeam Disaster Recovery Orchestrator;
2. Mail.ru Cloud DRaaS;
3. CloudMTS Disaster Recovery;
4. Множество других DRaaS сервисов.

2) *параллельная инфраструктура (Multi-Site Solution).*

Параллельно основной будет подготовлена резервная инфраструктура с репликой системы. Преднастроенные WAN-каналы, сеть и виртуальные машины со всем необходимым софтом будут ожидать своего часа.

Аналогично первому пункту можно организовать всё в облаке, но тогда стоит посчитать: в какие затраты это выльется за несколько лет. Возможно, в перспективе 5-10 лет на аренду вы потратите соизмеримую с покупкой и обслуживанием сумму, но владеть ничем не будете. Параллельная инфраструктура позволяет восстановиться очень быстро — от пары секунд до нескольких минут.

7. *Проверка работоспособности: учения и тесты failover.*

Будет обидно, если вы заранее побеспокоитесь об аварийном восстановлении, но система не запустится в критической ситуации, так как никто не проводил тесты. И важно учесть, что единоразовым начальным тестированием тут не отделаешься. Сформированная ответственная группа должна постоянно проводить учения и поддерживать актуальное состояние DR-системы, чтобы соответствовать меняющимся внешним и внутренним рискам.

Учения чаще всего проводят отдельно для каждого риска. Например, DR-команда отключает сетевой кабель и тестирует падение интернет-провайдера. Другой сценарий — оценивают влияние человеческого фактора, когда сисадмин пропал (не вышел на работу, не вернулся с обеда). Аналогично проводят и учения по ИБ (информационной безопасности), имитируя различные кибератаки и вмешательства.

14.3. Риск = Угроза + Уязвимость + Актив

Риск:

1. *Угроза* — что-то плохое.
2. *Уязвимость* — особенность актива.
3. *Актив* — любой объект.

Если совсем уж перефразировать, то можно сказать, что риск это когда что то плохое происходит из за какой то особенности чего то.

При таком расщеплении на атомы все становится на свои места, ясна связь между угрозами и рисками безопасности.

Угроза просто описывает нам что-то плохое, в то время как *Риск* еще сообщает из-за чего это плохое может произойти.

Несколько примеров рисков, сформулированных по такой конструкции:

Раскрытие ключей (паролей) доступа из-за возможности атаки SMB Relay в ОС Windows

В данном случае:

1. *Угроза* – Раскрытие ключей (паролей) доступа.
2. *Уязвимость* – Возможности атаки SMB Relay.
3. *Актив* – ОС Windows.

Боковое перемещение злоумышленника по локальной сети из-за возможности удаленного подключения через RDS Shadow в ОС Windows

Закрепление злоумышленника в ОС из-за возможности распространения скриптов через Network Logon Scripts в доменных службах Active Directory

Заражение вредоносным программным обеспечением из-за реагирования на мошеннические, фишинговые письма работником // В качестве актива выступает человек

Неработоспособность серверного оборудования из-за нарушения температурного режима в серверном помещении // классический ИТ риск на доступность активов

Недоступность ИТ/ИБ персонала из-за отсутствия квалифицированной кадровой политики в компании // Это тоже не про ИБ, скорее про кадровые и управленческие риски компании, но влияет на функционирование системы информационной безопасности. Активом выступает компания как юридическое лицо.

14.3.1. Технологические угрозы

Какие бывают технологические угрозы, например:

- отказы аппаратного и технического обеспечения (от вычислительной техники до электроэнергии);
- отказы информационного обеспечения (внешние БД, информационное обеспечение);
- ошибки в математическом и программном обеспечении (ошибки при реализации алгоритма, при развертывании новой версии софта программа перестала запускаться на старых устройствах).

14.3.2. Реагирование на риски

Архитектурные изменения – рефакторинг, выделение отдельных компонентов, проведение работ, направленных на улучшение масштабируемости, переносимости, целостности данных (MVC паттерн).

Технические изменения – привлечение сервисных провайдеров, использование готовых фреймворков и библиотек, с прогнозируемым уровнем надежности.

Организационные изменения – применение технических средств

направленных на автоматизацию ЖЦ нашего продукта или системы (код в гите, ci/cd конвейер, виртуализация, мониторинг); переход на методологию разработки наиболее эффективную в предметной области проекта на основе оценки качества продукта при помощи треугольника управления проектом – “объем работы-время-ресурсы(деньги)” (рис. 14.1).



Рисунок 14.1. Треугольник управления проектом

14.4. Задание

1. Выбрать 3 технологические угрозы и оценить их влияние на функционал из матрицы требований.

(влияние оценить по 5-ти бальной шкале:

0 - не влияет,

1 - влияет незначительно,

2 - функциональность сохраняется, но снижается ее быстроедействие, 3 - функциональность доступна только в определенные интервалы времени,

4 - функциональность частично недоступна,

5 - функциональность недоступна полностью)

2. Преобразовать технологические угрозы в риски, оценив вероятность их возникновения при эксплуатации данного проекта.

3. Предложить для каждого риска свой метод обработки:

Снижение риска, с обоснованием за счет каких изменений будет

организовываться снижение риска (архитектурных, технических и организационных изменений).

Принятие риска, с обоснованием нецелесообразности мероприятий по его снижению.

Отказ от риска, с обоснованием изменений (отказываемся от рискованного проекта или от рискованного функционала).

Передача риска, с обоснованием изменений и оценкой их возможности (привлечение третьей стороны (провайдер облака или сервиса) с разделением зон ответственности и заключением SLA).

15. ЗАЩИТА ПРОЕКТА

Реализация любого проекта завершается демонстрацией заказчику результатов проделанной работы. На данной демонстрации может приводиться обзор разработанного функционала, интерфейсов, а также демонстрироваться результаты внедрения реализованного программного продукта. На основании демонстрации складывается картина проделанной работы, помогающая определить, с какой целью реализовывался программный продукт, какой набор функций он имеет, и какой эффект может принести его использование.

Поскольку в процессе освоения дисциплины «Системная и программная инженерия» реализуется проект, данная дисциплина также завершается демонстрацией проделанной работы в виде презентации (защиты) проекта. Проект является учебным, поэтому помимо результатов разработки, а именно итогового функционала проекта, интерфейсов приложения и т.д., необходимо продемонстрировать и обосновать изначальную задумку, лежащую в основе проекта, а также описать все этапы реализации, в том числе описать реализованную документацию на проект.

Результаты выполнения завершающей практической работы должны быть представлены преподавателю-руководителю работы и студентам своей группы в виде презентации своего проекта.

В презентацию вносятся основные положения, рассмотренные в течение всего семестра, такие как: цель и задачи вашей разработки, описание проекта, какие требования предъявлялись к проекту и прочее.

Очередность выступления на защитах проектов будет определено жеребьевкой, которая будет происходить в начале пары или по порядку, определенному преподавателем. Выступление студентов с презентацией регламентируется 7 минутами, ответы на вопросы в течение 3 минут. Всего можно получить за защиту проекта 3 балла. Баллы распределяются следующим образом:

- 1 балл за качество выступления;
- 1 балл за полноту презентации;
- 1 балл за ответы на вопросы студентов и преподавателя, в случае если вопросов после выступления нет, то ставится 2 балла за полноту презентации.

Типовой структурой презентации следует считать следующий обязательный набор слайдов:

1. Титульный слайд.
2. Состав команды и роли.

3. Описание проекта.
4. Цели и требования.
5. Архитектура программного продукта.
6. Наиболее значимые риски проекта.
7. Описание стека технологий.
8. Диаграммы процессов проекта.
9. Описание функционала приложения.
10. Тестирование.
11. Развертывание (если таковое имеется).
12. Документация разработчика.
13. Документация пользователя / Post mortem.
14. Достоинства и недостатки приложения/системы (при наличии реализованного проекта).

Титульный слайд должен содержать название проекта, а также полный состав участников семестрового проекта.

На слайде «Состав команды и роли» необходимо описать состав команды вместе с описанием ролей каждого из участников. Возможно включить в выступление описание того, почему каждый из участников команды занимает именно свою роль, какими навыками обусловлен выбор этой роли.

На слайде «Описание проекта» должно быть приведено краткое описание выбранного проекта, а также его назначение – для каких задач его может использовать конечный пользователь.

На слайде «Цели и требования» необходимо описать, какую цель преследовала команда при реализации данного проекта, а также, какие требования были предъявлены к нему. Для наглядности можно включить на слайд некоторые, наиболее важные, пункты матрицы требований. Если в процессе реализации программного приложения ряд требований претерпевал изменения, необходимо также рассказать, какие требования и почему были изменены. Также, если в процессе реализации пришлось отказаться от каких-либо функциональных требований или привести новые, это необходимо отметить и кратко объяснить, почему были внесены такие изменения.

На слайде «Архитектура программного продукта» должна быть описана архитектура приложения/системы с обоснованием того, почему была выбрана именно такая архитектура, а не другая. Необходимо указать, какие затруднения могли возникнуть в случае, если проект был бы реализован на основании другой архитектуры.

На слайде «Наиболее значимые риски проекта» необходимо привести

описание самых серьезных рисков проекта, а также последствий их наступления с разработанными планами реагирования. Также для большей наглядности на слайд можно выносить часть матрицы описания рисков, а также матрицы реагирования. Если какой-то из описанных рисков действительно наступил, необходимо рассказать, была ли правильно выбрана стратегия реагирования на этот риск и получилось ли минимизировать его влияние на проект. Если же план реагирования не сработал, необходимо по возможности описать, какие действия могли в таком случае минимизировать образовавшиеся последствия наступления данного риска.

На слайде «Описание стека технологий» необходимо привести обоснование выбранных языков программирования, выбранных фреймворков, библиотек и т.д. Обосновать выбор СУБД, выбор технологий развертывания, в целом всех технологий, которые использовались при реализации программного продукта.

На слайде «Диаграммы процессов проекта» приводятся скриншоты следующих диаграмм: диаграмма вариантов использования, IDEF0 или BPMN, DFD (данные диаграммы должны быть связаны с вашим приложением, а не с процессом разработки проекта).

На слайде «Описание функционала приложения» приводится описание реализованного функционала приложения, доступного для конечного пользователя. С целью повышения наглядности на слайд могут быть вынесены скриншоты разработанных интерфейсов, демонстрирующих разработанный функционал.

На слайде «Тестирование» должен быть описан процесс тестирования разработанного приложения. Необходимо включить описание перечня применяемых методов тестирования, применялись ли при тестировании средства автоматизированного тестирования, если да, то какие. На слайд можно выносить часть разработанных сценариев тестирования, а также результаты проведенного тестирования.

На слайде «Развертывание» необходимо привести описания процесса внедрения проекта в эксплуатацию: как производилась сборка приложения (при наличии), каким образом приложение разворачивалось для использования конечным пользователем, какой сервис и по каким причинам был выбран для развертывания приложения. Если приложения не подразумевает развертывание, к примеру, мобильное приложение, возможно включить описание того, как планируется производить распространение приложения. Или же, какие действия должен совершить пользователь для запуска проекта на своей машине.

На слайде «Документация разработчика» приводится описание сгенерированной документации из комментариев к исходному коду. Также приводятся

скриншоты сгенерированных страниц документации. Даже в случае невозможности завершения проекта должна присутствовать документация хотя бы той части, которая была реализована в процессе разработки.

На слайде «Документация пользователя» должно быть приведено описание созданной Wiki на реализованный проект. Описание должно включать в себя перечень страниц разработанной Wiki. В презентацию должны быть вставлены скриншоты части страниц для возможности оценки информативности документации. Если же проект не был завершен, необходимо привести таблицу с *post mortem*, описывающую возникшие в ходе реализации проблемы. Также должны быть приведены попытки решения данных проблем в текущем проекте и предложены решения для будущих проектов, позволяющие избежать подобные проблемы.

На слайде «Достоинства и недостатки приложения/системы» необходимо привести сравнительный анализ разработанного приложения в сравнении с уже существующими аналогами. Сравнения должно осуществляться минимум по 5 параметрам, чтобы дать возможность более полно оценить плюсы и минусы разработанного решения и показать его возможные преимущества (или недостатки) в сравнении с его аналогами.

Перечисленные выше слайды являются обязательными к включению в презентацию. Если же есть необходимость отметить какие-то особенности реализованного решения, можно включить в презентацию дополнительные слайды.

В конце презентации должен присутствовать небольшой демонстрационный ролик, показывающий возможности разработанного приложения/системы. В процессе демонстрации должны быть даны комментарии к происходящему в ролике (можно их давать как в самом видеоролике, так и устно при защите проекта). Демонстрационный ролик может быть представлен в виде рекламного ролика, который показывает основные функции Вашей разработки. Видео должно быть в формате .mp4 или .mov и во временных рамках 2-3 минуты.

После завершения презентации и демонстрации команде будут задаваться вопросы по реализованному проекту. Участвовать в ответах также по возможности должна вся команда, и каждый участник должен отвечать за ту часть, которую он реализовывал в проекте. Так, к примеру, если возникнут вопросы к функционалу системы, отвечать на вопросы должен разработчик команды.

15.1. Провальный проект

В случае, если проект оказался неудачным и не может быть завершен, необходимо описать все проблемы и трудности, повлиявшие на процесс реализации проекта. Такое описание необходимо составить при помощи специальной

концепции post mortem. Данная концепция подразумевает описание возникших проблем в следующей форме: какие проблемы возникли при реализации проекта, причины их возникновения, какие попытки предпринимались для их устранения и что необходимо сделать для отсутствия этих проблем при реализации последующих проектов. Для повышения наглядности необходимо сформировать написанные post mortem'ы в таблицу (табл. 15.1):

Таблица 15.1. Пример таблицы с постмортемами

№	Проблема	Причины	Попытки устранения	Решение для последующих проектов
1	Постоянный перенос сроков выполнения задач	Неправильная оценка сроков реализации	Проведение анализа сроков в процессе разработки	Анализ сроков проекта на этапе его планирования
2	...	...	...	...

Данная концепция позволяет оценить ошибки проекта и разработать соответствующие решения этих ошибок для будущих проектов, чтобы избежать их повторения. Необходимо серьезно подойти к формированию таблицы, поэтому в результате она должна содержать в себе минимум десять возможных проблем, а также их решений для будущих проектов.

СПИСОК ИСТОЧНИКОВ И ЛИТЕРАТУРЫ

1. Батоврин В. К. Системная и программная инженерия: Словарь-справочник. - М.: ДМК Пресс, 2010. - 279 с.
2. Дешко И. П., Кряженков К. Г., Цветков В. Я. Системная и программная инженерия: учебное пособие. - М.: МАКС Пресс, 2018. - 80 с.
3. Черткова Е. А. Программная инженерия. Визуальное моделирование программных систем [Электронный ресурс]: Учебник для вузов. - Москва: Юрайт, 2021. - 147 с. – Режим доступа: <https://urait.ru/bcode/471564>
4. Баранюк В. В. Системная и программная инженерия: Методические указания по выполнению практических работ. Часть 1 [Электронный ресурс]: - Москва: РТУ МИРЭА, 2020. - 28 с. – Режим доступа: <https://e.lanbook.com/book/163906>
5. Баранюк В. В., Миронов А. Н., Крылова О. С. Системная и программная инженерия [Электронный ресурс]: Методические указания по выполнению практических работ. Ч. 1. - М.: РТУ МИРЭА, 2020. Режим доступа: <https://library.mirea.ru/secret/04122020/2454.iso>
6. Лаврищева Е. М. Программная инженерия и технологии программирования сложных систем [Электронный ресурс]: Учебник для вузов. - Москва: Юрайт, 2021. - 432 с. – Режим доступа: <https://urait.ru/bcode/470923>
7. Описание работы с основными элементами в нотации BPMN 2.0 [Электронный ресурс] / https://www.elma-bpm.ru/bpmn2/7_2.html (дата обращения: 06.03.2022)
8. Нотация IDEF0 [Электронный ресурс] / <https://www.businessstudio.ru/wiki/docs/v4/doku.php/ru/csdesign/bpmodeling/idef0> (дата обращения: 06.03.2022)
9. Построение архитектуры для игры [Электронный ресурс] / <https://radiant-escarpment-88463.herokuapp.com/ru/post/231087/> (дата обращения: 10.04.2022)
10. Инструмент для построения диаграмм [Электронный ресурс] / <https://www.yworks.com/products/yed> (дата обращения: 12.04.2022)
11. Документация при помощи Sphinx [Электронный ресурс] / <https://www.sphinx-doc.org/en/master/> (дата обращения: 14.05.2022)
12. Mediawiki [Электронный ресурс] / <http://www.mediawiki.org/wiki/MediaWiki> (дата обращения: 17.05.2022)
13. FANDOM [Электронный ресурс] / <https://www.fandom.com/> (дата обращения: 17.05.2022)

- 14.Проект Demon's work студентов ИКБО-07-18 / https://demon's-work.fandom.com/ru/wiki/Demon's_work_%D0%92%D0%B8%D0%BA%D0%B8#gallery-0 (дата обращения: 06.06.2022)
- 15.Классификация ЦОД Tier [Электронный ресурс] / https://habr.com/ru/company/cloud_mts/blog/332864/ (дата обращения: 21.06.2022)
- 16.Как разработать план аварийного восстановления [Электронный ресурс] / <https://zen.yandex.ru/media/mcs/kak-razrabotat-plan-avariinogo-vosstanovleniia-5d53e1fd8c5be800ae855347> (дата обращения: 21.06.2022)

Сведения об авторах

Миронов Антон Николаевич, старший преподаватель кафедры математического обеспечения и стандартизации информационных технологий Института информационных технологий РТУ МИРЭА

Воронцов Юрий Алексеевич, ассистент кафедры математического обеспечения и стандартизации информационных технологий Института информационных технологий РТУ МИРЭА

Михайлова Евгения Константиновна, ассистент кафедры математического обеспечения и стандартизации информационных технологий Института информационных технологий РТУ МИРЭА

Трушин Степан Михайлович, ассистент кафедры математического обеспечения и стандартизации информационных технологий Института информационных технологий РТУ МИРЭА